

MONDAY (2/MAR/2026) - C# REVISION	1
MONDAY (9/MAR/2026) - INTRODUCTION TO PYTHON	16
Group 3 Meeting	40
range 1 -15 if the number is divisable 3, output fizz if by 5 output buzz by both and then combine the two words. else output the exact number	49
syntax of a function	56
Write a function that calculates Bank Account Balance.....	58
2 parameter (deposit withdrawal)	58
Courier System delivery time distance / speed.....	61
Dictionaries	66
ADDING A NEW ITEM	66
ask for a number and decide wether its even or odd and display again if it is neg, positive or zero	69
Statistics	82
Datetime: datetime, timedelta	85
Scatter plot	91
#Scatter plot and add regression line ($y=mx+b$) y m slope, b value of y at y-intercept	92
DISPLAY NAME AND BALANCE	94
CLASS NAME : CUSTOMER 2 PARAMETER (NAME AND BALANCE).....	94
combined example instant variable for personal data Class variable for shared data	94
19 March 2026.....	94
Split data into training and testing.....	94
Train the regression model	94
evaluate the model	94
check model coefficience for interpretation.....	94
Displaying information about the dataframe, including data types and non-null values	94
Price vs. Square footage scatter plot	94
4. Build Linear Regression Model	94

X will contain all column except Price (The drop method removes the price column)	94
Model training	94
initialize the regression model. This create an instance of the model	94
Train the model using the training data	94
After training, we inspect the model's learned coefficients and intercepts $y=mx+b$	94
STEP 5: MODEL EVALUATION AND VISUALISATIONS	94
making predictions	94
Lets use the trained model to predict house prices for the test set features	94
y_pred will contain the predicted price	94
Display the first few actual and predicted prices to compare.....	94
25 March 2026	94

MONDAY (2/MAR/2026) - C# REVISION

*They say we'll mostly be using it for data structures or backend sites instead of its default coding abilities.

C# Recap Exercise:

MainWindow

CURRENCY CONVERTER

ENTER AMOUNT:

20

CONVERT FROM:

Rupees

CONVERT TO:

Rand

RESULTS

ZAR: 3,5999999999999996

CONVERT

Converts from Dollar to Dollar, Rand and Rupees.
 Converts from Rand to Rand, Dollar and Rupees.
 Converts from Rupees to Rupees, Rand and Dollar.

```
//Global variable
public MainWindow()
{
    InitializeComponent();
    string[] a = { "Rand", "Dollar", "Rupees" };
    string[] b = { "Dollar", "Rupees", "Rand" };

    cmbFrm.ItemsSource = a;
    cmbTo.ItemsSource = b;
}

double i = double.Parse(txtinput.Text);

//Rand To Other
if (cmbFrm.SelectedItem == "Rand" && cmbTo.SelectedItem == "Dollar")
{
    double resilt = i * 0.062;
    txtresult.Text = "US$: " + resilt.ToString();
}
if (cmbFrm.SelectedItem == "Rand" && cmbTo.SelectedItem == "Rupees")
{
```

```
        double resilt = i * 5.66;
        txtresult.Text = "INR: " + resilt.ToString();
    }
    if (cmbFrm.SelectedItem == "Rand" && cmbTo.SelectedItem ==
"Rand")
    {
        double resilt = i * 1;
        txtresult.Text = "ZAR: " + resilt.ToString();
    }

    //Dollar to Other
    if (cmbFrm.SelectedItem == "Dollar" && cmbTo.SelectedItem ==
"Rand")
    {
        double resilt = i * 16.16;
        txtresult.Text = "ZAR: " + resilt.ToString();
    }
    if (cmbFrm.SelectedItem == "Dollar" && cmbTo.SelectedItem ==
"Rupees")
    {
        double resilt = i * 91.48;
        txtresult.Text = "INR: " + resilt.ToString();
    }
    if (cmbFrm.SelectedItem == "Dollar" && cmbTo.SelectedItem ==
"Dollar")
    {
        double resilt = i * 1;
        txtresult.Text = "US$: " + resilt.ToString();
    }

    //Rupee To Other
    if (cmbFrm.SelectedItem == "Rupees" && cmbTo.SelectedItem ==
"Rand")
    {
        double resilt = i * 0.18;
        txtresult.Text = "ZAR: " + resilt.ToString();
    }
    if (cmbFrm.SelectedItem == "Rupees" && cmbTo.SelectedItem ==
"Dollar")
    {
        double resilt = i * 0.011;
        txtresult.Text = "US$: " + resilt.ToString();
    }
    if (cmbFrm.SelectedItem == "Rupees" && cmbTo.SelectedItem ==
"Rupees")
    {
```

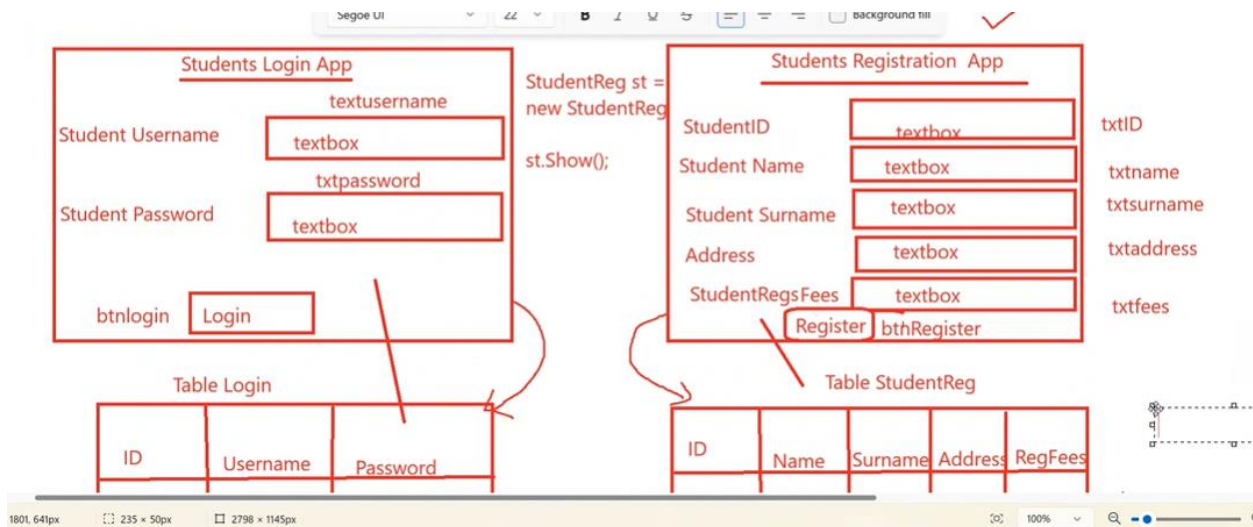


```

double resilt = i * 1;
txtresult.Text = "INR: " + resilt.ToString();
}

```

Possible Exercise with SQL integration:



The 1st database:

```

Create table StudentLogin
(
  ID int primary key not null,
  Username varchar(45) not null,
  Password varchar(45) not null
)
insert into StudentLogin
values (1, 'Francis', '123');

```

```

Create table StudentLogin
(
  ID int primary key not null,
  Username varchar(45) not null,
  Password varchar(45) not null
)
insert into StudentLogin
values (1, 'Francis', '123');

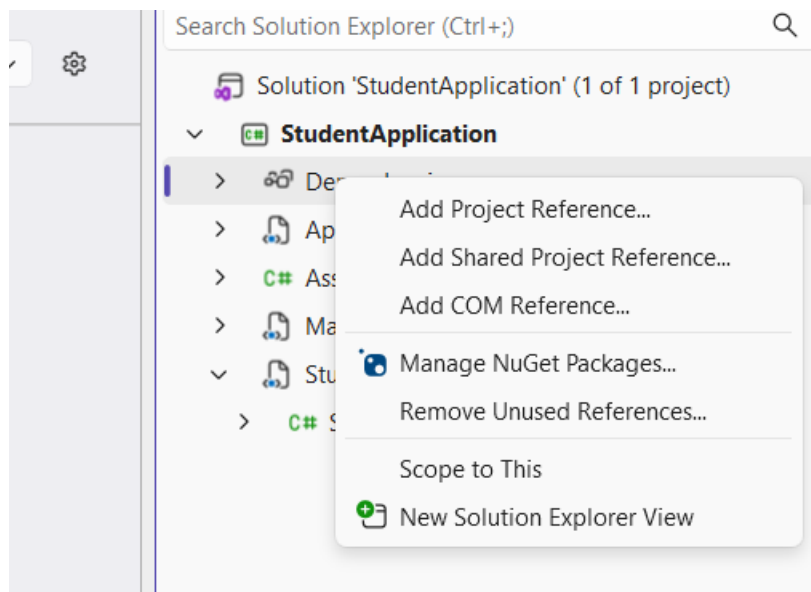
create table StudentReg
(

```

```
ID int primary key not null,  
Name varchar(56) not null,  
Surname varchar(56) not null,  
Address varchar(56) not null,  
RegFees varchar(56) not null  
)
```

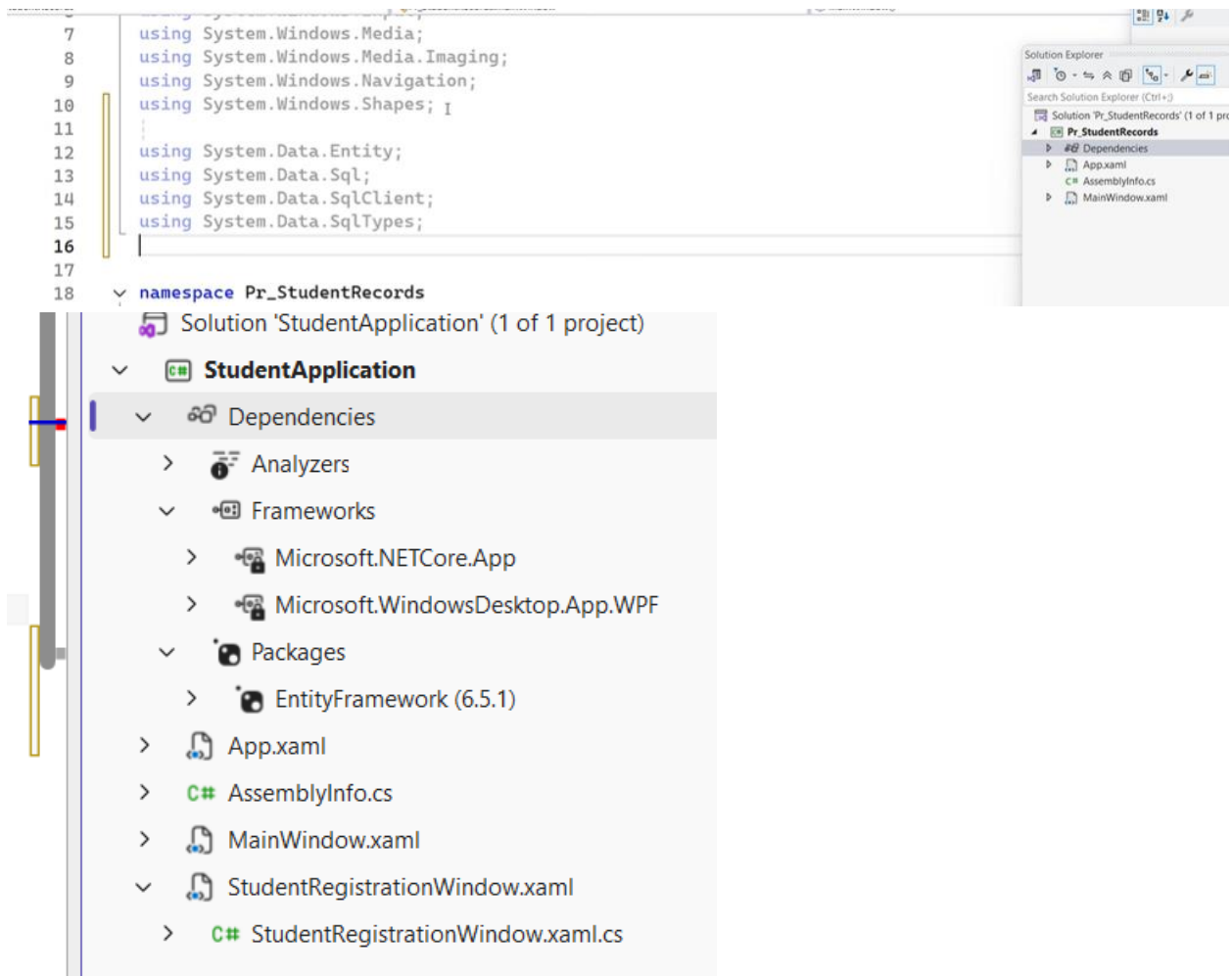
INSTALLING A FRAMEWORK THAT HAS SQL CLASS

-Project name (Dependencies)



Entity Framework Installation on the “Manage NuGet Packages”

-Then browse and search entities



```
using System.Data.Entity;  
using System.Data.Sql;  
using System.Data.SqlClient;  
using System.Data.SqlTypes;
```

*The SqlConnection is a class of the Entity Framework. This is so we have an object to connect to the database on the SQL side.

```
[SqlConnection con = new SqlConnection();]
```

1 reference

```
private void btnLogin_Click(object sender, RoutedEventArgs e)
{
    SqlConnection con = new SqlConnection();

    try{
        if(con.State == System.Data.ConnectionState.Closed)
            con.Open();
        String query = "select count (1) from StudentLogin Where Username=@Username AND Password=@Password";
        SqlCommand sqlcmd = new SqlCommand(query, con);
        sqlcmd.CommandType = System.Data.CommandType.Text;
        sqlcmd.Parameters.AddWithValue("@Username", txtUsername.Text);
        sqlcmd.Parameters.AddWithValue("@Password", txtPassword.Text);

        int count = Convert.ToInt32(sqlcmd.ExecuteScalar());

        int count = Convert.ToInt32(sqlcmd.ExecuteScalar());
        if (count == 1)
        {
            StudentRegistrationWindow st = new StudentRegistrationWindow();
            st.Show();
        }
        else
        {
            MessageBox.Show("Incorrect Username or Password");
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show(ex.Message);
    }
    finally {
        con.Close();
    }
}
```

Full Code:

```
using System.Text;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Data;
using System.Windows.Documents;
using System.Windows.Input;
using System.Windows.Media;
using System.Windows.Media.Imaging;
using System.Windows.Navigation;
using System.Windows.Shapes;

using System.Data.Entity;
using System.Data.Sql;
using System.Data.SqlClient;
using System.Data.SqlTypes;
using System.Linq.Expressions;
```

```

namespace StudentApplication
{
    /// <summary>
    /// Interaction logic for MainWindow.xaml
    /// </summary>
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void btnLogin_Click(object sender, RoutedEventArgs e)
        {
            SqlConnection con = new SqlConnection(@"Data
Source=WORKPC-MBULELO\SQLEXPRESS01;Initial
Catalog=studentAppConnect;Integrated Security=True;");

            try
            {
                if (con.State == System.Data.ConnectionState.Closed)
                    con.Open();
                String query = "select count (1) from StudentLogin
Where Username=@Username AND Password=@Password";//using a variable
in a string with the @ part
                SqlCommand sqlcmd = new SqlCommand(query, con);
                sqlcmd.CommandType = System.Data.CommandType.Text;
                sqlcmd.Parameters.AddWithValue("@Username",
txtUsername.Text);
                sqlcmd.Parameters.AddWithValue("@Password",
txtPassword.Text);

                int count =
Convert.ToInt32(sqlcmd.ExecuteScalar()); //single value is the
"Execute scalar" for exact matches, username and password
                if (count == 1) // we are equating the count variable
to 1 because the line "String query = "select count (1) from..."
says AND not OR, which means...
//they count as 1 value, which means
the "Execute scalar" command makes sure they are queried as one
Variable.
                {
                    StudentRegistrationWindow st = new
StudentRegistrationWindow();
                    st.Show();
                }
            }
        }
    }
}

```

```

        }
        else
        {
            MessageBox.Show("Incorrect Username or
Password");
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show(ex.Message);
    }
    finally {
        con.Close();
    }
}
}
}

```

Registration Part:

We create a class first named "ConnectToDBS"

```

using System.Data;
using System.Data.Sql;
using System.Data.Entity;
using System.Data.SqlClient;
using System.Data.SqlTypes;

namespace StudentApplication
{
    public class ConnectToDBS
    {
        SqlConnection conn = new SqlConnection(@"Data Source=WORKPC-
MBULELO\SQLEXPRESS01;Initial Catalog=studentAppConnect;Integrated
Security=True;");
        SqlCommand command;
        public DataTable table;
        DataRow row;
        SqlDataReader reader;
    }
}

```

```

        public void registerStudents( int ID, string Name, string
Surname, string Address, string RegFees)
        {
            conn.Open();
            command = new SqlCommand(string.Format("Insert into
StudentReg values ('{0}', '{1}', '{2}', '{3}',
'{4}')" ,ID,Name,Surname,Address,RegFees), conn); //This is basically
SQL, you can Insert, Update or Delete
            command.ExecuteNonQuery(); //Detect a query that failed,
conn.Close(); // then closes the connection
            command.Dispose(); //Show you the query that didn't
execute
        }
    }
}

```

*We then click on the Register Window (WPF)

```

namespace StudentApplication
{
    /// <summary>
    /// Interaction logic for StudentRegistrationWindow.xaml
    /// </summary>
    public partial class StudentRegistrationWindow : Window
    {
        public StudentRegistrationWindow()
    }
}

```

```

        {
            InitializeComponent();
        }

        private void btnRegister_Click(object sender, RoutedEventArgs
e)
        {
            ConnectToDBS dbc = new ConnectToDBS();
            dbc.registerStudents(Convert.ToInt32(txtId.Text),txtName.Text,txtSurn
ame.Text, txtAddress.Text,txtFees.Text);
            MessageBox.Show("Student Registered Successfully");
            txtId.Text = "";
            txtName.Text = "";
            txtSurname.Text = "";
            txtAddress.Text = "";
            txtFees.Text = "";
        }
    }
}

```

The Registration Page:

StudentRegistrationWindow

Student Registration Page

StudentID: 12

Student Name: Mbumtshi

Student Surname: Mtshutshu

Address: Brakpan

Student Reg Fees: 6000

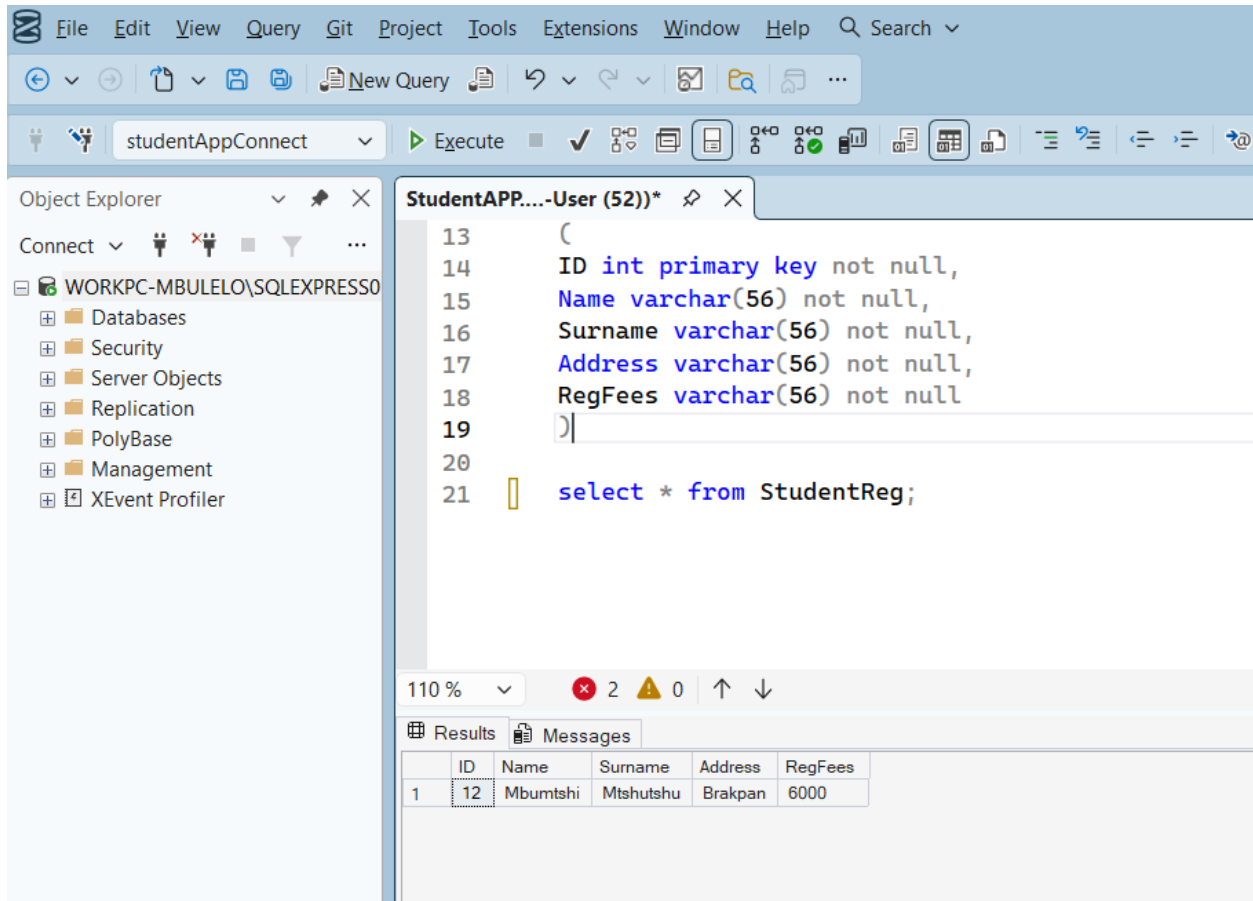
REGISTER

select * from StudentReg

Student Registered Successfully

OK

The database shows:



Three methods to register, to Update and to Delete:

StudentRegistrationWindow

Student Registration Page

StudentID:

Student Name:

Student Surname:

Address:

Student Reg Fees:

DELETE
REGISTER
UPDATE

```

using System;
using System.Collections.Generic;
using System.Text;

using System.Data;
using System.Data.Sql;
using System.Data.Entity;
using System.Data.SqlClient;
using System.Data.SqlTypes;

namespace StudentApplication
{
    public class ConnectToDBS
    {

        SqlConnection conn = new SqlConnection(@"Data Source=WORKPC-
MBULELO\SQLEXPRESS01;Initial Catalog=studentAppConnect;Integrated
Security=True;");
        SqlCommand command;
        public DataTable table;
        DataRow row;
        SqlDataReader reader;

        public void registerStudents( int ID, string Name, string
Surname, string Address, string RegFees)
        {
            conn.Open();
            command = new SqlCommand(string.Format("Insert into
StudentReg values ('{0}', '{1}', '{2}', '{3}',
'{4}')" ,ID,Name,Surname,Address,RegFees), conn); //This is basically
SQL, you can Insert, Update or Delete
            command.ExecuteNonQuery(); //Detect a query that failed,
conn.Close(); // then closes the connection
            command.Dispose(); //Show you the query that didn't
execute
        }

        public void deleteStudent(int ID) {
            conn.Open();
            command = new SqlCommand(string.Format("Delete from
StudentReg" + " where ID=@ID"), conn);
            command.Parameters.AddWithValue("@ID", ID);
            command.ExecuteNonQuery();
            conn.Close();
            command.Dispose();
        }
    }
}

```

```

    }

    public void updateStudentInf(int ID, string Name, string
Surname, string Address, string RegFees)
    {
        conn.Open();
        command = new SqlCommand(string.Format("Update StudentReg
set Name=@Name", "Surname=@Surname"+ " ,Address=@Address"+",
RegFees=@RegFees"+" where ID=@ID"), conn);
        command.Parameters.AddWithValue("@Name", Name);
        command.Parameters.AddWithValue("@Surname", Surname);
        command.Parameters.AddWithValue("@Address", Address);
        command.Parameters.AddWithValue("@RegFees", RegFees);
        command.ExecuteNonQuery();
        conn.Close();
        command.Dispose();

    }

}
}

```

The Update button:

```

private void buttonUpdate_Click(object sender, RoutedEventArgs e)
{
    ConnectToDBS dbc = new ConnectToDBS();
    dbc.updateStudentInf(Convert.ToInt32(txtId.Text), txtName.Text,
txtSurname.Text, txtAddress.Text, txtFees.Text);
    MessageBox.Show("The Student Info was\n Updated Successfully");
    txtId.Text = "";
    txtName.Text = "";
    txtSurname.Text = "";
    txtAddress.Text = "";
    txtFees.Text = "";
}

```

The Delete button:

```

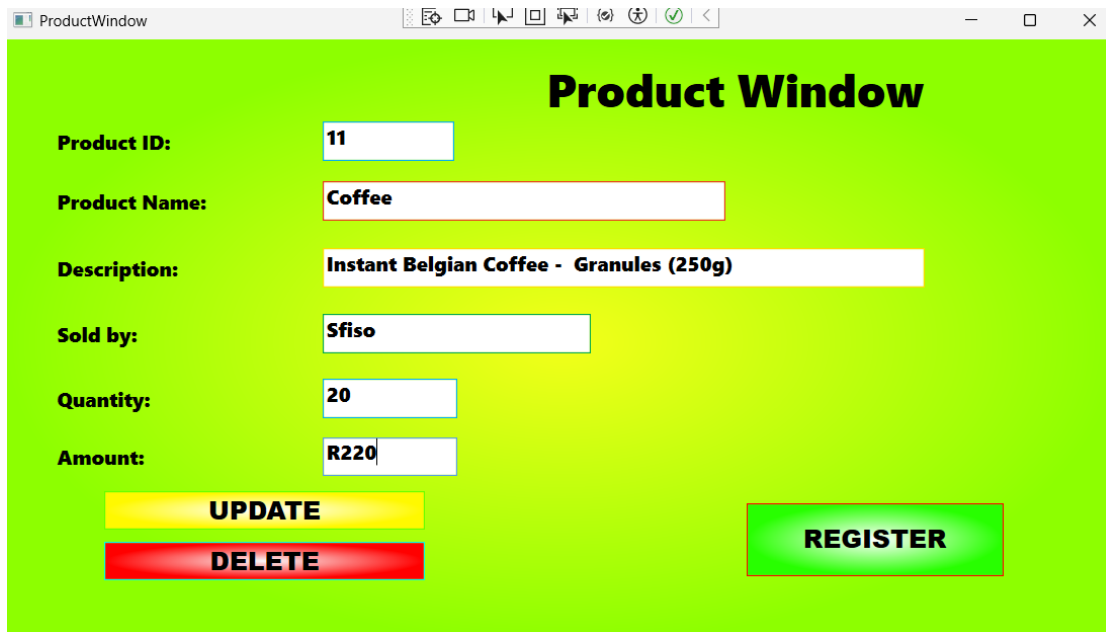
private void buttonDel_Click(object sender, RoutedEventArgs e)
{
    ConnectToDBS dbc = new ConnectToDBS();
    dbc.deleteStudent(Convert.ToInt32(txtId.Text));
}

```

```
    MessageBox.Show("Student Deleted Successfully");  
    txtId.Text = "";  
}
```

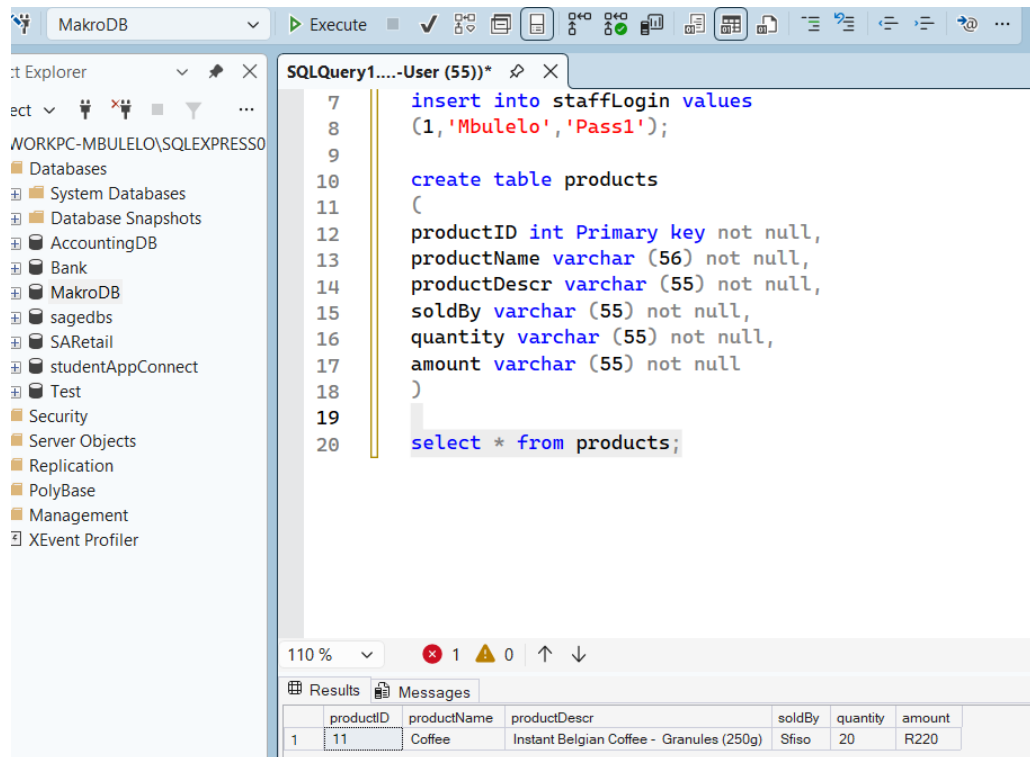
THE MAKRO SYSTEM (Last C# Project) - 05 March 2026

*We're using this one for the Presentation tomorrow. (EMail from Francis Anosike)



The screenshot shows a Windows application window titled "ProductWindow". The interface has a bright green background. At the top center, the title "Product Window" is displayed in large, bold, black font. Below the title, there are several input fields with labels to their left: "Product ID:" with a text box containing "11", "Product Name:" with a text box containing "Coffee", "Description:" with a text box containing "Instant Belgian Coffee - Granules (250g)", "Sold by:" with a text box containing "Sfiso", "Quantity:" with a text box containing "20", and "Amount:" with a text box containing "R220". At the bottom left, there are two buttons: a yellow "UPDATE" button and a red "DELETE" button. At the bottom right, there is a green "REGISTER" button. The window has a standard Windows title bar with minimize, maximize, and close buttons.

Here's the backend:



110 % 1 0						
Results Messages						
	productID	productName	productDescr	soldBy	quantity	amount
1	11	Coffee	Instant Belgian Coffee - Granules (250g)	Sfiso	20	R220
2	22	Pilchard Sardines	Small Sardines (100g)	Dan	64	R80

MONDAY (9/MAR/2026) - INTRODUCTION TO PYTHON

IDLE (Python)

*PCEP certification -(Python lv1)

Get PCEP Certified

Upon completion of Python Essentials 1, you will be prepared to attempt the qualification PCEP - Certified Entry-Level Python Programmer.

PCEP - Certified Entry-Level Python Programmer certification shows that you are familiar with universal computer programming concepts like data types, containers, functions, conditions, and loops, as well as the most important aspects of the Python programming language syntax, semantics, and the runtime environment.

Becoming PCEP certified ensures that you are acquainted with the most essential means provided by Python 3, which enable you to start your own studies at an intermediate level and to continue your professional development.

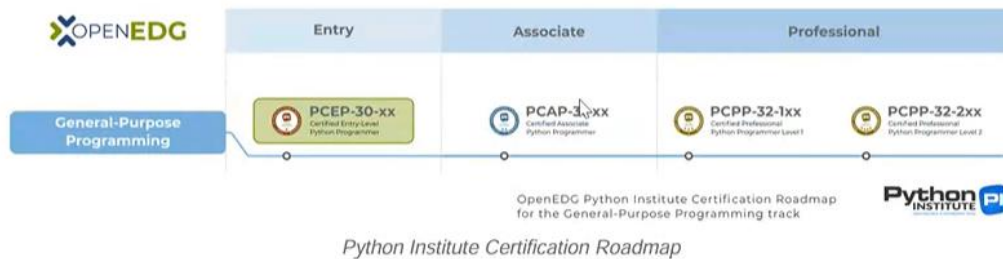
Certification Benefits

- Demonstrates your Python programming knowledge
- Access to a large network of Python programming professionals
- Interim step to PCAP certification
- Starting point to launch a career in software development
- Helps you stand out from other candidates



PCEP - Certified Entry-Level Python Programmer certification is an interim step to the PCAP - Certified Associate in Python Programming certification and the starting point to launch a career in software development, Python programming, and related technologies.

Becoming PCEP certified will help the certificate holder stand out from other candidates and get their foot in the door.



-History of Python. It's one of the simplest coding languages.

Frameworks: Django & Flask and...

Generally, Python is a great choice for:

- Web and Internet development (e.g., Django and Pyramid frameworks, Flask and Bottle micro-frameworks)
- Scientific and numeric computing (e.g., SciPy - a collection of packages for mathematics, science, and engineering)
- Education (it's a brilliant language for teaching programming!)
- Desktop GUIs (e.g., wxWidgets, Kivy, Qt)
- Software Development (build control, management, and testing - Scons, Buildbot, Apache Gump, Roundup, Trac)
- Business applications (ERP and e-commerce systems - Odoo, Tryton)

WHERE TO USE Python:

Where Do You Use Python?

Do you remember Battlefield 2, Battlefield 2142 and Battlefield Heroes - strategy and first person shooter games from EA DICE? All the games use Python for logic and server controls. Python is frequently used for creating open-source, free games, e.g., OpenRTS, PySol, Metin 2, or Frets On Fire - famous Guitar Hero-like games written in pygame.

And what about the major websites and services? Dropbox? UBER? Spotify? Pinterest? BuzzFeed? Yes. They were all written, to a greater or lesser extent, in Python.

Python Application Areas

Internet Applications: BitTorrent, Jogger Publishing Assistant, TheCircle, TwistedMatrix

3D CAD/CAM: FreeCAD, Fandango, Blender, Vintech RCAM

Enterprise Applications: Odoo, Tryton, Picalo, LinOTP 2, RESTx

Image Applications: Gnofract 4D, Gogh, imgSeek, MayaVi, VPython

Mobile Applications: Aarlogic C05/3, AppBackup, Pvroute

Office Applications: calibre, faces, Notalon, || meet.google.com is sharing your screen. Stop sharing Hide

Personal Information Managers: BitPim, Name Database, Task Force, WordPad

1.3 What is Python?

Python is a widely-used, interpreted, object-oriented, and high-level programming language with dynamic semantics, used for general-purpose programming.

And while you may know the python as a large snake, the name of the Python programming language comes from an old BBC television comedy sketch series called Monty Python's Flying Circus.

At the height of its success, the Monty Python team were performing their sketches to live audiences across the world, including at the Hollywood Bowl.

Since Monty Python is considered one of the two fundamental nutrients to a programmer (the other being pizza), Python's creator named the language in honor of the TV show.

Guido van Rossum - Creator of Python

Who Created Python?

One of the amazing features of Python is the fact that it is actually one person's work. Usually, new programming languages are developed and published by large companies employing lots of professionals, and due to copyright rules, it is very hard to name any of the people involved in the project. Python is an exception.

I
Python was created by Guido van Rossum, born in 1956 in Haarlem, the Netherlands. Of course, Guido van Rossum did not develop and evolve all the Python components himself. The speed with which Python has spread around the world is a result of the continuous work of thousands of programmers, testers, users and enthusiasts, but it must be said that the very first idea came to one head - Guido's.

A Hobby Programming Project

In December 1989, I was looking for a "hol
My office would be closed, but I had a hon

|| meet.google.com is sharing your screen.

Stop sharing

Hide

upied during the week around Christmas.
ecided to write an interpreter for the new

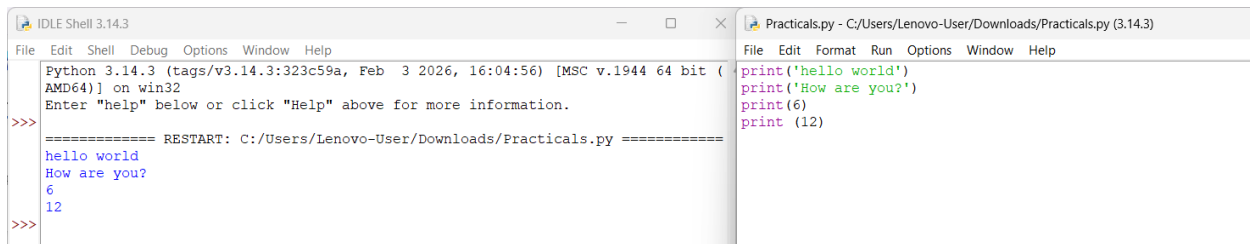
1.2 Compilation vs. Interpretation

Computer programming is the act of composing the selected programming language's elements in the order that will cause the desired effect. The effect could be different in every specific case - it's up to the programmer's imagination, knowledge and experience.

Of course, such a composition has to be correct in many senses:

- Alphabetically - a program needs to be written in a recognizable script
- Lexically - each programming language has its own set of rules for writing it
- Syntactically - each language has its own set of rules for writing it

The first code:



The screenshot shows two windows from the IDLE Python IDE. The left window, titled 'IDLE Shell 3.14.3', displays the Python interpreter's startup message and the output of the first code: 'hello world', 'How are you?', '6', and '12'. The right window, titled 'Practicals.py - C:/Users/Lenovo-User/Downloads/Practicals.py (3.14.3)', shows the source code of the program: `print('hello world')`, `print('How are you?')`, `print(6)`, and `print(12)`.

```
print('hello world')
print('How are you?')
print(6)
print(12)
```

Module 1 Summary

Congratulations! You have completed Module 1. Here's a short summary of the objectives you've covered:

- The fundamentals of computer programming
- The difference between compilation and interpretation
- Basic information about Python and its versions
- How to install Python and write your first program

MODULE 2

*Multiple Arguments

*String can be in single or double quotation marks

(Comment)

```
Python 3.14.3 (tags/v3.14.3:323c59a, Feb 3 2026, 16:04:56) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/Lenovo-User/Downloads/Practicals.py =====
hello world
How are you?
6
12
>>>
===== RESTART: C:/Users/Lenovo-User/Downloads/Practicals.py =====
hello world How are you?
6 12
>>>
```

```
Practicals.py - C:/Users/Lenovo-User/Downloads/Practicals.py (3.14.3)
File Edit Format Run Options Window Help
print('hello world','How are you?')
#This is a comment
print(6, 12)
```

```
print('hello world','How are you?')
#This is a comment
print(6, 12)
```

Or

```
Python 3.14.3 (tags/v3.14.3:323c59a, Feb 3 2026, 16:04:56) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/Lenovo-User/Downloads/Practicals.py =====
hello world, the time is: 9 : 45
>>>
```

```
Practicals.py - C:/Users/Lenovo-User/Downloads/Practicals.py (3.14.3)
File Edit Format Run Options Window Help
#This is a comment representing multiple arguments
print('hello world, the time is: ',9,':',45)
```

```
#This is a comment representing multiple arguments
print("hello world, the time is: ",9,':',45)
```

Escape characters:

Escape Characters

The backslash (`\`) has a very special meaning when used inside strings - this is called the escape character.

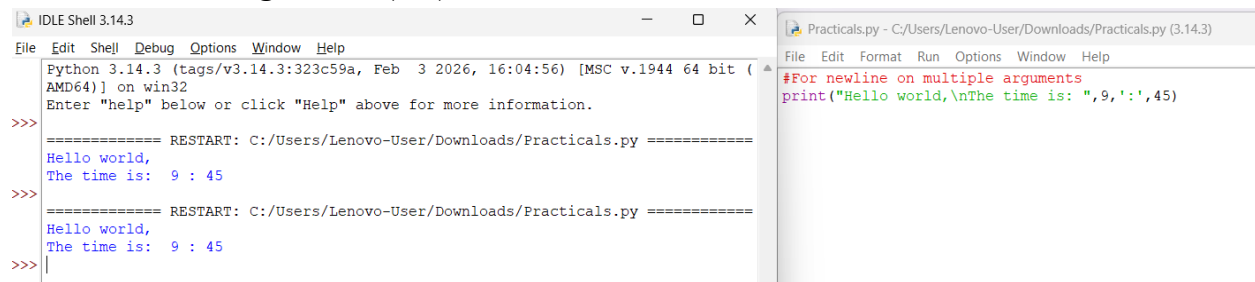
Newline character

```
print("Line 1\nLine 2\nLine 3")
# \n creates a new line
```

Common Escape Sequences

- `\n` - Newline
- `\t` - Tab
- `\\` - Backslash
- `\"` - Double quote
- `\'` - Single quote

For NewLine Argument (\n):



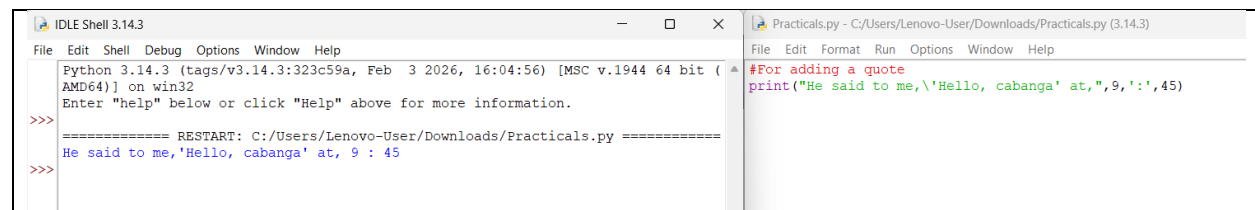
The screenshot shows two windows from the IDLE Python IDE. The left window, titled 'IDLE Shell 3.14.3', displays the output of a Python script: 'Hello world,' followed by 'The time is: 9 : 45' on a new line. The right window, titled 'Practicals.py - C:/Users/Lenovo-User/Downloads/Practicals.py (3.14.3)', shows the source code. It contains a comment '#For newline on multiple arguments' and a print statement: `print("Hello world,\nThe time is: ",9,':',45)`. The output in the shell confirms that the `\n` escape sequence successfully inserts a new line.

Double backslash:

```
[10]: print('yes\\no')
      yes\nno
```

The single quotation inside the sentence:

```
#For adding a single quote
print("He said to me,\n'Hello, cabanga' at,",9,':',45)
```



The screenshot shows the IDLE Python IDE. The left window, 'IDLE Shell 3.14.3', shows the output: 'He said to me,\'Hello, cabanga\' at, 9 : 45'. The right window, 'Practicals.py', shows the code: `print("He said to me,\n'Hello, cabanga' at,",9,':',45)`. The output in the shell shows that the single quote inside the double-quoted string is printed correctly without escaping.

The double quotation inside the sentence:

```
#For adding a double quote
print('He said to me,\n"Hello, cabanga" at,',9,':',45)
```

```
IDLE Shell 3.14.3
Python 3.14.3 (tags/v3.14.3:323c59a, Feb 3 2026, 16:04:56) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Lenovo-User/Downloads/Practicals.py =====
He said to me,'Hello, cabanga' at, 9 : 45
>>>
===== RESTART: C:/Users/Lenovo-User/Downloads/Practicals.py =====
He said to me,"Hello, cabanga" at, 9 : 45
>>>
```

Keyword Arguments

The print() function has two keyword arguments:

end keyword argument

```
print("Hello", end=" ")
print("World")
# Output: Hello World
```

sep keyword argument

```
print("A", "B", "C", sep="-")
# Output: A-B-C
```

The End

```
[21]: print('How are you?', end=" ")
      print('I like coding')
      How are you? I like coding
```

End= “ ” makes sure it concatenates the two print statements in one line.

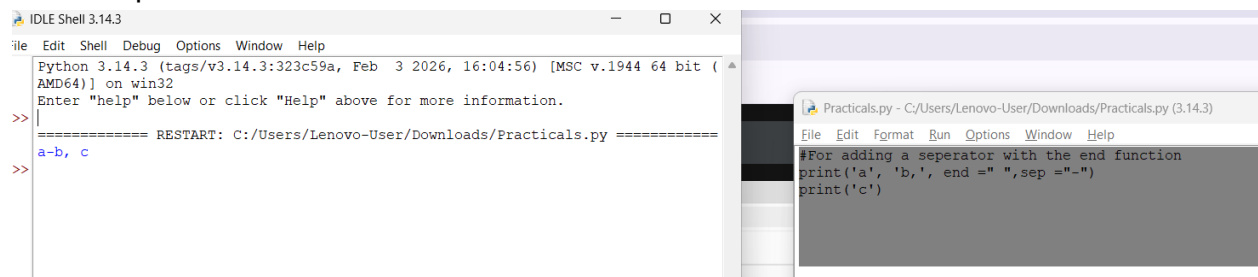
The Separator

```
>>>
===== RESTART: C:/Users/Lenovo-User/Downloads/Practicals.py =====
He said-to me,-Hello
>>>
```

#For adding a separator

```
print('He said', 'to me,', 'Hello', sep = "-")
```

Both Separator and the End function:

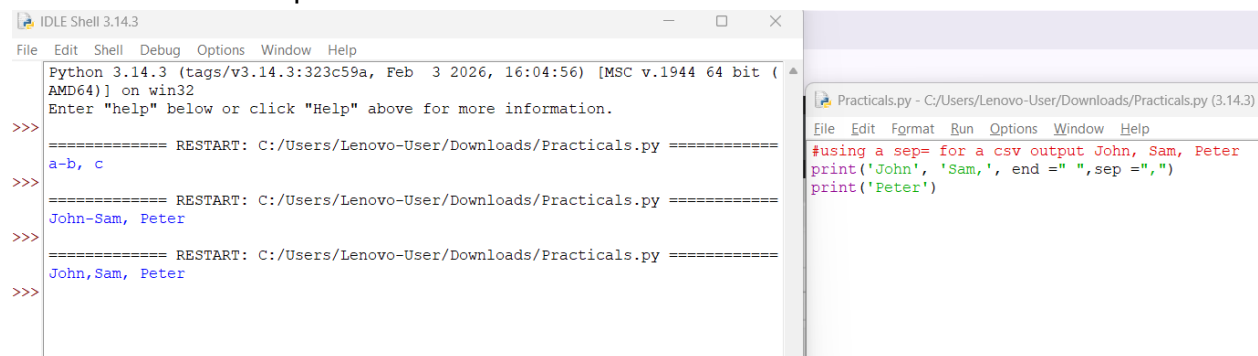


The screenshot shows the IDLE Shell 3.14.3 window on the left and a Python file named 'Practicals.py' on the right. The shell window displays the output of a Python script: 'a-b, c'. The Python file contains the following code:

```
File Edit Format Run Options Window Help
#For adding a separator with the end function
print('a', 'b,', end = " ", sep = "-")
print('c')
```

```
#For adding a separator with the end function
print('a', 'b,', end = " ", sep = "-")
print('c')
```

Here's more examples:



The screenshot shows the IDLE Shell 3.14.3 window on the left and a Python file named 'Practicals.py' on the right. The shell window displays the output of a Python script: 'John-Sam, Peter'. The Python file contains the following code:

```
File Edit Format Run Options Window Help
#using a sep= for a csv output John, Sam, Peter
print('John', 'Sam,', end = " ", sep = ",")
print('Peter')
```

```
#using a sep= for a csv output John, Sam, Peter
print('John', 'Sam,', end = " ", sep = ",")
print('Peter')
```

The \n practical example:

```
#Putting each item on a new line {one, two, three} with one print
statement
print(" one \n", "two \n","three \n");
print("one", "two","three", sep = "\n");
```

Literal Values and Datatypes

2.2 Literals and Data Types

A literal is data whose values are determined by the literal itself. Python has several types of literals:

Integer Literals

```
123      # decimal
0o123    # octal (prefix 0o)
0x123    # hexadecimal (prefix 0x)
11_111_111 # with underscores for readability
```

Float Literals

```
2.5      # standard float
.4        # 0.4 (leading zero can be omitted)
4.        # 4.0 (trailing zero can be omitted)
```

meet.google.com is sharing your screen. [Stop sharing](#) [Hide](#)

*Literal value is the literal value assigned to the data type, it's the data itself.

```
[ ]: # variable
salary = 4000
name = 'Peter'
Is_it_raining = True
```

e.g. Salary is the name of the variable, and then the literal value is 4000. It means automatically Python understands what the datatype is.

Boolean values represent truthfulness. There are two Boolean values in Python:

Boolean values

```
True     # represents "yes" or 1
False    # represents "no" or 0

print(True > False)  # True (1 > 0)
print(True < False)  # False (1 < 0)
```

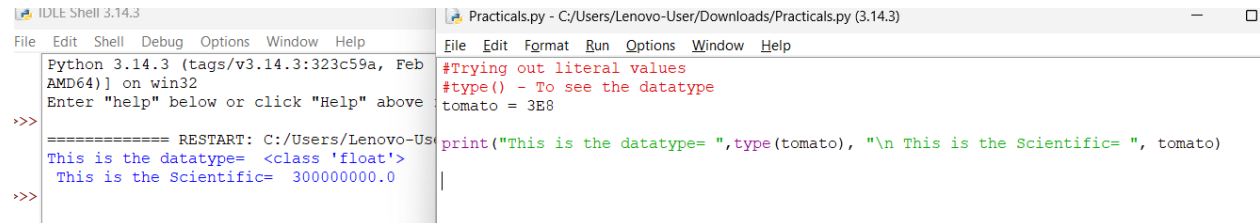
```
[1]: a = 12
b = 5
if a > 5:
    print(True)
else:
    print(False)

True
```

String Literals

```
"Hello"    # double quotes
'Hello'    # single quotes
"I'm happy" # apostrophe inside double quotes
'He said "Hi"' # quotes inside single quotes
```

Working with Variables:



The screenshot shows two windows. The left window is the IDLE Shell, and the right window is a file named 'Practicals.py'. The shell shows the execution of the code from the file, displaying the output of the print statement: 'This is the datatype= <class 'float'>' and 'This is the Scientific= 300000000.0'.

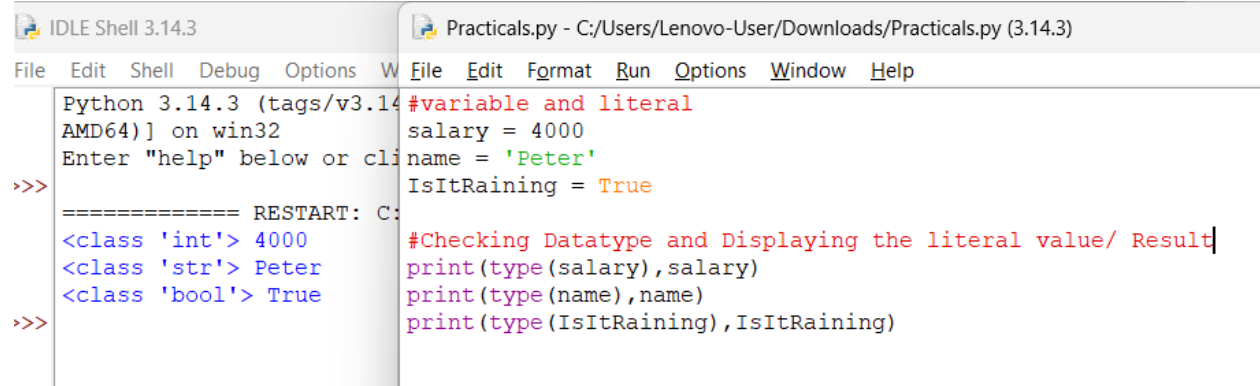
```
Python 3.14.3 (tags/v3.14.3:323c59a, Feb AMD64) on win32
Enter "help" below or click "Help" above
>>>
===== RESTART: C:/Users/Lenovo-US
This is the datatype= <class 'float'>
This is the Scientific= 300000000.0
>>>
```

```
Practicals.py - C:/Users/Lenovo-User/Downloads/Practicals.py (3.14.3)
File Edit Format Run Options Window Help
#Trying out literal values
#type() - To see the datatype
tomato = 3E8
print("This is the datatype= ",type(tomato), "\n This is the Scientific= ", tomato)
```

```
#Trying out literal values
#type() - To see the datatype
tomato = 3E8

print("This is the datatype= ",type(tomato), "\n This is the
Scientific= ", tomato)
```

Another example:



The screenshot shows two windows. The left window is the IDLE Shell, and the right window is a file named 'Practicals.py'. The shell shows the execution of the code from the file, displaying the output of the print statements: '4000', 'Peter', 'True', '4000', 'Peter', and 'True'.

```
Python 3.14.3 (tags/v3.14.3:323c59a, Feb AMD64) on win32
Enter "help" below or click "Help" above
>>>
===== RESTART: C:/Users/Lenovo-US
<class 'int'> 4000
<class 'str'> Peter
<class 'bool'> True
>>>
```

```
Practicals.py - C:/Users/Lenovo-User/Downloads/Practicals.py (3.14.3)
File Edit Format Run Options Window Help
#variable and literal
salary = 4000
name = 'Peter'
IsItRaining = True
#Checking Datatype and Displaying the literal value/ Result
print(type(salary),salary)
print(type(name),name)
print(type(IsItRaining),IsItRaining)
```

```
#variable and literal
salary = 4000
name = 'Peter'
```

```
IsItRaining = True
```

```
#Checking Datatype and Displaying the literal value/ Result
```

```
print(type(salary),salary)
```

```
print(type(name),name)
```

```
print(type(IsItRaining),IsItRaining)
```

THE RULES OF PYTHON:

2.3 Variables and Naming Conventions

A variable is a named location in the computer's memory that stores a value. You can think of it as a labeled box that holds data.

Variables example

```
# Variable assignment
name = "Python"
version = 3.11
is_popular = True

# Using variables
print(name)      # Python
print(version)   # 3.11
print(is_popular) # True
```

Variable Naming Rules

1. Must start with a letter (a-z, A-Z) or underscore (_)
2. Can contain letters, digits, and underscores
3. Cannot be a Python keyword (if, else, for, while, etc.)
4. Case-sensitive (Name and name are different)
5. Should be descriptive and meaningful

-Python is case sensitive

The Good Naming conventions:

Variable naming examples

```
# Good variable names
user_name = "Alice"
total_price = 99.99
is_valid = True

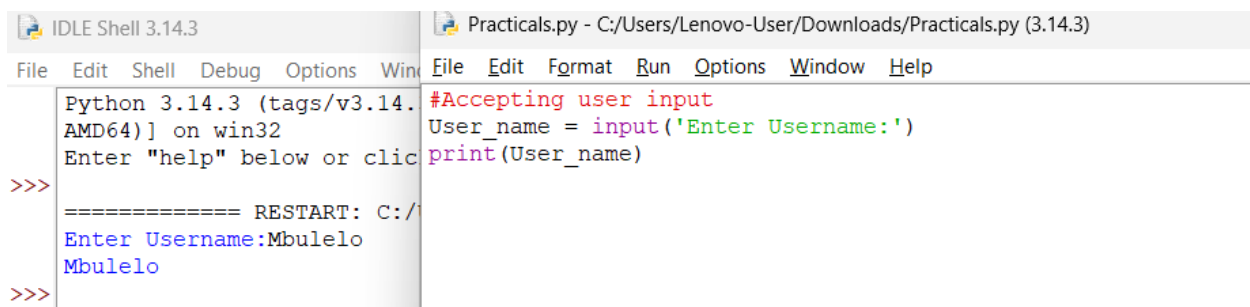
# Bad variable names
x = "Alice"      # not descriptive
2name = "Bob"    # starts with digit
my-var = "Carol" # contains hyphen
```

2.4 Input and Output Operations

The input() function allows you to get user input from the keyboard.

```
] user_name = input('Enter Username:|')
```

Accepting user input in Python:



```
IDLE Shell 3.14.3
Python 3.14.3 (tags/v3.14.3 AMD64) on win32
Enter "help" below or click the ? icon for help
>>>
===== RESTART: C:\Python314\IDLE\shell.py =====
>>> Enter Username: Mbulelo
Mbulelo
>>>
```

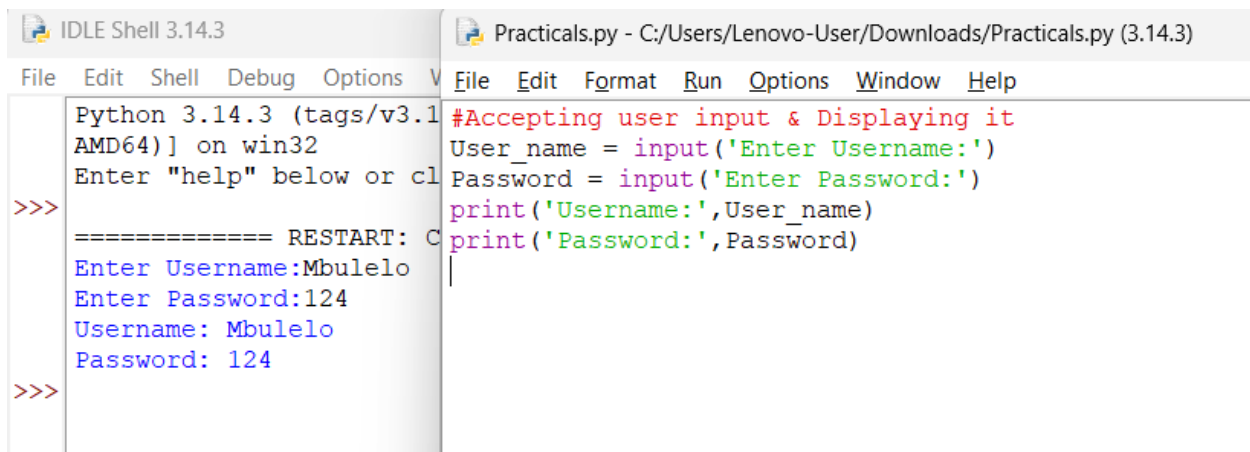
```
#Accepting user input
User_name = input('Enter Username:')
print(User_name)
```

The If else statement (early):

```
*[16]: # Using the input() function
user_name = input('Enter Username:')
Password = input("Enter password:")
if Password == 1122:
    print ('Password correct!')
else:
    print('Incorrect Password')

Enter Username: Peter
Enter password: 1122
```

Accepting user input & Displaying it



```
Python 3.14.3 (tags/v3.14.3:amd64) on win32
Enter "help" below or click the ? icon for help
>>>
===== RESTART: C:\Python314\IDLE\shell.py =====
Enter Username:Mbulelo
Enter Password:124
Username: Mbulelo
Password: 124
>>>
```

```
#Accepting user input & Displaying it
User_name = input('Enter Username:')
Password = input('Enter Password:')
print('Username:',User_name)
print('Password:',Password)
```

*When we want to convert the input from string to a specific datatype you write it like this:

input() function

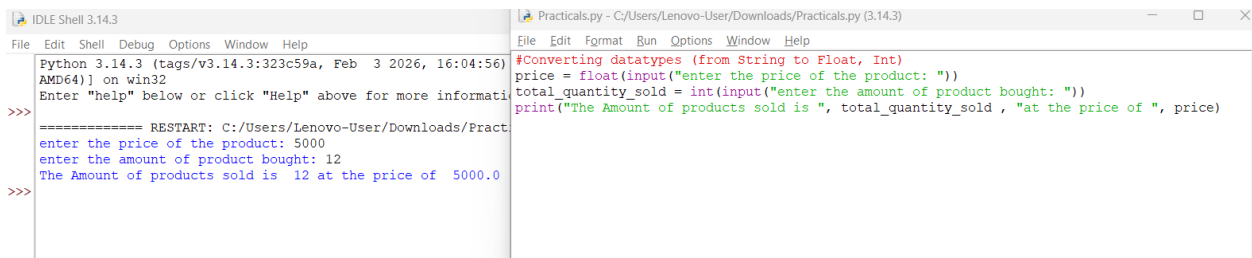
```
name = input("Enter your name: ")
print("Hello,", name)
```

Important Note

The input() function always returns a string. If you need a number, you must convert it:

```
age = int(input("Enter your age: "))
price = float(input("Enter the price: "))
```

HERE's an example:



The image shows two windows from a Python IDE. The left window, titled 'IDLE Shell 3.14.3', displays the execution of a Python script. It shows the prompt 'Python 3.14.3 (tags/v3.14.3:323c59a, Feb 3 2026, 16:04:56) AMD64) on win32' followed by a prompt to enter 'help'. The user enters '5000' for the price and '12' for the amount, resulting in the output: 'The Amount of products sold is 12 at the price of 5000.0'. The right window, titled 'Practicals.py - C:/Users/Lenovo-User/Downloads/Practicals.py (3.14.3)', shows the source code of the script. The code uses `float()` and `int()` to convert user input from strings to the appropriate data types before performing calculations and printing the result.

```
Python 3.14.3 (tags/v3.14.3:323c59a, Feb 3 2026, 16:04:56)
AMD64) on win32
Enter "help" below or click "Help" above for more informati
>>>
===== RESTART: C:/Users/Lenovo-User/Downloads/Pract
enter the price of the product: 5000
enter the amount of product bought: 12
The Amount of products sold is 12 at the price of 5000.0
>>>
```

```
#Converting datatypes (from String to Float, Int)
price = float(input("enter the price of the product: "))
total_quantity_sold = int(input("enter the amount of product bought: "))
print("The Amount of products sold is ", total_quantity_sold , "at
the price of ", price)
```

```
#Converting datatypes (from String to Float, Int)
price = float(input("enter the price of the product: "))
total_quantity_sold = int(input("enter the amount of product bought:
"))
print("The Amount of products sold is ", total_quantity_sold , "at
the price of ", price)
```

Arithmetic Operators

Arithmetic Operators

Arithmetic operators

```
# Basic arithmetic
print(2 + 3) # 5 (addition)
print(5 - 2) # 3 (subtraction)
print(3 * 4) # 12 (multiplication)
print(10 / 3) # 3.333... (division - always float)
print(10 // 3) # 3 (integer division)
print(10 % 3) # 1 (modulo - remainder)
print(2 ** 3) # 8 (exponentiation)
```

```
[23]: # usage of + as an operator
```

```
print(12+4)
```

```
16
```

```
[24]: a = 100
```

```
b = 120
```

```
c = a+b
```

```
print(c)
```

```
220
```

Perimeter Problem (Arithmetic Operations):

```
[24]: a = 100
      b = 120
      c = a+b
      print(c)

220

[26]: length = int(input('Enter the length of the football pitch:'))
      width = int(input('Enter the width of the stadium:'))
      perimeter = (length+width+length+width)

Enter the length of the football pitch: 100
Enter the width of the stadium: 100

[28]: print('The perimeter of the stadium in square m is', perimeter)

The perimeter of the stadium in square m is 400
```

Concatenate:

```
[29]: #concatnet - to glue together
      name = 'Peter'
      Surname = 'James'
      print(name+Surname)

PeterJames
```

Minus operation:

```
: #using the minus operator
gross_salary = 1000
deductions = float(input('Enter total deductions:'))
net_salary = (gross_salary-deductions)
print('The gross salary is', gross_salary, 'and the total deductions is ', deductions, '\n The Net Salary is', net_salary)
```

An example (Multiplied):

```
[33]: # The usage of * as a multiplication operator
      price = 2.00
      scoan = int(input('Enter the number of scoans sold today:'))
      revenue = (price*scoan)
      print('The total amount of revenue generated today is ', revenue)

Enter the number of scoans sold today: 100
The total amount of revenue generated today is 200.0

[34]: print(9*7)

63
```

```
[35]: length = float(input('Enter length: '))
      Width = float(input('enter width: '))
      area = length*Width
      print('Area of the shape is', area)

Enter length: 20
enter width: 10
Area of the shape is 200.0
```

Here's my example

```
===== RESTART: C:/Users/Lenovo-User/
Enter Length:100
Enter Width:200
The Length is: 100.0 & The Width is: 200.0

The total Area of the shape is = 20000.0

Practicals.py - C:/Users/Lenovo-User/Downloads/Practicals.py (3.14.3)
File Edit Format Run Options Window Help
length = float(input("Enter Length:"))
width = float (input("Enter Width:"))
area = length * width

print ("The Length is:", length, end = " ")
print ("% The Width is:", width)
print ("\n The total Area of the shape is =", area)
```

```

length = float(input("Enter Length:"))
width = float (input("Enter Width:"))
area = length * width

print ("The Length is:", length, end = " ")
print ("& The Width is:", width)
print ("\n The total Area of the shape is =", area)

```

The division:

```

[ ]: # / for division
pizza_slices = int(input('Enter the number of pizza slices:'))
Number_of_people = int(input('Enter the number of people:'))
slices_per_person = pizza_slices/Number_of_people
print('The number of slices each person get is',slices_per_person)

```

The screenshot shows a Python IDE with two panes. The left pane displays the code for calculating the number of slices per person, and the right pane shows the output of the program.

```

Python 3.14.3 (tags/v3.14.3:323c59a, Feb  3 2026, 16:04:56) [AMD64] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Lenovo-User/Downloads/Practicals.py
Enter no of pizza slices:12
Enter the number of people:3
The no of pizza slice is: 12 & The the number of people is: 3
>>>
The total number of slices each person gets is = 4.0

```

```

pizza_slices = int(input("Enter no of pizza slices:"))
No_of_people = int (input("Enter the number of people:"))
slices_per_person = (pizza_slices / No_of_people)

print ("The no of pizza slice is:", pizza_slices, end = " ")
print ("& The the number of people is:", No_of_people)
print ("\n The total number of slices each person gets is =",
slices_per_person)

```

Integer division (//):

Returns the divided with out the remainder.

The screenshot shows two windows from the IDLE Python environment. The left window, titled 'IDLE Shell 3.14.3', shows the execution of a program. It prompts the user to enter the number of pizza slices (12) and the number of people (3). The program then calculates and displays that each person gets 4.0 slices. The right window, titled 'Practicals.py - C:/Users/Lenovo-User/Downloads/Practicals.py (3.14.3)', shows the source code of the program. The code uses the division operator (//) to calculate the number of slices per person.

```
python 3.14.3 (tags/v3.14.3:323c59a, Feb 3 2026, 16:04:56) [MS
AMD64] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Lenovo-User/Downloads/Practicals.py
Enter no of pizza slices:12
Enter the number of people:3
The no of pizza slice is: 12 & The the number of people is: 3

The total number of slices each person gets is = 4.0
>>>
===== RESTART: C:/Users/Lenovo-User/Downloads/Practicals.py
Enter no of pizza slices:13
Enter the number of people:3
The no of pizza slice is: 13 & The the number of people is: 3

The total number of slices each person gets is = 4
>>>
```

```
Practicals.py - C:/Users/Lenovo-User/Downloads/Practicals.py (3.14.3)
File Edit Format Run Options Window Help
pizza_slices = int(input("Enter no of pizza slices:"))
No_of_people = int (input("Enter the number of people:"))
slices_per_person = (pizza_slices // No_of_people)

print ("The no of pizza slice is:", pizza_slices, end = " ")
print ("& The the number of people is:", No_of_people)
print ("\n The total number of slices each person gets is =", slices_per_person)
```

```
python 3.14.3 (tags/v3.14.3:323c59a, Feb 3 2026, 16:04:56) [MS
AMD64] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Lenovo-User/Downloads/Practicals.py
Enter no of pizza slices:12
Enter the number of people:3
The no of pizza slice is: 12 & The the number of people is: 3

The total number of slices each person gets is = 4.0
>>>
===== RESTART: C:/Users/Lenovo-User/Downloads/Practicals.py
Enter no of pizza slices:13
Enter the number of people:3
The no of pizza slice is: 13 & The the number of people is: 3

The total number of slices each person gets is = 4
>>>
```

```
Practicals.py - C:/Users/Lenovo-User/Downloads/Practicals.py (3.14.3)
File Edit Format Run Options Window Help
pizza_slices = int(input("Enter no of pizza slices:"))
No_of_people = int (input("Enter the number of people:"))
slices_per_person = (pizza_slices // No_of_people)

print ("The no of pizza slice is:", pizza_slices, end = " ")
print ("& The the number of people is:", No_of_people)
print ("\n The total number of slices each person gets is =", slices_per_person)
```

Using the modulus operator:

The screenshot shows two windows from the IDLE Python environment. The left window, titled 'IDLE Shell 3.14.3', shows the execution of a program. It prompts the user to enter any number (5) and then displays the result of dividing 5 by 3, showing a remainder of 2. The right window, titled 'Practicals.py - C:/Users/Lenovo-User/Downloads/Practicals.py (3.14.3)', shows the source code of the program. The code uses the modulus operator (%) to calculate the remainder.

```
Python 3.14.3 (tags/v3.14.3:323c59a, Feb 3 2026, 16:04:56) [MS
AMD64] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Lenovo-User/Downloads/Practicals.py
Enter any number:5
The remainder: 1
>>>
===== RESTART: C:/Users/Lenovo-User/Downloads/Practicals.py
Enter any number:5
5 divided by 3
remainder is, 2
>>>
```

```
Practicals.py - C:/Users/Lenovo-User/Downloads/Practicals.py (3.14.3)
File Edit Format Run Options Window Help
#modulus operator
number = int(input("Enter any number:"))
num = 3
remainder =(number%num)
print(number, "divided by ", num, "\n remainder is,", remainder)
```

```
Python 3.14.3 (tags/v3.14.3:323c59a, Feb 3 2026, 16:04:56) [MS
AMD64] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Lenovo-User/Downloads/Practicals.py
Enter any number:5
The remainder: 1
>>>
===== RESTART: C:/Users/Lenovo-User/Downloads/Practicals.py
Enter any number:5
5 divided by 3
remainder is, 2
>>>
```

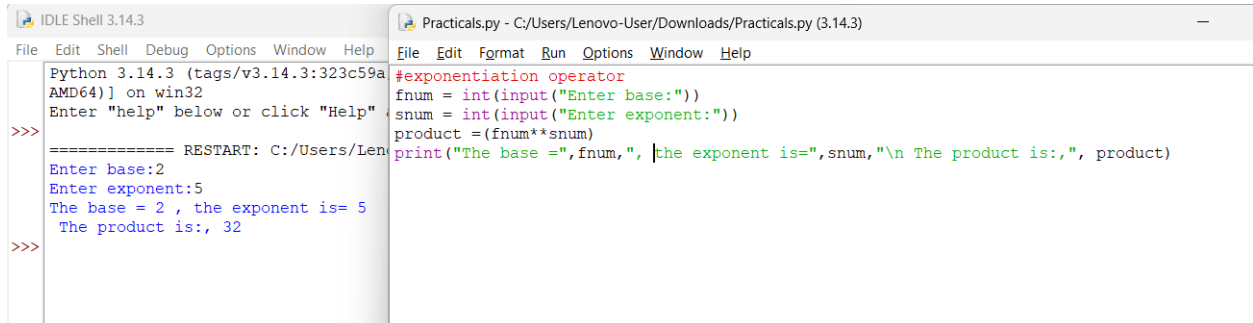
```
Practicals.py - C:/Users/Lenovo-User/Downloads/Practicals.py (3.14.3)
File Edit Format Run Options Window Help
#modulus operator
number = int(input("Enter any number:"))
num = 3
remainder =(number%num)
print(number, "divided by ", num, "\n remainder is,", remainder)
```

In case you forgot:

Arithmetic operators

```
# Basic arithmetic
print(2 + 3)    # 5 (addition)
print(5 - 2)    # 3 (subtraction)
print(3 * 4)    # 12 (multiplication)
print(10 / 3)   # 3.333... (division - always float)
print(10 // 3)  # 3 (integer division)
print(10 % 3)   # 1 (modulo - remainder)
print(2 ** 3)   # 8 (exponentiation)
```

Exponential:

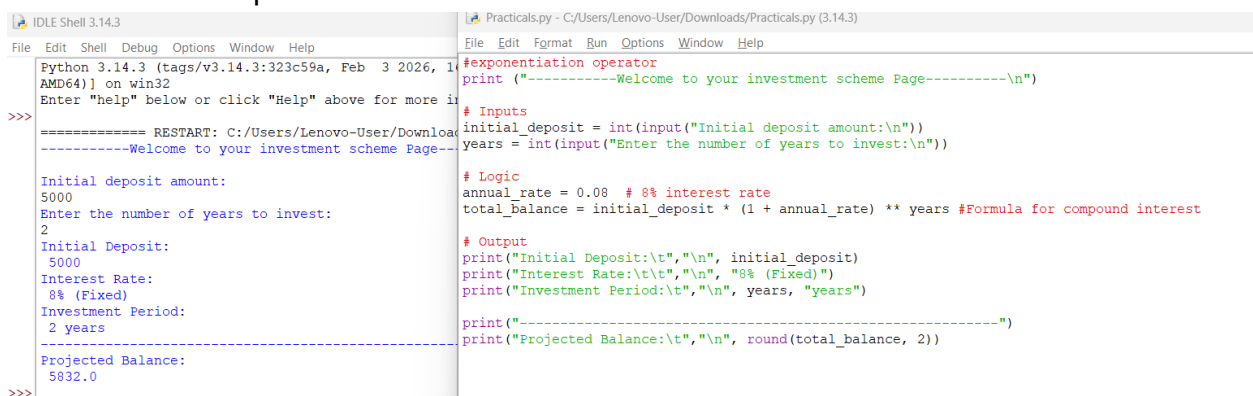


```
Python 3.14.3 (tags/v3.14.3:323c59a, Feb 3 2026, 1
AMD64) on win32
Enter "help" below or click "Help" above for more i
>>>
===== RESTART: C:/Users/Lenovo-User/Downloa
Enter base:2
Enter exponent:5
The base = 2 , the exponent is= 5
The product is:, 32
>>>
```

#exponentiation operator

```
fnum = int(input("Enter base:")) snum = int(input("Enter exponent:"))
product =(fnum**snum) print("The base =",fnum," , the exponent
is=",snum,"\n The product is:," , product)
```

Here's more exponential:



```
Python 3.14.3 (tags/v3.14.3:323c59a, Feb 3 2026, 1
AMD64) on win32
Enter "help" below or click "Help" above for more i
>>>
===== RESTART: C:/Users/Lenovo-User/Downloa
-----Welcome to your investment scheme Page-----
Initial deposit amount:
5000
Enter the number of years to invest:
2
Initial Deposit:
5000
Interest Rate:
8% (Fixed)
Investment Period:
2 years
-----
Projected Balance:
5832.0
>>>
```

#exponentiation operator

```
print ("-----Welcome to your investment scheme Page-----
\n")
```

#inputs

```

initial_deposit = int(input("Initial deposit amount:\n")) years =
int(input("Enter the number of years to invest:\n"))

#Logic

annual_rate = 0.08 # 8% interest rate total_balance = initial_deposit
* (1 + annual_rate) ** years #Formula for compound interest

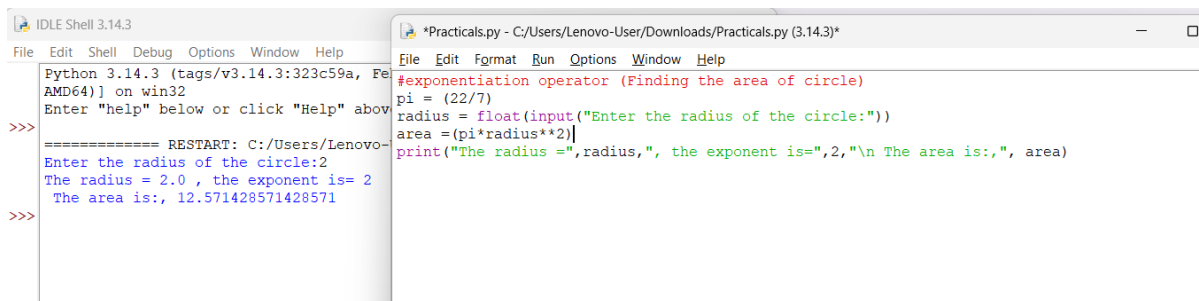
#Outputs

print("Initial Deposit:\t","\n", initial_deposit) print("Interest
Rate:\t\t","\n", "8% (Fixed)") print("Investment Period:\t","\n",
years, "years")

print("-----")
print("Projected Balance:\t","\n", round(total_balance, 2))

```

Another example of the exponential:



```

Python 3.14.3 (tags/v3.14.3:323c59a, Feb 19 2020) on win32
Enter "help" below or click "Help" above
>>>
===== RESTART: C:/Users/Lenovo-User/Python314/Python314.3/IDLE Shell
>>> Enter the radius of the circle:2
The radius = 2.0 , the exponent is= 2
The area is:, 12.571428571428571
>>>

```

```

#exponentiation operator (Finding the area of circle) pi = (22/7)
radius = float(input("Enter the radius of the circle:")) area
=(pi*radius**2) print("The radius =",radius," , the exponent
is=",2,"\n The area is:", area)

```

Order of operation in Arithmetic logic:

*Follows Bodmas


```
.5]: #Bodmas
result = (2+5)*10
print(result)

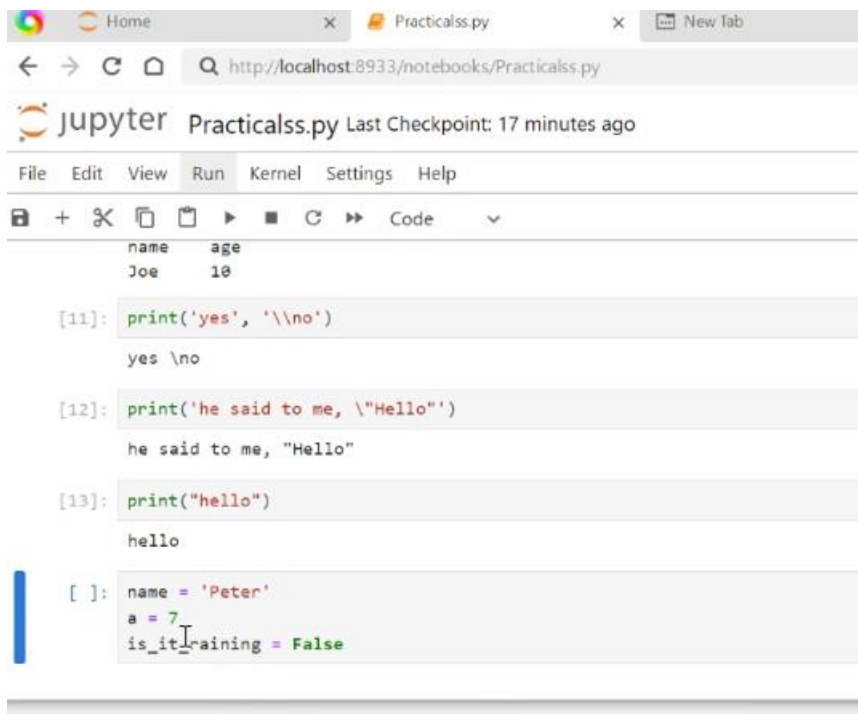
70
```

XX

Tuesday – 10 March 2026

*Anaconda Navigator App install (Has Jupyter Notebook which is similar to the Python IDLE).

Recap of Modules 1 & 2.



```
name    age
Joe      10

[11]: print('yes', '\\no\\')
      yes \\no

[12]: print('he said to me, \\\"Hello\\\"')
      he said to me, "Hello"

[13]: print("hello")
      hello

[ ]:  name = 'Peter'
      a = 7
      is_it_raining = False
```

Arithmetic operation example:



```
[2]: selling_price = 200
      perc = 0.10
      discount = selling_price * perc
      total = selling_price - discount
      print("The selling price is=", selling_price, "@10%\n", "Therefore the price after discount is=", total,
            "Where the discount amount is=",discount)
```

```
The selling price is= 200 @10%
Therefore the price after discount is= 180.0 Where the discount amount is= 20.0
```

```
[ ]:
```

```
selling_price = 200
perc = 0.10
discount = selling_price * perc
total = selling_price - discount
print("The selling price is=", selling_price, "@10%\n", "Therefore
the price after discount is=", total,
      "Where the discount amount is=",discount)
```

Or

```
[6]: selling_price = float(input("Enter The Selling Price:"))
      perc = 0.10
      discount = selling_price * perc
      total = selling_price - discount
      print("The selling price is=", selling_price, "\t@10%")
      print("Therefore the price after discount is=", total,
            "Where the discount amount is=",discount)

Enter The Selling Price: 600
The selling price is= 600.0      @10%
Therefore the price after discount is= 540.0 Where the discount amount is= 60.0
```

```
selling_price = float(input("Enter The Selling Price:"))
perc = 0.10
discount = selling_price * perc
total = selling_price - discount
print("The selling price is=", selling_price, "\t@10%")
print("Therefore the price after discount is=", total,
      "Where the discount amount is=",discount)
```

The division revision:

```
[10]: total_cost = float(input("Enter The Total cost:"))
      unit_num = float(input("Enter The number of units"))
      avg_cost = (total_cost/unit_num)
      print("The average cost per unit is:", avg_cost)
```

```
Enter The Total cost: 500
Enter The number of units 81
The average cost per unit is: 6.172839506172839
```

```
[ ]:
```

```
total_cost = float(input("Enter The Total cost:"))
unit_num = float(input("Enter The number of units"))
avg_cost = (total_cost/unit_num)
print("The average cost per unit is:", avg_cost)
```

Using the above example using the fstream~

```
[11]: total_cost = float(input("Enter The Total cost:"))
      unit_num = float(input("Enter The number of units"))
      avg_cost = (total_cost/unit_num)
      print(f"The average cost per unit is: {avg_cost}")
```

```
Enter The Total cost: 5000
Enter The number of units 600
The average cost per unit is: 8.333333333333334
```

```
[ ]:
```

```
total_cost = float(input("Enter The Total cost:"))
unit_num = float(input("Enter The number of units"))
avg_cost = (total_cost/unit_num)
print(f"The average cost per unit is: {avg_cost}")
```

The float method:
(//)-ensures the result is rounded off

```
[17]: items = float(input("Enter Total items:"))
      box_capacity = 12
      fullboxes = (items//box_capacity)
      print(f"The number of fullboxes for shipping {fullboxes}")

Enter Total items: 1000
The number of fullboxes for shipping 83.0
```

```
[ ]:
```

```
items = float(input("Enter Total items:"))
box_capacity = 12
fullboxes = (items//box_capacity)
print(f"The number of fullboxes for shipping {fullboxes}")
```

The modulus operation:

```
[20]: items = float(input("Enter Total items:"))
      box_capacity = 12
      fullboxes = (items//box_capacity)
      remaining_items = (items%box_capacity)
      print(f"The number of fullboxes for shipping {fullboxes} and there are {remaining_items} remaining items")

Enter Total items: 1000
The number of fullboxes for shipping 83.0 and there are 4.0 remaining items
```

```
[ ]:
```

```
items = float(input("Enter Total items:"))
box_capacity = 12
fullboxes = (items//box_capacity)
remaining_items = (items%box_capacity)
print(f"The number of fullboxes for shipping {fullboxes} and there are {remaining_items} remaining items")
```

The exponentiation example in class, done by me:

```
•[24]: principle = int(input("principle amount:\n"))
      total_balance = (principle * (1.05) ** 2) #= principle * (1.05) ** years
      print(f"Principle Amount: {principle} \t Interest Rate: 5% \n 2 year period\t Total accumulated is:{total_balance}")

principle amount:
100
Principle Amount: 100    Interest Rate: 5%
2 year period    Total accumulated is:110.25
```

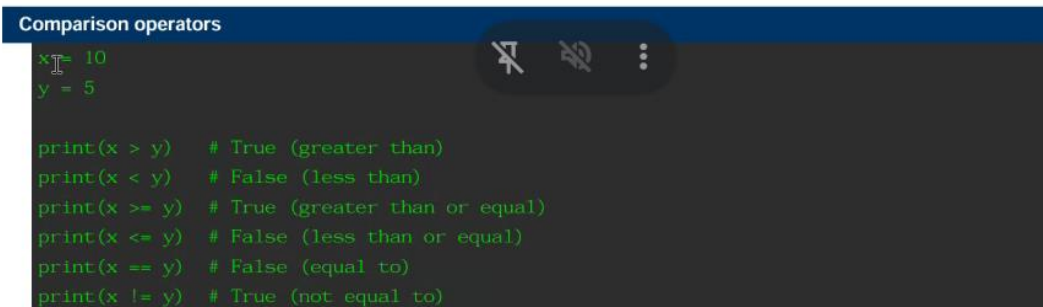
```
principle = int(input("principle amount:\n")) total_balance =  
(principle * (1.05) ** 2) #= principle * (1.05) ** years  
print(f"Principle Amount: {principle} \t Interest Rate: 5% \n 2 year  
period\t Total accumulated is:{total_balance}")
```

BOOLEANS (Today's topic)

Comparison operators: “False” and “True” must be written as Capital not small letters.

Comparison Operators

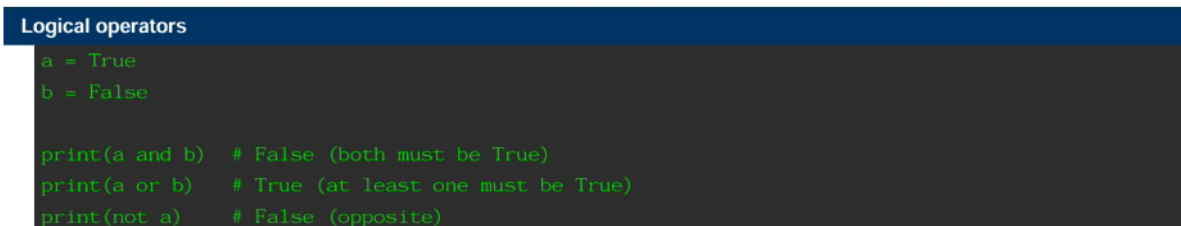
Comparison operators compare two values and return a Boolean result:

A screenshot of a code editor with a dark background. The title bar of the editor window says "Comparison operators". The code inside is as follows:

```
x = 10  
y = 5  
  
print(x > y)    # True (greater than)  
print(x < y)    # False (less than)  
print(x >= y)   # True (greater than or equal)  
print(x <= y)   # False (less than or equal)  
print(x == y)   # False (equal to)  
print(x != y)   # True (not equal to)
```

Logical operators:

Logical operators combine Boolean values:

A screenshot of a code editor with a dark background. The title bar of the editor window says "Logical operators". The code inside is as follows:

```
a = True  
b = False  
  
print(a and b)  # False (both must be True)  
print(a or b)   # True (at least one must be True)  
print(not a)    # False (opposite)
```

i.e.

AND (Both conditions must be true):

```
7]: principle = 10
total_balance = 12
print(principle==10 and total_balance==12)
```

True

OR (One or both must be true):

```
28]: principle = 10
total_balance = 12
print(principle==11 or total_balance==12)
```

True

```
principle = 10
total_balance = 12
print(principle==11 or total_balance==11)
```

False

NOT:

```
: principle = True
total_balance = False
print( total_balance== True)
```

False

This is correct

The Not:

```
: principle = True
total_balance = False
print(not total_balance== True)
```

True

If statement template:

3.2 Conditional Execution

Conditional statements allow your program to make decisions based on conditions.

if-else statement

```
age = 18

if age >= 18:
    print("You are an adult")
else:
    print("You are a minor")
```

```
age = 18
if age >= 18:
    print("You are an adult")
else:
    print("You are a minor")
```

.....

Group 3 Meeting

- Went through the presentation slides, Ushanka created and we divided roles amongst the team.
 - My part is the Makro project
 - Meeting was 30 minutes
 - Practice presentation on Thursday after class (with cameras on)
-

11 March 2026 – Wednesday

```
[13]: # SETA GRADING SYSTEM
# mark using input method      condition      what must then happen      else      (m>50 competent, Less than NYC)
mark = int(input('Enter your mark(0-100):'))
if mark>=50:
    print(f'the mark is {mark} and its Competent')
elif mark > 100:
    print('Invalid mark')
else:
    print(f'The mark is {mark} and its NYC')
```

Enter your mark(0-100): 102
the mark is 102 and its Competent

```
mark = int(input("Enter your grade:"))
if mark >= 50 and mark <=100:
    print("you're competent")
elif mark > 100:
    print("This score doesn't exist, enter a mark between 0 - 100")
else:
    print("not competent")
```

Class Example 1 – To do:

```
: # Stock reorder Level: stock input()to enter current stock Level
# out of stock_ urgent order needed
# if its below 10 (low stock Level)
# else stock is normal
```

Here is the answer:

```
[ ]: # Stock reorder level: stock input() to enter current stock level
# out of stock_ urgent order needed
# if its below 10 (Low stock level)
# else stock is normal
stock_level = int(input('Enter current stock level:'))
if stock_level <= 0:
    print('Out of stock! Urgent order required')
elif stock_level < 10:
    print(f'Low stock level! Current stock : {stock_level} units. Please place an order soon!')
elif stock_level < 50:
    print(f'Stock level is normal! Current stock: {stock_level} units.')
else:
    print(f'Stock level high! Current stock: {stock_level} units. |')
```

Class Example 2:

```
[ ]: #Customer status check
#points at least 1000 point (Platinum customer), 500 point (Gold customer) 200 point (silver customer) else (basic)
```

#Customer status check

#points at least 1000 points - Platinum, 500 points - gold, 200 points silver & else is basic

```
points_level = int(input("Enter Your Points To See Your Tier"))
#Customer status check
#points at least 1000 points - Platinum, 500 points - gold, 200 points silver & else is basic
if points_level == 0:
    print("Not a customer")
elif points_level >= 200:
    print(f"Silver Tier Customer - {points_level} points")
elif points_level >= 500:
    print(f"Gold Tier Customer - {points_level} points")
elif points_level >= 1000:
    print(f"Platinum Tier Customer - {points_level} points")
else:
    print("Basic Tier Customer")
```

```
points_level = int(input("Enter Your Points To See Your Tier"))
#Customer status check
#points at least 1000 points - Platinum, 500 points - gold, 200
points silver & else is basic
if points_level == 0:
    print("Not a customer")
elif points_level >= 200:
    print(f"Silver Tier Customer - {points_level} points")
elif points_level >= 500:
    print(f"Gold Tier Customer - {points_level} points")
elif points_level >= 1000:
    print(f"Platinum Tier Customer - {points_level} points")
```



```
else:  
    print("Basic Tier Customer")
```

Class Example 3:

#Choose option for this banking app and each choice should be interactive.

```
[13]: #Simple Banking App  
# 1. Check Balance, 2. Deposit Money, 3.Withdraw Money & 4. Exit choice  
balance = 1000  
print ("Select option: \n1. Check Balance \n2. Deposit Money \n3. Withdraw Money \n4. Exit")  
choice = input("Choose Your option [1 - 4]:")  
  
if option == "1":  
    print(f"Your current balance is: {balance}")  
elif choice == "2":  
    amount = float(input('Enter Deposit Amount: R' ))  
    balance = (balance + amount)  
    print('Deposit successful! New balance: R', balance)  
elif choice == "3":  
    amount = float(input('Enter Withdrawal Amount:R' ))  
    if amount <= balance:  
        balance = balance-amount  
    print('Withdrawal Successfull New balance is R', balance)  
else:  
    print('Not enough money')
```

```
Select option:  
1. Check Balance  
2. Deposit Money  
3. Withdraw Money  
4. Exit  
Choose Your option [1 - 4]: 2  
Enter Deposit Amount: R 2500  
Deposit successful! New balance: R 3500.0
```

```
#Simple Banking App:  
# 1. Check Balance, 2. Deposit Money, 3.Withdraw Money & 4. Exit  
choice  
balance = 1000  
print ("Select option: \n1. Check Balance \n2. Deposit Money \n3.  
Withdraw Money \n4. Exit")  
choice = input("Choose Your option [1 - 4]:")
```

```
if option == "1":
    print(f"Your current balance is: {balance}")
elif choice == "2":
    amount = float(input('Enter Deposit Amount: R' ) )
    balance = (balance + amount)
    print('Deposit successful! New balance: R', balance)
elif choice == "3":
    amount = float(input('Enter Withdrawal Amount:R') )
    if amount <= balance:
        balance = balance-amount
    print('Withdrawal Successfull New balance is R', balance)
else:
    print('Not enough money')
```

*Pay Attention to the If statement indentation, it should be exactly like the picture.

LOOPS

The while loop

The while Loop

The while loop executes as long as a condition is True:

```
while loop
count = 0

while count < 5:
    print(count)
    count += 1 # Don't forget to update!

# Output: 0 1 2 3 4
```

Here's an example:

```
[16]: count = 0
      while count <5:
          print(count)
          count +=1

0
1
2
3
4
```

```
count = 0
while count <5:
    print(count)
    count +=1
```

Example 2:

```
•[18]: #example: using while loop print number from 1 to 10
count = 1 #start condition
while count <11: #limit condition
    print(count) #display condition
    count +=1 #Increment condition
```

```
1
2
3
4
5
6
7
8
9
10
```

```
#example: using while loop print number from 1 to 10
count = 1 #start condition
while count <11: #limit condition
    print(count) #display condition
    count +=1 #Increment condition
```

Or

```
[19]: #example: using while loop print number from 1 to 10
count = 1 #start condition
while count <= 10: #limit condition
    print(count) #display condition
    count +=1 #Increment condition
```

```
1
2
3
4
5
6
7
8
9
10
```

Example:

```
#example 3: using while loop print even number between 20 to 100
name_variable = 20 #start condition
while name_variable <= 100: #limit condition
    print(name_variable) #display condition
    count +=2 #Increment condition
```

Negries's Version:

```
number = 20
while number<100:
    if number % 2 == 0:
        print(number)
        number += 1
```

For loop:

The for loop iterates over a sequence:

for loop

```
# Using range()
for i in range(5):
    print(i) # 0 1 2 3 4

# Iterating over a list
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)
```

range() function

```
# range() variations
range(5)      # 0, 1, 2, 3, 4
range(2, 5)   # 2, 3, 4
range(0, 10, 2) # 0, 2, 4, 6, 8 (step of 2)
```

```
[13]: a = range(10)
      print(a)
      range(0,10)
```

```
a = range(10)
print(a)
```

Another example:

```
: for numbers in range(5):  
    print(numbers)
```

```
0  
1  
2  
3  
4
```

```
for numbers in range (5):  
    print(numbers)
```

Integrating what we have learned with lists:

*List in python are the Square brackets []. It takes on different datatypes.

```
] : fruits = ['orange', 100, 10.4, True]
```

```
[19]: fruits = ['orange', 100, 10.4, True]  
      print(fruits)
```

```
['orange', 100, 10.4, True]
```

```
[20]: for elements in fruits:  
      print(elements)
```

```
orange  
100  
10.4  
True
```

```
[ ]:
```

Or

```
[23]: colors = ('green', 'red', 'yellow')  
      for c in colors:  
          print(c)
```

```
green  
red  
yellow
```

Here's the For loop stepping sequence:

```
[24]: for n in range(2,11,2): # start at 2, go upto 11 and finally step by 2  
      print(n)
```

```
2  
4  
6  
8  
10
```

```
for n in range(2,11,2): # start at 2, go upto 11 and finally step by 2
    print(n)
```

My Examples For-Loop:

1. For-Loop + Lists:

```
#Colours of the Rainbow to iterate and display using the for loop from the list
rainbow = ["Red","Orange","Yellow","Green","Blue","Indigo","Violet"]
for values in rainbow:
    print(values)
```

```
Red
Orange
Yellow
Green
Blue
Indigo
Violet
```

```
#Colours of the Rainbow to iterate and display using the for loop
from the list
rainbow = ["Red","Orange","Yellow","Green","Blue","Indigo","Violet"]
for values in rainbow:
    print(values)
```

2. For-Loop + Ranges

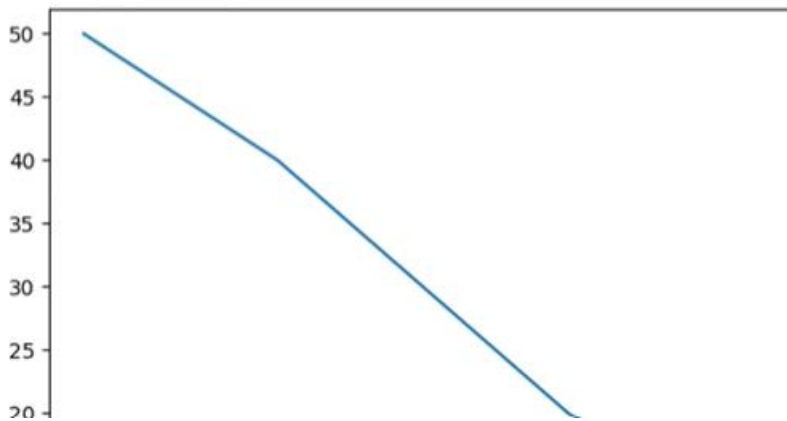
```
[42]: for numbers in range(5,50,5):
      print(numbers)

5
10
15
20
25
```

```
for numbers in range(5,50,5):
    print(numbers)
```

[FOR FUTURE in Python]

```
[31]: Price = [10, 20, 40, 50]
      Demand = [100, 80, 50, 30]
      from matplotlib import pyplot as plt
      import pandas as pd
      plt.plot(Demand, Price)
      plt.show()
```



```
2]: Price = [10, 20, 40, 50]
    Demand = [100, 80, 50, 30]
    from matplotlib import pyplot as plt
    import pandas as pd
    plt.plot(Demand, Price)
    plt.xlabel('Demand of bread')
    plt.ylabel('Price')
    plt.show()
```

```
Price = [10, 20, 40, 50]
Demand = [100, 80, 50, 30]
from matplotlib import pyplot as plt
import pandas as pd
plt.plot(Demand, Price)
plt.xlabel("Price")
plt.ylabel("Demand of bread")
plt.show()
```

Homework exercise:

range 1 -15 if the number is
divisible 3, output fizz if by 5
output buzz by both and then
combine the two words. else output
the exact number

My Attempt:

```
[86]: #The Fizz-Buzz Challenge

for Numbers in range (1,16):
    if Numbers % 3 == 0 and Numbers % 5 ==0:
        print("FizzBuzz")
    elif Numbers % 3 == 0:
        print("Fizz")
    elif Numbers % 5 == 0:
        print("Buzz")
    else:
        print(Numbers)
```

```
1
2
Fizz
4
Buzz
Fizz
7
8
Fizz
Buzz
11
Fizz
13
14
FizzBuzz
```

```
#The Fizz-Buzz Challenge
```

```
for Numbers in range (1,16):
```



```

if Numbers % 3 == 0 and Numbers % 5 == 0:
    print("FizzBuzz")
elif Numbers % 3 == 0:
    print("Fizz")
elif Numbers % 5 == 0:
    print("Buzz")
else:
    print(Numbers)

```

12 March 2026

*Recap session of all we did yesterday

*WHILE loop with a pin example:

```

[*]: # Banking System While Loop - Ask for a pin until the correct is entered!
correct_pin = "1234"
entered_pin = input('Enter Password:')
while entered_pin != correct_pin:
    print('Incorrect pin! Please try again')
    entered_pin = input('Enter Password:')
print('Password accepted! Welcome')

Enter Password: 111
Incorrect pin! Please try again
Enter Password: 1243
Incorrect pin! Please try again
Enter Password: 

```

Example 2:

```

[23]: quantity = int(input("Enter quantity:"))
while quantity <= 0:
    print("Quantity May not be 0 or below")
    quantity = int(input("Please Enter quantity:"))
print(f"Successful! The quantity is: {quantity}")

Enter quantity: 1
Successful! The quantity is: 1

```

```

quantity = int(input("Enter quantity:"))
while quantity <= 0:
    print("Quantity May not be 0 or below")
    quantity = int(input("Please Enter quantity:"))

```

```
print (f"Successful! The quantity is: {quantity}")
```

Example (While Loop):

```
[7]: # While Loop to retard stock until its zero
stock = 5
while stock > 0:
    print('items sold Remaining stock:', stock)
    stock -= 1
print('Stock is finished. Reorder needed!')
```

```
items sold Remaining stock: 5
items sold Remaining stock: 4
items sold Remaining stock: 3
items sold Remaining stock: 2
items sold Remaining stock: 1
Stock is finished. Reorder needed!
```

Example (While Loop):

```
[9]: # Ride hailing app
driver_found = False
while not driver_found:
    print('searching for a driver...')
    user_input = input('Drive found ? (yes/no)')
    if user_input == 'yes':
        driver_found = True
print('driver is on the way')
```

```
searching for a driver...
Drive found ? (yes/no) yes
driver is on the way
```

Lists-

(Indexing example)

```
: List = ['cement', 'bricks', 'tiles']
# Accessing Elements (index)
print(List[0])
```

```
[26]: List = ["Air", "Water", "Fire", "Earth"]
print(List[1])
```

```
Water
```

Last element on the list is using -1:

```
7]: List = ["Air", "Water", "Fire", "Earth"]
    print(List[-1])
```

Earth

Slicing:

```
•[33]: #Slicing
       List = ["Air", "Water", "Fire", "Earth", "Metal"]
       print(List[1:3])

       ['Water', 'Fire']
```

The last two numbers:

```
] : # Slicing : Get the three trans for audit
    transactions = [1500, -200, -350, 800, 120]
    print(transactions[3:])

    [800, 120]
```

Skipping a number by 2 from 0:

```
[24]: # Slicing : Get the three trans for audit
      transactions = [1500, -200, -350, 800, 120]
      print(transactions[0::2])

      [1500, -350, 120]
```

Reversing numbers:

```
# reverse list elements
print(transactions[::-1])

[1500, -350, 120]
[120, 800, -350, -200, 1500]
```

Appending a List:

```
: products = ['cement', 'brick']
  #add a new product
  products.append('Tiles')
  for i in products:
      print(i)

  cement
  brick
  Tiles
```

*Specifying an index and inserting:

```
[33]: # insert at a specific place
products.insert(0, 'Paint')
print(products)

['Paint', 'cement', 'brick', 'Tiles']
```

Removing an element from a list:

```
# insert at a specific place
products.insert(0, 'Paint')
# remove an item
products.remove('Tiles')
print(products)

['Paint', 'cement', 'brick', 'Tiles']
```

Sorting:

```
# sort
products.sort()
print(products)

['Paint', 'Paint', 'Paint', 'Paint', 'brick', 'cement']
```

```
[38]: trans = [3500, -500, -200, 800]
trans.sort()
print('sorted', trans)

sorted [-500, -200, 800, 3500]
```

Counting elements in the list:

```
1): # count number of entries
print('Number of transactions is', len(trans))

Number of transactions is 4
```

```
11): # count number of entries
length = len(trans)
# sum
print(length)
```

```
[58]: #scenario
students = ['Nomsa', 'Thabo']
students.append('Sibongile')
students.insert(1,100)
students.remove(100)
print(students)
print('The length of this list is:', len(students))
```

['Nomsa', 'Thabo', 'Sibongile']
The length of this list is: 3

MODULE 4

Module 4: Functions, Tuples, Dictionaries, Exceptions

Learning Objectives

In this module, you will learn:

- How to define and use functions
- What tuples are and how to use them
- How to work with dictionaries
- How to handle exceptions

Functions:(You know these as Methods)

4.1 Functions

A function is a reusable block of code that performs a specific task.

Function definition

```
# Defining a function
def greet(name):
    """Say hello to the given name."""
    print(f"Hello, {name}!")

# Calling the function
greet("Alice") # Hello, Alice!
greet("Bob")   # Hello, Bob!
```

Here's a function structure:

```
# syntax of a function
def function_name (parameter):
    #code
    return value
```

syntax of a function

```
def function_name (parameter):
```

```
#code
```

```
return value
```

Example:

```
# syntax of a function
def greet (name):
    print(f'Hello, {name}')
greet('Joseph')
```

Hello, Joseph

```
def greeet (name):
    print(f"Hello, {name}")
greeet("Mbulelo")
```

Hello, Mbulelo

```
def greet (name):
    print(f"Hello, {name}")
```

```
greet("Mbulelo")
```

```
def greet (name,age):  
    print(f"Hello, {name}, your age is {age}")  
greet("Mbulelo",20)
```

Hello, Mbulelo, your age is 20

Example:

```
def stock_value(quantity,price):  
    return quantity*price  
stock_value(1000,20)
```

20000

Or

```
def stock_value(quantity,price):  
    return quantity*price  
value = stock_value(1000,20)  
print (f'The total monetary value of the stock at hand is {value}')
```

The total monetary value of the stock at hand is 20000

Example (Uber Price calculator):

```
#Function to calculate Uber fare  
def uber_calculator (distance, price_per_km):  
    return distance * price_per_km  
fare = uber_calculator(40,10)  
print (f"The price for 10KM is R{fare}")
```

The price for 10KM is R400

```
#Function to calculate Uber fare  
def uber_calculator (distance, price_per_km):  
    return distance * price_per_km  
fare = uber_calculator(40,10)  
print (f"The price for 40KM is R{fare}")
```

Example:

```
# Write a function that calculates Bank Account Balance
# 2 parameter (deposit withdrawal)
def bank_calc (deposit, withdrawal):
    return deposit - withdrawal
balance = bank_calc(1000,500)
print(f"The bank balance is {balance}")
```

The bank balance is 500

Write a function that calculates Bank Account Balance

2 parameter (deposit withdrawal)

```
def bank_calc (deposit, withdrawal): return deposit - withdrawal
balance = bank_calc(1000,500) print(f"The bank balance is {balance}")
```

Sidd's example:

```
[72]: # Write a function that calculate Bank Account Balance
# 2 parameter (deposit withdrawal)
balance1 = float(input('Enter the account balance:'))
def account_balance(balance1, deposit, withdrawal):
    balance1+=deposit
    balance1-=withdrawal
    print('Balance is', balance1)
account_balance(balance1,1000, 500)
```

```
Enter the account balance: 2000
Balance is 2500.0
```

Or alt:


```

: # Write a function that calculate Bank Account Balance
# 2 parameter (deposit withdrawal)
balance1 = float(input('Enter the account balance:'))
deposit = float(input('Enter the deposit amount:'))
Withdrawal = float(input('Enter the Withdrawal amount:'))
def account_balance(balance1, deposit, withdrawal):
    balance1+=deposit
    balance1-=withdrawal
    print('Balance is', balance1)
account_balance(balance1,deposit, Withdrawal)

```

Enter the account balance:

```

Enter the account balance: 2000
Enter the deposit amount: 1000
Enter the Withdrawal amount: 500
Balance is 2500.0

```

Another Example of a grading system:

```

#Grading System: If mark is > (75 return distinction), (mark >= 50 return pass) else it must be a fail
def grading_system (grade):
    if grade >= 75:
        return "Distinction!"
    elif grade >= 50:
        return "Pass!"
    else:
        print("Failed!")
grade = int(input(f"Enter Grade:"))
result = grading_system(grade)
print(f"Your Grade is {grade}% {result}")

```

```

Enter Grade: 75
Your Grade is 75% Distinction!

```

```

#Grading System: If mark is > (75 return distinction), (mark >= 50
return pass) else it must be a fail
def grading_system (grade):
    if grade >= 75:
        return "Distinction!"
    elif grade >= 50:
        return "Pass!"
    else:
        print("Failed!")
grade = int(input(f"Enter Grade:"))
result = grading_system(grade)
print(f"Your Grade is {grade}% {result}")

```

Katleho's example:

```
num_students = int(input('Enter number of students:'))
def grading_system(mark):
    if mark >=75:
        return 'Distinction'
    elif mark >=50:
        return 'Pass'
    else:
        return 'Fail'
for i in range(num_students):
    mark = float(input('Enter student mark:'))
    result = grading_system(mark)
    print('Result', result)
```

Messaging example:

```
#Messaging system: takes one parameter, then return message,
#Hello {Name}, your order is ready for collection
name = input(f"Enter Name:")
def messaging_system(name):
    name_person1 = print(f"Hello {name} your order is ready for collection")
    return name_person1
messaging_system(name)
```

```
Enter Name: Michael
Hello Michael your order is ready for collection
```

```
#Messaging system: takes one parameter, then return message,
#Hello {Name}, your order is ready for collection
name = input(f"Enter Name:")
def messaging_system(name):
    name_person1 = print(f"Hello {name} your order is ready for
collection")
    return name_person1
messaging_system(name)
```

Courier system:

```
# Courier System delivery time distance / speed
name = input('Enter name:')
distance = float(input('Enter Delivery distance (Km):'))
speed = float(input('Enter speed (km/h):'))
def delivery_time(distance, speed):
    time = f"The time is: {distance/speed} in hours"
    return time
delivery_time(distance, speed)
```

```
Enter name: Mbulelo
Enter Delivery distance (Km): 100
Enter speed (km/h): 120
'The time is: 0.8333333333333334 in hours'
```

Courier System delivery time distance / speed

```
name = input('Enter name:') distance = float(input('Enter Delivery
distance (Km):')) speed = float(input('Enter speed (km/h):')) def
delivery_time(distance, speed): time = f"The time is:
{distance/speed} in hours" return time delivery_time(distance, speed)
```

Or simply:

```
] def delivery_time(distance, Speed):
    return distance/speed
print(delivery_time(60, 80))
```

0.75

alt:

```
def delivery_time(distance, Speed):
    return distance/speed
time = (delivery_time(60, 80))
print(f'The estimated delivery time is {time}')
```

The estimated delivery time is 0.75

DEFAULT FUNCTIONS (Functions without parameters):

```
# Functions without parameter
def show_menu():
    print('1. check balance:')
    print('2. Deposit')
show_menu()
```

1. check balance:
2. Deposit

And also:

```
#Default parameter
def fuel_cost(distance, rate=20):
    return distance*rate
fuel_cost(10)
```

200

\With calculation:

```
] def sales_summary(quantity, price):
    total = quantity * price
    vat = total* 0.15
    return total, vat
print(sales_summary(10, 50))

(500, 75.0)
```

Global & Local Variables (Scope)

Scope

Variable Scope

Local variables: Defined inside a function, only accessible within it

Global variables: Defined outside functions, accessible everywhere

Use the 'global' keyword to modify a global variable inside a function.

Scope example

```
x = 10 # Global variable

def my_function():
    x = 5 # Local variable (different from global x)
    print(x) # 5
```

Global Variable and Local Variable:

```

> x = 10 # Global variable
def my_function():
    x = 5 # Local variable (different from global x)
    print(x) # 5
my_function()
print(x)
5
10

```

13 March 2026 - Friday

Tuples, Dictionaries & Exceptions

*Tuples use () parenthesis.

4.2 Tuples

A tuple is an ordered, immutable collection of items.

Tuples

```

# Creating tuples
coordinates = (10, 20)
rgb = (255, 128, 0)
single = (42,) # Single element tuple needs comma

# Accessing elements
print(coordinates[0]) # 10
print(rgb[-1]) # 0

# Tuple unpacking
x, y = coordinates
print(x, y) # 10 20

```

```

[5]: location = (-26.2041, 28.0473)
print('Latitude:', location[0])
print('Longitude:', location[1])

```

```

Latitude: -26.2041
Longitude: 28.0473

```

```

[6]: for element in location:
    print(element)

```

```

-26.2041
28.0473

```

*Tuples are great for read only data

*Besides the types of bracket, what's the difference between a List and a Tuple? Cause they seem the same.

-Tuples Cannot be manipulated, they are fixed, but lists can be, they're flexible (sorting, rearranged)

*How do you decide which to use:

- Permanent Information that doesn't need to change you use Tuples.
Flexibility you use Lists.

DICTIONARIES

We use {} curly braces for dictionaries

A dictionary is an unordered collection of key-value pairs.

Dictionaries

```
# Creating a dictionary
person = {
    "name": "Alice",
    "age": 30,
    "city": "New York"
}

# Accessing values
print(person["name"])    # Alice
print(person.get("age")) # 30

# Adding/modifying
person["email"] = "alice@example.com"
person["age"] = 31
```

```
: # Dictionaries
account = {
}
```

```
car = {'driver': 'James', 'model': 'Toyota', 'rating': 4.8}
```

250

```
# Dictionaries
account = {
    'owner' : 'Peter',
    'balance': 1500,
    'Type': 'Saving'
}
```

```
[4]: car = {'driver': 'James', 'model': 'Toyota', 'rating' : 4.8}
     print(car['driver'])
```

James

*A dictionary is like a paired key & value system, you have the key it fetches the corresponding value, i.e.

```
car = {'driver': 'James', 'model': 'Toyota', 'rating' : 4.8}
print(car['model'],car['rating'])
```

Toyota 4.8

-The first value pulls the second value.

-This is an example of the “For loop” retrieving the initial keys and not the assigned value to it.

```
# Dictionaries
account = {
    'owner' : 'Peter',
    'balance': 1500,
    'Type': 'Saving'
}
for infor in account:
    print(infor)
```

owner
balance
Type

Here's a way to add information to the Dictionary:

```
# Dictionaries
account = {
    'owner' : 'Peter',
    'balance': 1500,
    'Type': 'Saving'
}
# ADDING A NEW ITEM
account['Location'] = 'Jhb'
print(account)

{'owner': 'Peter', 'balance': 1500, 'Type': 'Saving', 'Location': 'Jhb'}
```

Dictionaries

```
account = { 'owner' : 'Peter', 'balance': 1500, 'Type': 'Saving' }
```

ADDING A NEW ITEM

```
account['Location'] = 'Jhb'
print(account)
```

Modifying a dictionary item:

```
# Modifying Existing information
account['balance'] = 2500
print(account)

{'owner': 'Peter', 'balance': 2500, 'Type': 'Saving', 'Location': 'Jhb'}
```

Deleting a dictionary item:

```
# Delete inf from a dictionary
del account['Location']
print(account)

{'owner': 'Peter', 'balance': 2500, 'Type': 'Saving'}
```

Using the WHILE loop:


```

*]: contact = {}
   while True:
       print('\n ____ CONTACT BOOK ____') I

```

```

: contact = {}
  while True:
      print('\n ____ CONTACT BOOK ____')
      print('1. Add Contact \n2. Search Contact \n3. View All \n4 Delete contact \n5. Exit')

```

```

I ____ CONTACT BOOK ____
1. Add Contact
2. Search Contact
3. View All
4 Delete contact
5. Exit

```



Exception Handling

Exceptions are errors that occur during program execution. You can handle them gracefully using try-except blocks.

Exception handling

```

# Basic exception handling
try:
    number = int(input("Enter a number: "))
    result = 10 / number
    print(f"Result: {result}")
except ValueError:
    print("That's not a valid number!")
except ZeroDivisionError:
    print("Cannot divide by zero!")

```

```

try:
    number = int(input('Enter a number:'))
    result = 10/number
    print(f'Result: {result}')
except ValueError:
    print('That not a valid number') I
except ZeroDivisionError:
    print('Cannot divide by zero')

```

Result:

```

try:
    number = int(input('Enter a number:'))
    result = 10/number
    print(f'Result: {result}')
except ValueError:
    print('That not a valid number')
except ZeroDivisionError:
    print('Cannot divide by zero')

```

```

Enter a number: str
That not a valid number

```

Using more of the try - exception handling:

```

: balance = 1000
try:
    amount = float(input('Enter amount to Withdraw:'))
    if amount <= 0:
        raise ValueError('Amount must be positive')
    if amount > balance:
        raise ValueError('Insufficient Fund!')
except ValueError as e:
    print('Error', e)
else:
    balance -= amount
    print(f'Withdrawal successful! New bal: {balance}')
    print('Thank you for doing business with us')

```

```

Enter amount to Withdraw: -200
Error Amount must be positive

```

Instead of `balance -= amount` we can use `balance = balance - amount` (they are the same).

Exercise As a Recap in class:

```

# ask for a number and decide wether its even or odd and display again if it is neg, positive or zero
number = int(input("Enter number(1-10):"))
if number % 2 == 0:
    print ("This is an Even number")
else:
    print ("This is an Odd Number")

if number < 0:
    print ("This is a negative Number")
elif number == 0:
    print ("This is equal to zero")
else:
    print ("This is a positive Number")

```

```

Enter number(1-10): 2
This is an Even number
This is a positive Number

```

ask for a number and decide wether its even or odd and display again if it is neg, positive or zero

```

number = int(input("Enter number(1-10):")) if number % 2 == 0: print ("This is an Even number") else: print ("This is an Odd Number")

if number < 0: print ("This is a negative Number") elif number == 0: print ("This is equal to zero") else: print ("This is a positive Number")

```

Exercise 2:

```

#number guessing game. the program has a secret number between 1 and 10
#the user must keep on guessing until they get it right
#Give hints (too high/too Low)

```

```
: #number guessing game. the program has a secret number between 1 and 10
#the user must keep on guessing until theyget it right
#Give hints (too high/too low)

secret_num = 5
while True:
    user_num = int(input("Please Enter Number:"))
    if user_num == secret_num:
        print(f"Congratulations you got it! The number was {secret_num}")
        break
    elif user_num < secret_num:
        print(f"Sorry, Number is Too Low!")
    else:
        print(f"Out of bounds, Sorry, Number is Too High!")
```

```
Please Enter Number: 2
Sorry, Number is Too Low! The correct number was 5
Please Enter Number: 7
Out of bounds, Sorry, Number is Too High! The correct number was 5
Please Enter Number: 5
Congratulations you got it! The number was 5
```

```
#number guessing game. the program has a secret number between 1 and
10
#the user must keep on guessing until theyget it right
#Give hints (too high/too low)

secret_num = 5
while True:
    user_num = int(input("Please Enter Number:"))
    if user_num == secret_num:
        print(f"Congratulations you got it! The number was
{secret_num}")
        break
    elif user_num < secret_num:
        print(f"Sorry, Number is Too Low!")
    else:
        print(f"Out of bounds, Sorry, Number is Too High!")
```

.....

16 March 2026

2. Chapter 2: Building Blocks - Modules, Packages, and Error Handling
3. Chapter 3: The Blueprint - Object-Oriented Programming (OOP)
4. Chapter 4: The Data-Driven Edge - Visualization and Machine Learning
5. Chapter 5: Your First Web Service - A Deep Dive into Flask
6. Chapter 6: Capstone Project: The Intelligent Sales API (In-Class Project)
7. Chapter 7: The Developer's Mindset - PCAP-Style Problem Solving

Lambda Functions (Any number of arguments):

1.2 Lambda Functions: Quick, Anonymous Functions

What are Lambda Functions?

A `lambda` function is a small, unnamed function in Python. It's defined using the `lambda` keyword and can take any number of arguments, but it must contain only one expression. The result of this single expression is what the `lambda` function returns. Think of `lambda` functions as concise, one-line functions for simple tasks that you might use only once.

Syntax

The basic structure of a `lambda` function is:

The syntax: Can have many arguments but a single expression.

Syntax

The basic structure of a `lambda` function is:

```
lambda arguments: expression
```

- `lambda`: This special keyword signals the creation of an anonymous function.
- `arguments`: These are the inputs the function takes, similar to parameters in a regular function. You can have one or more, separated by commas.
- `expression`: This is a single operation or calculation. The `lambda` function evaluates this expression and returns its result. It cannot contain complex statements like `if/else` blocks or loops, but it can use conditional expressions.

An example:

```
[1]: add=lambda a,b: a+b  
     print(add(3,5))  
8
```

Or

```
[2]: #Lambda arguments: expression  
     add=lambda a=3,b=5: a+b  
     print(add(3))  
8
```

And

```
[3]: #Lambda arguments: expression  
     add=lambda a=3,b=5: a+b  
     print(add())  
8
```

What is the use for a lambda function:

Reasoning Behind Lambda Functions

lambda functions are particularly useful when you need a simple function for a short duration, often as an argument to other functions that expect a function (known as higher-order functions), such as `map()`, `filter()`, or `sorted()`. They allow you to write compact code without the overhead of formally defining a function using `def` when that function is only needed for a specific, immediate purpose.

Real-Life Business Example: Calculating Sales Tax

Imagine Joseph runs a small business and needs to quickly calculate the total price of items after adding a 15% sales tax. A lambda function can do this very concisely.

```
: # 15% , calculating total price after tax has been added.  
cal_price = lambda price : price*1.05  
print(cal_price(100))  
105.0
```

Here's another example:

```
[5]: # $1: R16 # convert us$ to Rand
us_to_ZAR = lambda us: us*16
print(us_to_ZAR(10))

160
```

Or

```
7]: # $1: R16 # convert us$ to Rand
current_rate = float(input('Enter the current rate:'))
us_to_ZAR = lambda current_rate: current_rate*16
print(us_to_ZAR(current_rate))
```

Example:

```
[13]: # num1, num2 (multiply two numbers)
Multiply = lambda num1, num2: num1*num2
print(Multiply(10,20))

200
```

```
[4]: Subtract = lambda num_1, num_2: num_1-num_2
print(Subtract(100, 50))

50
```

First Examples (Class examples):

```
[9]: #Give 10% on A product using the Lambda
discount_prod = lambda price: price*0.90
print(f"R{discount_prod(100)} After Discount")

R90.0 After Discount
```

```
[8]: #Give 10% on A product using the Lambda (with user input)
price = int(input("Enter Price"))
discount_prod = lambda price: price*0.90
print(f"R{discount_prod(price)} Price After discount")

Enter Price 100
R90.0 Price After discount
```

#Give 10% on A product using the Lambda discount_prod = lambda price: price*0.90 print(f"R{discount_prod(100)} After Discount")
#Give 10% on A product using the Lambda (with user input) price = int(input("Enter Price"))

```
discount_prod = lambda price: price*0.90
print(f"R{discount_prod(price)} Price After discount")
```

Here's an Example of Modulus:

```
: # Lambda function to check even or odd numbers
is_even = lambda x : x%2 == 0
print(is_even(13))

False
```

This is to show the last number:

```
: # Lambda function returns the last digit of a number
last_digit = lambda num: num%10
print(last_digit(54321))

1
```

THE MAP FUNCTION (iterative elements)

1.3 The `map()` Function: Applying a Function to Every Item

What is the `map()` Function?

The `map()` function is a built-in Python function that applies a specified function to each item in an iterable (such as a list, tuple, or string) and returns a `map` object. This `map` object is an iterator, meaning it generates results one by one as they are needed, rather than creating a complete new list in memory immediately. You can convert it to a list or other iterable type if you need all the results at once.

Syntax

The general syntax for `map()` is:

```
map(function, iterable, ...)
```

Example:

An Iterative-

```
[23]: numbers = [1,2,3,4]
      for x in numbers:
          print(x)

1
2
3
4
```

The Iterative with the Lambda function makes a map.

*From the list, each item is altered using the map function since there is an iteration of each item.

```
[5]: # map(function, iterable)
      numbers = [1,2,3,4]
      result = list(map(lambda x: x*2,numbers ))
      print(result)

[2, 4, 6, 8]
```

Another Example:

```
[29]: names = ['john', 'mary', 'peter']
      capitalize = list(map(lambda n: n.upper(),names))
      print(capitalize)

['JOHN', 'MARY', 'PETER']

[27]: first_name = 'joseph'
      first_name.upper()

[27]: 'JOSEPH'
```

Class Example:

```
: #A map function that gets the length of each word in a list
  colors = ["red","blue","brown","purple"]
  results = list(map(lambda n : len(n),colors))
  print(results)

[3, 4, 5, 6]
```

#A map function that gets the length of each word in a list colors = ["red","blue","brown","purple"]

```
results = list(map(lambda n : len(n),colors))
print(results)
```

Or

```
#A map function that gets the length of each word in a list
colors = ["red","blue","brown","purple"]
results = list(map(len,colors))
print(results)
```

```
[3, 4, 5, 6]
```

```
#A map function that gets the length of each word in a list
colors = ["red","blue","brown","purple"]
results = list(map(len,colors))
print(results)
```

Example:

```
[18]: a = [1,2,3,4,5]
      b = [2,4,6,8,10]
      additional = list(map(lambda x,y: x+y,a,b))
      print(additional)
```

```
[3, 6, 9, 12, 15]
```

```
[20]: d = ["Willow","Ariana","Dua"]
      e = ["Harry", "Leon", "Justin"]
      result = list(map(lambda c,v: v+c, d,e))
      print(result)
```

```
['HarryWillow', 'LeonAriana', 'JustinDua']
```

```
a = [1,2,3,4,5]
b = [2,4,6,8,10]
additional = list(map(lambda x,y: x+y,a,b))
print(additional)
```

```
d = ["Willow","Ariana","Dua"]
e = ["Harry", "Leon", "Justin"]
result = list(map(lambda c,v: v+c, d,e))
print(result)
```

Reasoning Behind `map()`

`map()` is highly useful for performing the same transformation on every element within a collection of data. Instead of writing a traditional `for` loop to iterate through

Python Fundamentals Level 2 - PCAP Certification Training | SCI-Thagorus (Pty) Ltd

a list and modify each item individually, `map()` offers a more concise and often more efficient way to achieve the same outcome. It aligns with the functional programming principle of applying a transformation across a sequence of data, which is a very common task in business data processing, such as `standardizing formats` or applying calculations to a batch of records.

The Filter Function:

1.4 The `filter()` Function: Selecting Items Based on a Condition

What is the `filter()` Function?

The `filter()` function is a built-in Python function that creates an iterator from elements of an iterable for which a given function returns `True`. In simple terms, it allows you to select specific items from a collection that meet a certain condition, effectively filtering out the items that don't satisfy the condition. The result is an iterator, which can be converted to a list if all filtered items are needed at once.

Syntax

The general syntax for `filter()` is:

```
filter(function, iterable)
```

- **function** : This is a function that takes one argument and returns a boolean value (`True` or `False`). `filter()` will apply this function to each item in the `iterable`. Only items for which the function returns `True` will be included in the result. This function is often called a "predicate function."
- **iterable** : This is the collection (e.g., a list, tuple) whose items will be tested by the function.

*If the condition of the filter returns true, it will show/display it.

-In the following, the condition for even numbers it is therefore true:

```
[36]: # filter(function, iterable
      numbers = [1, 2,3,4,5,6,7,8,9,10]
      evens = list(filter(lambda x : x%2==0, numbers))
      print(evens)

      [2, 4, 6, 8, 10]
```

Class Example (#filter out names longer than 3 letters):

```
]: #filter out names longer than 3 letters
names = ["sam", "John", "Amy" ,"Peter"]
select_names = list(filter(lambda v: len(v)<=3,names))
print(select_names)

['sam', 'Amy']
```

```
#filter out names longer than 3 letters
names = ["sam", "John", "Amy" ,"Peter"]
select_names = list(filter(lambda v: len(v)<=3,names))
print(select_names)
```

Filter out empty strings:

```
7]: #filter out empty strings
words = ["apples", " ", "bananas" ," ","kiwis"]
clean_word = list(filter(lambda v: v != " ",words))
print(clean_word)

['apples', 'bananas', 'kiwis']
```

```
#filter out empty strings
words = ["apples", " ", "bananas" ," ","kiwis"]
clean_word = list(filter(lambda v: v != " ",words))
print(clean_word)
```

Using Tuples:

```
] : products = [('laptop', 10), ('Mouse', 0), ('keyboard',4) ]
    in_stock = list(filter(lambda x: x[1]>0, products))
    print(in_stock)
```

```
products = [('laptop', 10), ('Mouse', 0), ('keyboard',4) ]
in_stock = list(filter(lambda x: x[1]>0, products))
print(in_stock)
```

Reasoning Behind `filter()`

`filter()` is fundamental for data selection and cleaning. In business, you frequently need to work with subsets of data that meet specific criteria. For instance, you might need to find all sales orders above a certain value, identify employees in a particular department, or select only active customer accounts. `filter()` provides a clean, readable, and efficient way to perform these selections without writing manual `for` loops with `if` statements, making your code more concise and easier to understand.

Real-Life Business Example: Identifying High-Value Transactions

Joseph needs to review all financial transactions that are above a certain amount, say R1000, as these are considered "high-value" and might require special attention or approval. The `filter()` function can quickly extract these transactions from a list.

Python Fundamentals Level 2 - PCAP Certification Training | SCI-Thagorus (Pty) Ltd

Chapter 2: Building Blocks - Modules, Packages, and Error Handling

2.1 Introduction to Modules and Packages

As your Python programs grow, putting all your code into a single file becomes messy and hard to manage. This is where **modules** and **packages** become essential. They are fundamental tools for organizing your code, making it more readable, reusable, and maintainable. Think of them as a structured filing system for your code, allowing you to categorize and easily locate specific functionalities.

Why is this important for business?

In a business setting, software projects often involve multiple developers, various functionalities (e.g., finance, HR, sales), and a need for consistent code quality. Modules and packages facilitate:

Importing a module:

Syntax for Importing Modules

Python offers several ways to import and use modules:

1. `import module_name` : This is the most common way. It imports the entire module. You then access its contents using `module_name.function_name` or `module_name.variable_name` .

```
import math
print(math.pi) # Accessing the 'pi' variable from the 'math' module
```

2. `from module_name import specific_item` : This imports only specific functions, classes, or variables from a module. You can then use `specific_item` directly without the `module name.` prefix.

Here are some of them:

Import calendar

Import math (math = square roots, powers, pi, trigonometry)

Import random

Import statistics

Import json

The Import Calendar:

2026

January							February							March						
Mo	Tu	We	Th	Fr	Sa	Su	Mo	Tu	We	Th	Fr	Sa	Su	Mo	Tu	We	Th	Fr	Sa	Su
			1	2	3	4							1							1
5	6	7	8	9	10	11	2	3	4	5	6	7	8	2	3	4	5	6	7	8
12	13	14	15	16	17	18	9	10	11	12	13	14	15	9	10	11	12	13	14	15
19	20	21	22	23	24	25	16	17	18	19	20	21	22	16	17	18	19	20	21	22
26	27	28	29	30	31		23	24	25	26	27	28		23	24	25	26	27	28	29
														30	31					

April							May							June							
Mo	Tu	We	Th	Fr	Sa	Su	Mo	Tu	We	Th	Fr	Sa	Su	Mo	Tu	We	Th	Fr	Sa	Su	
			1	2	3	4						1	2	3	1	2	3	4	5	6	7
6	7	8	9	10	11	12	4	5	6	7	8	9	10	8	9	10	11	12	13	14	
13	14	15	16	17	18	19	11	12	13	14	15	16	17	15	16	17	18	19	20	21	
20	21	22	23	24	25	26	18	19	20	21	22	23	24	22	23	24	25	26	27	28	
27	28	29	30				25	26	27	28	29	30	31	29	30						

July							August							September										
Mo	Tu	We	Th	Fr	Sa	Su	Mo	Tu	We	Th	Fr	Sa	Su	Mo	Tu	We	Th	Fr	Sa	Su				
			1	2	3	4							1	2					1	2	3	4	5	6
6	7	8	9	10	11	12	3	4	5	6	7	8	9	7	8	9	10	11	12	13				
13	14	15	16	17	18	19	10	11	12	13	14	15	16	14	15	16	17	18	19	20				
20	21	22	23	24	25	26	17	18	19	20	21	22	23	21	22	23	24	25	26	27				
27	28	29	30	31			24	25	26	27	28	29	30	28	29	30								
							31																	

October							November							December										
Mo	Tu	We	Th	Fr	Sa	Su	Mo	Tu	We	Th	Fr	Sa	Su	Mo	Tu	We	Th	Fr	Sa	Su				
				1	2	3							1						1	2	3	4	5	6
5	6	7	8	9	10	11	2	3	4	5	6	7	8	7	8	9	10	11	12	13				
12	13	14	15	16	17	18	9	10	11	12	13	14	15	14	15	16	17	18	19	20				
19	20	21	22	23	24	25	16	17	18	19	20	21	22	21	22	23	24	25	26	27				

```
import calendar
year = 2026
print(calendar.TextCalendar().formatyear(year))
```

```
import calendar
year = 2026
print(calendar.TextCalendar().formatyear(year))
```

The Math Import:

```
import math
print(math.sqrt(9))
print(math.pi)
print(math.e)
```

Random:

```
: import random
print(random.randint(1,10))
print(random.choice(["yes", "no"]))

5
no
```

```
import random
print(random.randint(1,10))
print(random.choice(["yes", "no"]))
```

```
: import random
print(random.randint(1,10))
print(random.choice(["yes", "no"]))

5
no
```

Import Statistics:

```
# Statistics
import statistics as stats
num = [10, 20, 20, 50]
print(stats.mean(num))
```

25

Statistics

```
import statistics as stats
num = [10, 20, 20, 50]
print(stats.mean(num))
```

Average:

```
: average = stats.mean(num)
|
```

JSON (from Json String to python object and vice versa)

*JavaScript Object Notation (it is a module in Python)

```
[70]: #Json
import json
data = {'name': 'Alice', 'Age': 25}
s = json.dumps(data)
print(s)
```

```
{"name": "Alice", "Age": 25}
```

```
#Json
import json
data = {'name' : 'Alice', 'Age' : 25}
s = json.dumps(data)
print(s)
*From Python String to Javascript string
```

WORKING WITH PACKAGES

2.3 Packages: Structuring Larger Applications

What is a Package?

A **package** is a way to organize related modules into a directory hierarchy. It's essentially a folder that contains multiple module files (`.py` files) and a special file named `__init__.py`. The `__init__.py` file (which can be empty) tells Python that the directory should be treated as a package. Packages can also contain sub-packages, creating a nested structure, much like folders within folders on your computer.

Syntax for Importing from Packages

When working with packages, you use dot notation (`.`) to specify the path to the module or item you want to import:

Python Fundamentals Level 2 - PCAP Certification Training | SCI-Thagorus (Pty) Ltd

```
# Import an entire module from a package
import package_name.module_name

# Import a specific item (function, class, variable) from a module within a
package
from package_name.module_name import specific_item

# Import an entire sub-package
import package_name.sub_package_name
```

Reasoning Behind Packages

Packages are crucial for managing complexity in large-scale applications. They allow you to group related functionalities (modules) together in a logical and hierarchical manner. For example, a financial application might have a `finance` package, which in turn contains modules like `payroll.py`, `taxes.py`, and `invoicing.py`. This structure

*Importing from the Package rather than the Python module library:

```
# Datetime: datetime, timedelta
from datetime import datetime, timedelta    #package module
now = datetime.now()
print(now.strftime('%Y-%m-%d %H:%M'))
```

2026-03-16 14:18

```
# Datetime: datetime, timedelta
from datetime import datetime, timedelta    #package module
now = datetime.now()
print(now.strftime('%Y-%m-%d %H:%M'))
print(now+ timedelta(days=7))
```

“From datetime” is the package. And “Import datetime” is a module.

Datetime: datetime, timedelta

```
from datetime import datetime, timedelta now = datetime.now()
print(now. strftime('%Y-%m-%d %H:%M' ))
```

Or
print(now)

*There will be 3rd party environment to install modules. (More specifically Flask).

In Python, a **module is a physical file** used to logically group related functions, classes, and variables, while a **class is a blueprint** for creating objects that have their own state and behavior.

Real-Life Business Example: Simple Financial Operations Package

Let's create a simplified financial operations package for Joseph's business. We'll organize it into a `finance_ops` package with a `tax_calculations` module for tax logic and a `currency_converter` module for currency exchange.

First, let's set up the directory structure. On your desktop, you would create a folder named `finance_ops`. Inside `finance_ops`, you would create an empty file named `__init__.py` and two other files: `tax_calculations.py` and `currency_converter.py`.

Directory Structure:

provided code into it.

6. **Open your terminal/command prompt:** Navigate to your Desktop directory (`cd Desktop`).

7. **Run the script:** Execute `python run_finance.py` .

This setup demonstrates how modules (`tax_calculations.py` , `currency_converter.py`) are organized within a package (`finance_ops`) and how a main script (`run_finance.py`) can import and utilize functionalities from these structured components.

*Packages, we'll return to them on Wednesday. We need to study OOP first.

Chapter 4: The Data-Driven Edge - Visualization and Machine Learning

4.1 Introduction to the Data-Driven Edge

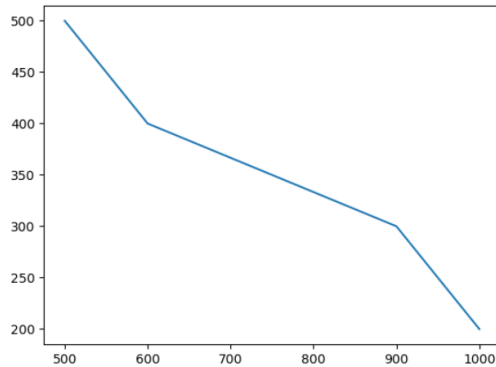
In today's competitive business world, data is crucial for making informed decisions and understanding trends. This chapter introduces you to using Python for data science, focusing on two key areas:

1. **Data Visualization:** Turning raw data into easy-to-understand charts and graphs. This helps Joseph and his team quickly spot patterns and insights.
2. **Machine Learning:** Building models that learn from data to make predictions. This can help businesses forecast sales or understand customer behavior.

Why is this important for business?

An example of plotting a graph (Sales & Price):
(Use of Matplotlib package and the pyplot module)
Stage 1:

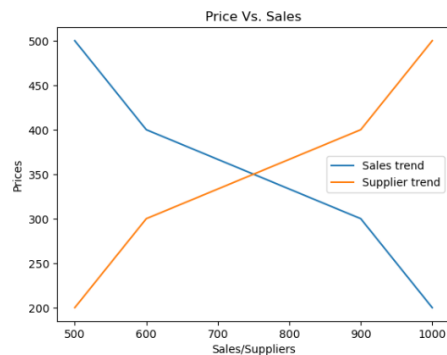
```
[46]: #Data Analysis and Visualization
price = [200,300,400,500]
sales = [1000,900,600,500]
#Module called Pandas (with alias), this is important for Visualization
import pandas as pd
#Matplotlib is a package name, pyplot is the module of the package
from matplotlib import pyplot as plt
plt.plot(sales, price)
plt.show()
```



```
#Data Analysis and Visualization
price = [200,300,400,500]
sales = [1000,900,600,500]
#Module called Pandas (with alias), this is important for
Visualization
import pandas as pd
#Matplotlib is a package name, pyplot is the module of the package
from matplotlib import pyplot as plt
plt.plot(sales, price)
plt.show()
```

Stage 2:

```
[48]: #Data Analysis and Visualization
price = [200,300,400,500]
sales = [1000,900,600,500]
supplies = [500,600,900,1000]
#Module called Pandas (with alias), this is important for Visualization
import pandas as pd
#Matplotlib is a package name, pyplot is the module of the package
from matplotlib import pyplot as plt
plt.plot(sales, price)
plt.plot(supplies, price)
plt.title("Price Vs. Sales")
plt.xlabel("Sales/Suppliers")
plt.ylabel("Prices")
plt.legend(["Sales trend", "Supplier trend"])
plt.show()
```

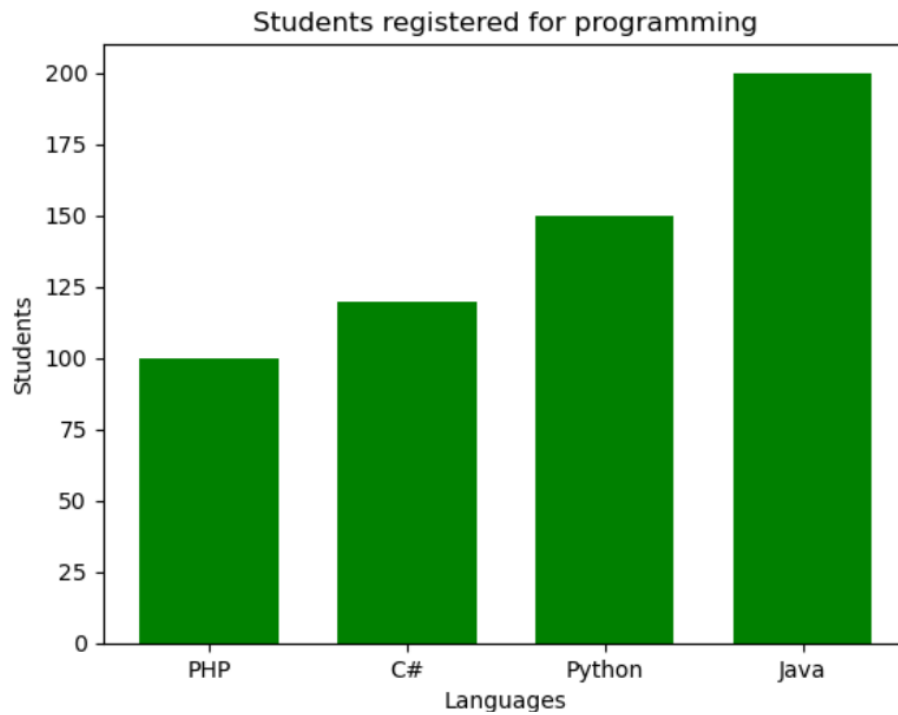


```
#Data Analysis and Visualization
price = [200,300,400,500]
sales = [1000,900,600,500]
supplies = [500,600,900,1000]
#Module called Pandas (with alias), this is important for
Visualization
import pandas as pd
#Matplotlib is a package name, pyplot is the module of the package
from matplotlib import pyplot as plt
plt.plot(sales, price)
plt.plot(supplies, price)
plt.title("Price Vs. Sales")
plt.xlabel("Sales/Suppliers")
plt.ylabel("Prices")
plt.legend(["Sales trend","Supplier trend"])
plt.show()
```

-plt.plot(supplies,price) [That line is to customize the line graph]

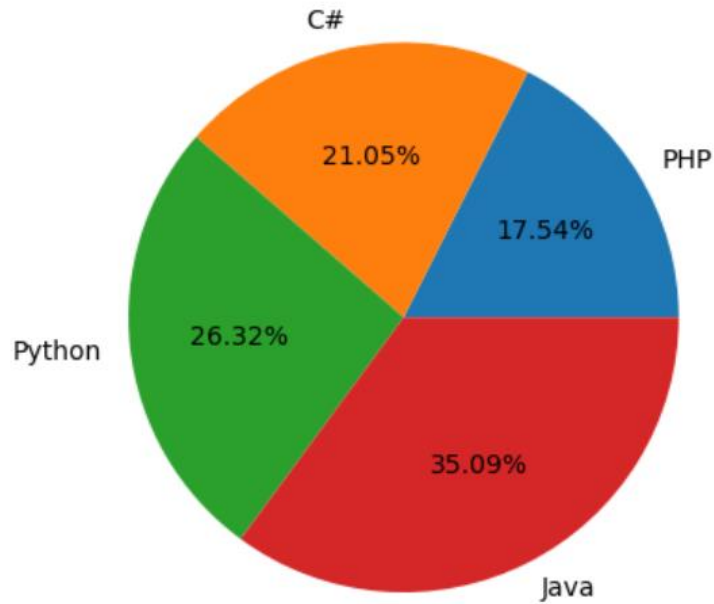
```
from matplotlib import pyplot as plt
plt.plot(sales, price)
plt.plot(supplies, price, marker = 'o', color = 'red', styleline = '-')
plt.xlabel('sales or Supplier')
```

Bar Chart



```
#Bar graph
languages = ['PHP', 'C#', 'Python', 'Java' ]
students = [100, 120, 150, 200]
plt.bar(languages, students, width = 0.7, color = 'green' )
plt.title('Students registered for programming')
plt.xlabel('Languages')
plt.ylabel('Students')
plt.show()
```

Pie Chart



```
languages = ['PHP', 'C#', 'Python', 'Java' ]  
students = [100, 120, 150, 200]  
plt.pie(students, labels = languages, autopct= "%1.2f%%",)  
plt.show()
```

.....
17 March 2026

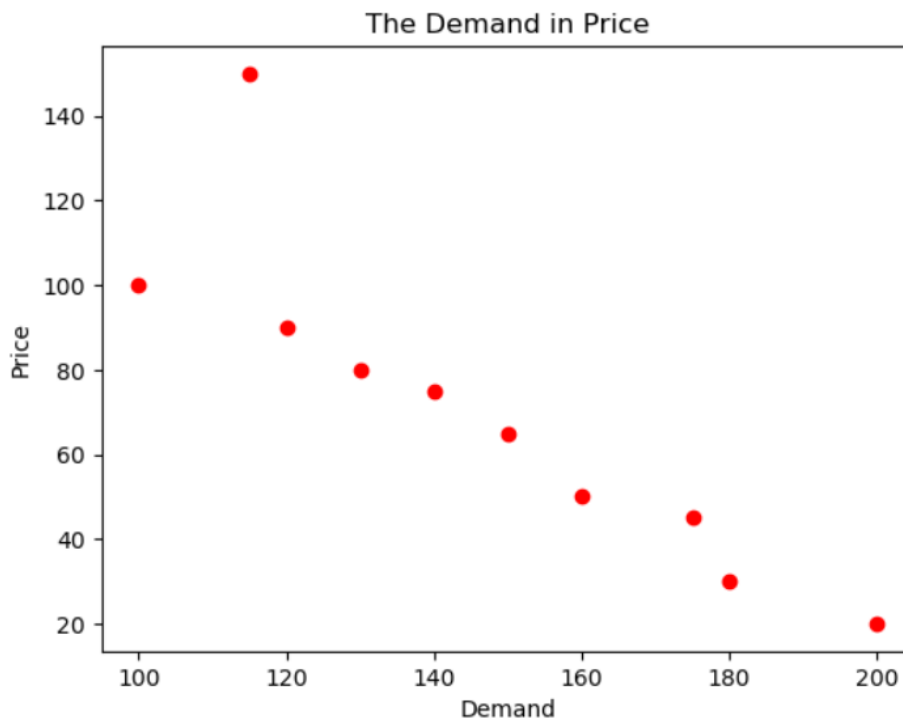
*RECAP- Lambda Functions (30 min)

Scatter Plots (Linear regression)

(The regression line is the slope $y=mx+b$ /gradient)

STAGE 1


```
# Scatter plot
import pandas as pd
#Matplotlib is a package name, pyplot is the module of the package
from matplotlib import pyplot as plt
price = [20, 30, 45, 50, 65, 75, 80, 90, 150,100 ]
demand = [200, 180, 175, 160, 150, 140, 130,120, 115, 100 ]
plt.scatter(demand, price,color = "red" )
plt.title('The Demand in Price')
plt.xlabel('Demand')
plt.ylabel('Price')
plt.show()
```



Scatter plot

```
import pandas as pd #Matplotlib is a package name, pyplot is the
module of the package from matplotlib import pyplot as plt price =
[20, 30, 45, 50, 65, 75, 80, 90, 150,100 ] demand = [200, 180, 175,
160, 150, 140, 130,120, 115, 100 ] plt.scatter(demand, price,color =
"red" ) plt.title('The Demand in Price') plt.xlabel('Demand')
plt.ylabel('Price') plt.show()
```

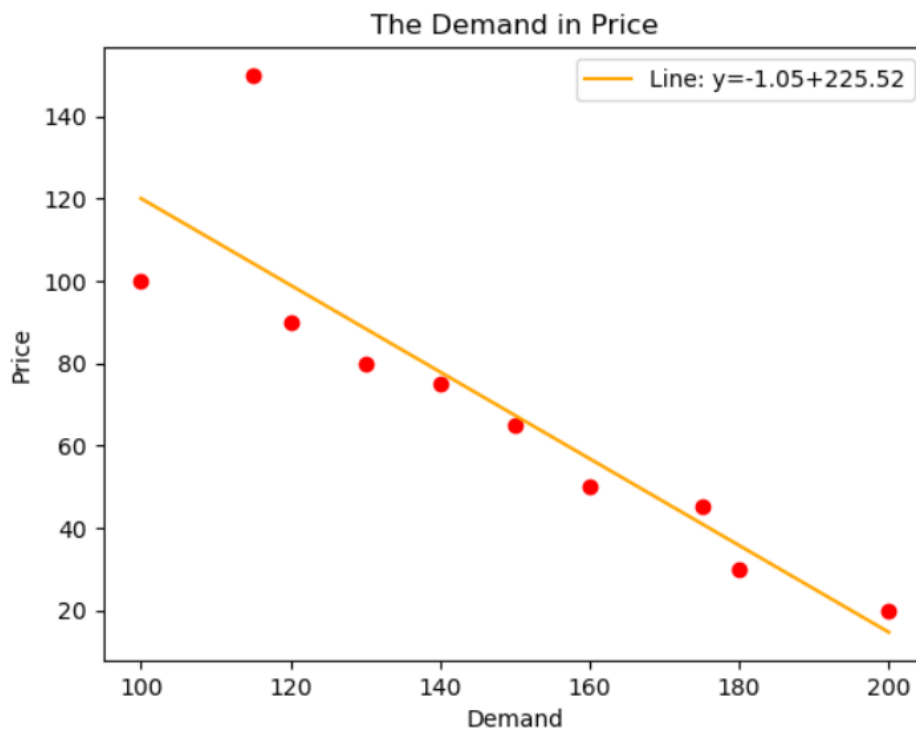
STAGE 2:

This is to model and predict the data the company might supply

-Linear regression Model

```
: # Scatter plot and add regression line (y=mx+b) y m slope, b value of y at y-intercept
import pandas as pd
import numpy as np
from matplotlib import pyplot as plt
price = [20, 30, 45, 50, 65, 75, 80, 90, 150, 100]
demand = [200, 180, 175, 160, 150, 140, 130, 120, 115, 100]
m,b = np.polyfit(demand,price,1)
#create line values accross the range
x_line = np.linspace(min(demand), max(demand), 100)#for creating a line between the x-range, the 100 is for a smooth line
y_line = m*x_line+b
#Plot regression line
plt.plot(x_line, y_line, color = "orange", label = f"Line: y={m:.2f}+{b:.2f}")
plt.legend()

plt.scatter(demand, price,color = "red" )
plt.title('The Demand in Price')
plt.xlabel('Demand')
plt.ylabel('Price')
plt.show()
```



#Scatter plot and add regression line ($y=mx+b$) y m slope, b value of y at y-intercept

```
import pandas as pd import numpy as np from matplotlib import pyplot
as plt price = [20, 30, 45, 50, 65, 75, 80, 90, 150,100 ] demand =
[200, 180, 175, 160, 150, 140, 130,120, 115, 100 ] m,b =
np.polyfit(demand,price,1) #create line values accross the range
x_line = np.linspace(min(demand), max(demand), 100) #for creating a
line between the x-range, the 100 is for a smooth line

y_line = m*x_line+b #Plot regression line plt.plot(x_line, y_line,
color = "orange", label = f"Line: y={m:.2f}+{b:.2f}") plt.legend()

plt.scatter(demand, price,color = "red" ) plt.title('The Demand in
Price') plt.xlabel('Demand') plt.ylabel('Price') plt.show()
```

*That “:.2f” tells us how many numbers come after the decimals.

You can use “.1f” or even “.0f” and respectively this would be something like 2,1 and 2, the last one makes it a whole number.

.....

Chapter 3: The Blueprint - Object-Oriented Programming (OOP)

3.1 Introduction to Object-Oriented Programming (OOP)

Object-Oriented Programming (OOP) is a powerful programming paradigm that organizes software design around **objects** rather than functions and logic. An object is a self-contained unit that bundles together data (attributes) and the functions that operate on that data (methods). The main idea behind OOP is to model real-world entities (like employees, products, or invoices) as software objects, making complex systems easier to design, understand, and maintain.

Why is this important for business?

Many business systems deal with complex entities that have distinct properties and behaviors. OOP provides a natural and intuitive way to represent these entities in code:

- **Modularity:** Each object is a self-contained unit, making it easier to develop, test, and debug parts of a large system independently.
- **Reusability:** Classes (which are blueprints for objects) can be reused across different parts of an application or even in different projects, saving development time and ensuring consistency.
- **Maintainability:** Changes to one object are less likely to affect other parts of the system, simplifying updates and bug fixes.
- **Scalability:** OOP helps systems grow, making them

3.2 Classes and Objects: Blueprints and Instances

What^I are Classes and Objects?

- A **Class** is like a blueprint, a template, or a factory for creating objects. It defines the common attributes (data) and methods (functions) that all objects created from this class will possess. For example, an `Invoice` class would define that every invoice has an `invoice_number`, `customer_name`, `amount`, and methods like `calculate_total()` or `send_to_customer()`.
- An **Object** (or **instance**) is a concrete item created from a class. It's a specific realization of the blueprint. So, if `Invoice` is the class, then `invoice_123` (with `invoice_number=123`, `customer_name="Joseph"`, `amount=500`) would be an
- An **Object** (or **instance**) is a concrete item created from a class. It's a specific realization of the blueprint. So, if `Invoice` is the class, then `invoice_123` (with `invoice_number=123`, `customer_name="Joseph"`, `amount=500`) would be an object of the `Invoice` class. Each object has its own unique set of data based on

Class Syntax in Python:

Defining a class:

```
class ClassName:
    # The constructor method: called automatically when a new object is
    # created.
    # `self` refers to the instance of the class being created.
    def __init__(self, param1, param2):
        # Instance attributes: unique to each object.
        self.param1 = param1
        self.param2 = param2

    # An instance method: operates on the object's data.
    # `self` must be the first parameter to access the object's attributes.
    def instance_method(self):
        return f"Method called on object with param1: {self.param1}"
```

```
#OOP BASIC CLASS TEMPLATE
class ClassName:
    def __init__(self, param1, param2):
        self.param1 = param1
        self.param2 = param2
    def instant_method(self):
        return f"param1 is {self.param1} and {self.param2}"
obj = ClassName("Hello", 123)
print(obj.instant_method())
```

param1 is Hello and 123

```
#OOP BASIC CLASS TEMPLATE class ClassName: def init(self, param1,
param2): self.param1 = param1 self.param2 = param2 def
instant_method(self): return f"param1 is {self.param1} and
{self.param2}" obj = ClassName("Hello", 123)
print(obj.instant_method())
```

Another example:

```
: #OOP BASIC CLASS TEMPLATE
class Product:
    def __init__(self, name, price):
        self.name = name
        self.price = price
    def label(self):
        return f"{self.name} - R{self.price}"
p = Product("Computer HP", 14500)
print(p.label())
```

Computer HP - R14500

```
#OOP BASIC CLASS TEMPLATE class Product: def init(self, name, price):
self.name = name self.price = price def label(self): return
f"{self.name} - R{self.price}" p = Product("Computer HP", 14500)
print(p.label())
```

Another example done in 3 different ways:

```
[25]: #calculate total price (name, price, qntbought)
class Total_Price:
    def __init__(self, name, price, qnt):
        self.name = name
        self.price = price
        self.qnt = qnt

    def useful_method(self):
        return f"{self.name} - R{self.price}, Quantity is {self.qnt}"
p = Total_Price("Computer HP", 14500,60)
print(p.useful_method())
```

Computer HP - R14500, Quantity is 60

```
#calculate total price (name, price, qntbought) class Total_Price:
def init(self, name, price, qnt): self.name = name self.price = price
self.qnt = qnt

def useful_method(self):
    return f"{self.name} - R{self.price}, Quantity is {self.qnt}"

p = Total_Price("Computer HP", 14500,60) print(p.useful_method())
```

Or

```

: #calculate total price (name, price, qntbought)
class Total_Price:
    def __init__(self, name, price, qnt):
        self.name = name
        self.price = price
        self.qnt = qnt

    def useful_method(self):
        return f"{self.name} - R{self.price}"
    def final_price(self):
        return self.price*self.qnt
p = Total_Price('Computer Hp', 2500, 4)
print(p.useful_method())
print('Quantity:', p.qnt)
print('The total price is', p.final_price())

```

Computer Hp - R2500
Quantity: 4
The total price is 10000

```

#calculate total price (name, price, qntbought) class Total_Price:
def init(self, name, price, qnt): self.name = name self.price = price
self.qnt = qnt

def useful_method(self):
    return f"{self.name} - R{self.price}"
def final_price(self):
    return self.price*self.qnt
p = Total_Price('Computer Hp', 2500, 4)
print(p.useful_method())
print('Quantity:', p.qnt)
print('The total price is', p.final_price())

```

Or


```

#calculate total price (name, price, qntbought)
class Product:
    def __init__(self, name, price, quantity):
        self.name = name
        self.price = price
        self.quantity = quantity
    def label(self):
        return f'{self.name}- R{self.price}'
    def total_price(self):
        return f'\nQuantity bought: {self.quantity}\nTotal price: {self.price*self.quantity}'
p = Product('Computer Hp', 2500, 4)
print(p.label(), p.total_price())

```

Computer Hp- R2500
 Quantity bought: 4
 Total price: 10000

```

#calculate total price (name, price, qntbought)
class Product:
def init_(self, name, price, quantity):
self.name = name
self.price = price
self.quantity = quantity
def label(self):
return f'{self.name}- R{self.price}'
def total_price(self):
return f'\nQuantity bought: {self.quantity}\nTotal price:
{self.price*self.quantity}'
p = Product('Computer Hp', 2500, 4)
print(p.label(), p.total_price())

```

Another example done in 2 different ways:

```

: #class BankAccount default parameter for the balance owner, balance deposit an amount
class BankAccount:
    def __init__(self, balance ,owner):
        self.balance = balance
        self.owner = owner

    def depositing(self):
        self.balance += self.owner
        return self.balance
pp = BankAccount(0,1000)
print("The Amount deposited is", pp.depositing())

```

The Amount deposited is 1000

```

#class BankAccount default parameter for the balance owner, balance
deposit an amount
class BankAccount:
    def __init__(self, balance, owner):
        self.balance = balance
        self.owner = owner

    def depositing(self):
        self.balance += self.owner
        return self.balance

pp = BankAccount(0, 1000)
print("The Amount deposited is", pp.depositing())

```

Or

```

#class BankAccount default parameter for the balance owner, balance deposit an amount
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance
    def depositAmt(self, deposit):
        self.balance += deposit
owner = input('Enter name:')
obj = BankAccount(owner)
deposit = float(input('Enter Deposit Amount:'))
balance = obj.depositAmt(deposit)
print(obj.owner, '\n Account balance:', obj.balance)

Enter name: Peter
Enter Deposit Amount: 3000
Peter
Account balance: 3000.0

```

```

#class BankAccount default parameter for the balance owner, balance
deposit an amount

class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance

    def depositAmt(self, deposit):
        self.balance += deposit
owner = input('Enter name:')
obj = BankAccount(owner)

deposit = float(input('Enter Deposit Amount: '))

```

```
balance = obj.depositAmt(deposit)
print(obj.owner, ('\n Account balance:'), obj.balance)
---
Enter name: Peter
Enter Deposit Amount: 3000
Peter
Account balance: 3000.0
```

*Doesn't need a return statement, also takes in input from the user.

Another example done in 3 different ways:

```
# DISPLAY NAME AND BALANCE
# CLASS NAME : CUSTOMER 2 PARAMETER (NAME AND BALANCE)
class Customer:
    def __init__(self, name, balance):
        self.name = name
        self.balance = balance
    def display(self):
        return f'Customer:{self.name}, balance:{self.balance}'
C = Customer('Peter', 300)
print(C.display())

Customer:Peter, balance:300
```

DISPLAY NAME AND BALANCE

CLASS NAME : CUSTOMER 2 PARAMETER (NAME AND BALANCE)

```
class Customer:
    def init_(self, name, balance):
        self.name = name
        self.balance = balance
    def display(self):
        return f'Customer: {self.name}, balance: {self.balance}'
C = Customer('Peter', 300)
print(C.display())
```

Or

```
# DISPLAY NAME AND BALANCE
# CLASS NAME : CUSTOMER 2 PARAMETER (NAME AND BALANCE)
class Customer:
    def __init__(self, name, balance):
        self.name = name
        self.balance = balance
    def display(self):
        return f'Customer:{self.name}, balance:{self.balance}'
a = input('enter your name')
C = Customer(a, 300)
print(C.display())
```

```
enter your name Peter
Customer:Peter, balance:300
```

Or

(With User Input)

```
[83]: class CustomerInfo:
    def __init__(self, name, balance):
        self.name = name
        self.balance = balance
    def showing(self):
        return f"Customer name{self.name} and Balance {self.balance}"
name = input("Enter Your Name:")
obj = CustomerInfo(name,300)

balance = float(input("Enter Your Balance:"))
obj2 = CustomerInfo(name,balance)

print("The customer name is", obj.name, "and their balance is:", obj2.balance)
```

```
Enter Your Name: Mhiha
Enter Your Balance: 3000
The customer name is Mhiha and their balance is: 3000.0
```

```
class CustomerInfo: def init(self, name, balance): self.name = name
self.balance = balance def showing(self): return f"Customer
name{self.name} and Balance {self.balance}" name = input("Enter Your
Name:") obj = CustomerInfo(name,300)

balance = float(input("Enter Your Balance:")) obj2 =
CustomerInfo(name,balance)
```

```
print("The customer name is", obj.name, "and their balance is:",  
obj2.balance)
```

*Instance Variable:

(s1,s2 & s3)are all Local variables, local to the method/function

```
: #Instant variables  
class Student:  
    def __init__(self,name, score):  
        self.name = name  
        self.score = score  
s1 = Student('Alice', 80)  
s2 = Student('Peter', 90)  
s3 = Student('Bob', 20)  
print(s1.name,s2.score)
```

Alice 90

*Class Variable:

(Global and accessible to)

```
: #Class variables  
class Student:  
    school = "DUT" #Global variable (Overrides all the local variables)  
    def __init__(self,name, score):  
        self.name = name  
        self.score = score  
s1 = Student('Alice', 80)  
s2 = Student('Peter', 90)  
s3 = Student('Bob', 20)  
print(s3.school)
```

DUT

Here's an example:

```

: class Student:
    count = 0 #Global variable (Overrides all the local variables)
    def __init__(self, name, score):
        self.name = name
        self.score = score
        Student.count += 1
s1 = Student('Alice', 80)
s2 = Student('Peter', 90)
s3 = Student('Bob', 20)
print(f"{s1.name} is the 1st student, {s2.name} is 2nd and {s3.name} is the third,\n the number of students is {s1.count}")

```

Alice is the 1st student, Peter is 2nd and Bob is the third,
the number of students is 3

```

class Student: count = 0 #Global variable (Overrides all the local
variables) def init(self, name, score): self.name = name self.score =
score Student.count += 1 s1 = Student('Alice', 80) s2 =
Student('Peter', 90) s3 = Student('Bob', 20) print(f"{s1.name} is the
1st student, {s2.name} is 2nd and {s3.name} is the third,\n the
number of students is {s1.count}")

```

Another Example:

```

2]: # combined example instant variable for personal data Class variable for shared data
#class name: Employee , company for class variable, name and salary for instant variable
class Employee:
    company = "Daily Investors"
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary
e1 = Employee('Alice', 8000)
e2 = Employee('Peter', 9000)
e3 = Employee('Bob', 2000)
print(f"{e1.name}, {e1.salary}, {e1.company},\n{e2.name}, {e2.salary}, {e2.company}\n{e3.name}, {e3.salary}, {e3.company}")

```

Alice, 8000, Daily Investors,
Peter, 9000, Daily Investors
Bob, 2000, Daily Investors

combined example instant variable for personal data Class variable for shared data

```
#class name: Employee , company for class variable, name and salaryy  
for instant variable class Employee: company = "Daily Investors" def  
init(self,name, salary): self.name = name self.salary = salary e1 =  
Employee('Alice', 8000) e2 = Employee('Peter', 9000) e3 =  
Employee('Bob', 2000) print(f"{e1.name}, {e1.salary},  
{e1.company},\n{e2.name}, {e2.salary}, {e2.company}\n{e3.name},  
{e3.salary}, {e3.company}")
```

Or

```
self.salary = salary  
def display(self):  
    return f'Company:{Employee.company}, Name: {self.name}, Salary:R{self.salary}'  
e1 = Employee('Joe', 5000)  
e2 = Employee('Blessing', 10000)  
print(e1.display())  
print(e2.display())
```

```
Company:ABSA Ltd, Name: Joe, Salary:R5000  
Company:ABSA Ltd, Name: Blessing, Salary:R10000
```

And Even:

```
104]: class Employee:  
    company = "Sage"  
    employeeList = []  
    def __init__(self,name,salary):  
        self.name = name  
        self.salary = salary  
        Employee.employeeList.append(self)  
  
e1 = Employee("Marta",10000)  
e2 = Employee("Sive",20000)  
  
for emp in Employee.employeeList:  
    print(f"Company: {emp.company} \nEmployee name: {emp.name} \nSalary: {emp.salary}")  
  
Company: Sage  
Employee name: Marta  
Salary: 10000  
Company: Sage  
Employee name: Sive  
Salary: 20000
```

```

class Employee: company = "Sage" employeeList = [] def
init(self,name,salary): self.name = name self.salary = salary
Employee.employeeList.append(self)

e1 = Employee("Marta",10000) e2 = Employee("Sive",20000)

for emp in Employee.employeeList: print(f"Company: {emp.company}
\nEmployee name: {emp.name} \nSalary: {emp.salary}")

```

3.3 Instance vs. Class Variables: Data Scope

What are Instance and Class Variables?

- **Instance Variables:** These are variables that belong to a specific object (instance) of a class. Each object gets its own unique copy of instance variables. Changes to an instance variable in one object do not affect other objects. They are typically defined within the `__init__` method using `self.variable_name`.
- **Class Variables:** These are variables that are shared by all instances of a class. They are defined directly within the class body, outside of any method. If a class variable is changed, that change is reflected across all instances of that class, because all instances refer to the same single copy of the variable.

*Inheritance & Polymorphism:

3.4 Inheritance and Polymorphism: Building on Existing Structures

What are Inheritance and Polymorphism?

- **Inheritance:** This is a fundamental OOP concept that allows a new class (called a **child class** or **subclass**) to derive attributes and methods from an existing class (called a **parent class** or **superclass**). The child class automatically gets all the public attributes and methods of the parent. It can then add its own unique features or modify (override) the inherited ones. Inheritance promotes code
- **Polymorphism:** Meaning “many forms,” polymorphism allows objects of different classes to be treated as objects of a common superclass. This means you can write code that works with objects of a parent class, and it will automatically work correctly with objects of any child class that inherits from it, even if the child classes implement a common method differently. It enables flexibility and extensibility in your code.

Syntax

Syntax for Inheritance:

```
class ParentClass:
    def common_method(self):
        print("This is a common method from the parent.")

class ChildClass(ParentClass): # ChildClass inherits from ParentClass
    def specific_method(self):
        print("This is a method specific to the child.")

    def common_method(self): # Overriding the common_method
        super().common_method() # Optionally call the parent's version
        print("This is the child's version of the common method.")
```

Here's an example (Inheritance):

```

#inheritance
class Person:
    def __init__(self, name):
        self.name = name
    def greet(self):
        return f'Hello, i am {self.name}'
class Student(Person):
    def __init__(self, name, student_id):
        super().__init__(name)
        self.student_id = student_id
s = Student('Peter', 'S123')
print(s.greet())
print(s.student_id)

```

```

Hello, i am Peter
S123

```

```

#inheritance
class Person:
    init_(self, name):
    self.name = name
    def greet(self):
    return f'Hello, i am {self.name}'
class Student(Person):
    def
    super().
    self.student_id = student_id
s = Student('Peter', 'S123')
print(s.greet())
print(s.student_id)

```

My Inheritance example:

```

class Employee:
    def __init__(self, name):
        self.name = name
    def greet(self):
        return f'Hello {self.name},'
class Person(Employee):
    def __init__(self, name, department):
        super().__init__(name)
        self.department = department
e = Person("Jonathan", "Sales")
print(e.greet(),"Your department is:",e.department)

```

Hello Jonathan, Your department is: Sales

```

class Employee:
    def init(self, name):
        self.name = name
    def greet(self):
        return f'Hello {self.name},'
class Person(Employee):
    def init(self, name, department):
        super().init(name)
        self.department = department
e = Person("Jonathan", "Sales")
print(e.greet(),"Your department is:",e.department)

```

.....

18 March 2026

*The recap:



```

[ ]:
class, class variable, constructor, instant variable, method, attributes, inherit, super()

```

1. Class - for us to have an object we need the blueprint.
2. Class variable - It is the object of the class.
3. Constructor - We use it to access the object's data (self). This sets the initial values "self.name = name" part.
4. Instant attribute/variable - data that is unique to each object.
5. Method is the function - After the constructor when you define the second def statement it is the method since it does the actual operation of the code.

6. Inheritance - Attributes of the Super class are inherited by the subclass. The Super.constructor ().__init__(name) or any parameter from the parent class.
 7. Polymorphism - “Many forms”, Enables flexibility and extensibility in code.
-

Syntax of Polymorphism

```
class ParentClass:
    def common_method(self):
        print("This is a common method from the parent.")

class ChildClass(ParentClass): # ChildClass inherits from ParentClass
    def specific_method(self):
        print("This is a method specific to the child.")

    def common_method(self): # Overriding the common_method
        super().common_method() # Optionally call the parent's version
        print("This is the child's version of the common method.")
```

Example in class:

```
#class, class variable, self, constructor, instant attribute, method, attributes, inherit, su
class Animal:
    def sound(self):
        return 'Some sound'
class Dog(Animal):
    def sound(self):
        return 'bark'
class Cat(Animal):
    def sound(self):
        return 'mew'
animals = [Dog(), Cat()]
for a in animals:
    print(a.sound())
```

bark

meet.google.com is sharing your screen. [Stop sharing](#) [Hide](#)

```
class Animal:
def sound(self):
return 'Some sound'
class Dog(Animal):
def sound(self):
return 'bark'
```

```

class Cat(Animal):
def sound(self):
return 'mew'
animals = [Dog(), Cat()]
for a in animals:
print(a.sound())

```

And

```

class Employees:
    def calculate_salary(self):
        return 'Basic Salary'
class Managers(Employees):
    def calculate_salary(self):
        return 'Basic salary + Manager allowance'
class interns(Employees):
    def calculate_salary(self):
        return 'Stipend only'
workers = [Employees(), Managers(), interns()]
for w in workers:
    print(w.calculate_salary())

```

Self-example in polymorphism:

```

1]: #Class Employee: Define Work in the Method function, the manager is planning, the intern is learning
class Employee:
    def work_define(self):
        return "Employee works"
class Manager:
    def work_define(self):
        return "The manager is planning"
class Intern:
    def work_define(self):
        return "The intern is learning"
workers = [Employee(),Manager(),Intern()]
for w in workers:
    print(w.work_define())

Employee works
The manager is planning
The intern is learning

```

```

#Class Employee: Define Work in the Method function, the manager is
planning, the intern is learning
class Employee:
    def work_define(self):
        return "Employee works"
class Manager(Employee):
    def work_define(self):
        return "The manager is planning"

```

```
class Intern(Manager):
    def work_define(self):
        return "The intern is learning"
workers = [Employee(),Manager(),Intern()]
for w in workers:
    print(w.work_define())
```

The shapes example:

```
class Shapes:
    def area(self):
        return 0
class Rectangle(Shapes):
    def __init__(self, l, w):
        self.l = l
        self.w = w
    def area(self):
        return self.l*self.w
class Circle(Shapes):
    def __init__(self,r):
        self.r =r
```

```
class Rectangle(Shapes):
    def __init__(self, l, w):
        self.l = l
        self.w = w
    def area(self):
        return self.l*self.w
class Circle(Shapes):
    def __init__(self,r):
        self.r =r
    def area(self):
        return 3.142*self.r*self.r
shapes =[Rectangle(10,6), Circle(7)]
```

.....

ENCAPSULATION & Abstraction

3.5 Encapsulation and Abstraction: Protecting and Simplifying

What are Encapsulation and Abstraction?

- **Encapsulation:** This is the bundling of data (attributes) and the methods that operate on that data into a single unit (the object). It also involves restricting direct access to some of an object's components, meaning that the internal state is hidden and protected from external modification. In Python, encapsulation is achieved by convention (using leading underscores for private-like members) rather than strict access modifiers like in some other languages.
- **Abstraction:** This is the process of hiding the complex implementation details and showing only the essential features of an object. It focuses on *what* an object does rather than *how* it does it. Abstraction simplifies the view of complex systems, allowing users to interact with objects at a higher level without needing to understand all the underlying intricacies.

Syntax (Pythonic Encapsulation)

Python doesn't have strict `private` or `protected` keywords like Java or C++. Instead, it uses conventions to indicate intended visibility:

- **Single leading underscore (`_variable_name`):** This indicates that a variable or method is intended for internal use within the class or module. It's a convention, not an enforcement; you *can* still access it from outside, but it signals to other developers that they generally shouldn't.
- **Double leading underscore (`__variable_name`):** This triggers a process called "name mangling." Python internally renames the attribute to `_ClassName__variable_name`. This makes it harder (though not impossible) to

Python Fundamentals Level 2 - PCAP Certification Training | SCI-Thagorus (Pty) Ltd

access from outside the class, providing a stronger form of privacy, primarily to avoid naming conflicts in inheritance.

```
class MySecureClass:
    def __init__(self):
        self.public_data = "I am public."
        self._protected_data = "I am protected by convention."
        self.__private_data = "I am truly private (name-mangled)."
```



```
    def get_public_data(self):
```

```

class MySecureClass:
    def __init__(self):
        self.public_data = "I am public."
        self._protected_data = "I am protected by convention."
        self.__private_data = "I am truly private (name-mangled)."

    def get_public_data(self):
        return self.public_data

    def get_protected_data(self):
        return self._protected_data

    def get_private_data(self):
        return self.__private_data

```

*IN encapsulation 1 underscore is reserved for protected, and two underscores are for the private data.

Example:

```

# Encapslatn
# encapsulation using private variable
class Student:
    def __init__(self, name):
        self.__name = name    # private variable
    def get_name(self):
        return self.__name    # getter method
    def set_name(self, new_name):
        self.__name = new_name    # setter method

```

```

#create object
s = Student('Joseph')
print(s.get_name())    # accessing through the method
s.set_name('Peter')
print(s.get_name())

```

Joseph

Peter

Another encapsulation (Private) example:

```
# Protecting a bank balance
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance
    def deposit(self, amount):
        self.__balance += amount
    def get_balance(self):
        return self.__balance
acc = BankAccount(100)
acc.deposit(50)
print(acc.get_balance())
```

Protected Example:

```
# _var for protected variable
class Car:
    def __init__(self, brand):
        self._brand = brand
c = Car('Toyota')
print(c._brand)
```

Toyota

.....

FLASK

Flask is more manual than Django, but they're the same web services.

Chapter 5: Your First Web Service - A Deep Dive into Flask

5.1 Introduction to Web Services and Flask

In today's connected world, applications rarely work in isolation. They communicate with each other over the internet using **web services** or **APIs** (Application Programming Interfaces). A web service allows one application to request data or trigger actions in another. For example, a mobile app might use a web service to get the latest product information from a company's server.

*Api's are important the make the connection between the two applications, such as Uber having a google maps integration.

What is Flask:

Flask is a lightweight and flexible Python framework for building web services and web applications. It's known for being easy to learn and use, making it a perfect choice for beginners. With Flask, you can quickly create a web service that can be accessed by other applications, such as a web browser, a mobile app, or another server.

* download sublime text 3

Installation for flask:

Before we can build our web service, we need to set up a clean and isolated environment for our project. This is a professional best practice that prevents conflicts between different projects' dependencies.

Step 1: Create a Project Folder

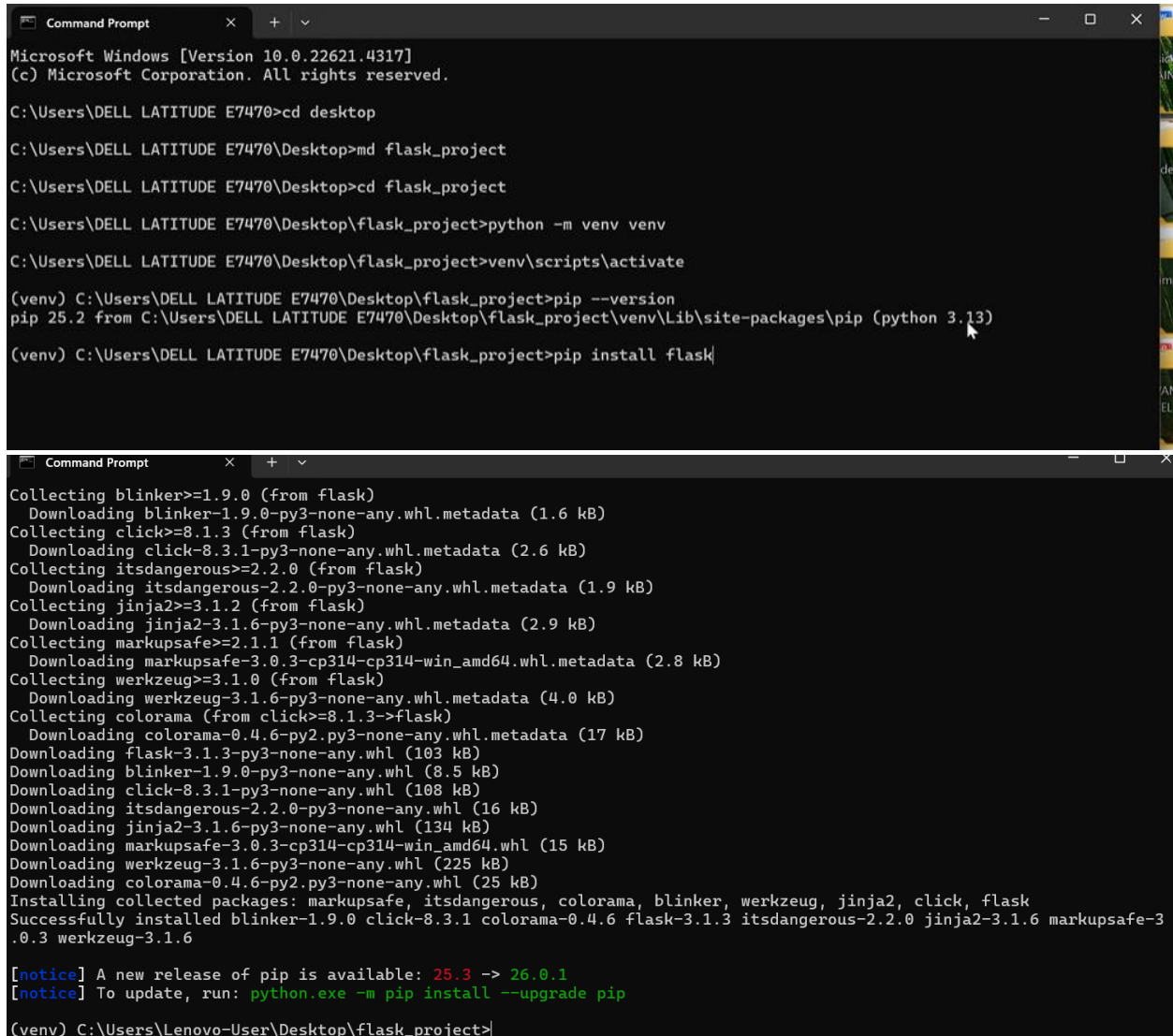
First, let's create a dedicated folder for our Flask project on your Desktop.

Open your **Command Prompt** (on Windows) or **Terminal** (on macOS/Linux) and run the following commands:

```
# Navigate to your Desktop
cd Desktop
```

```
*cd desktop
*md flask_project
*cd flask_project
*>python -m venv venv #creates a virtual environment
```

```
-There are files that are created
*venv\scripts\activate
*pip --version
*pip install flask
```



The image shows two screenshots of a Windows Command Prompt window. The top screenshot shows the initial setup steps: navigating to the desktop, creating a directory named 'flask_project', and activating the virtual environment. The bottom screenshot shows the output of the 'pip install flask' command, which lists the dependencies being downloaded and installed, including blinker, click, itsdangerous, jinja2, markupsafe, werkzeug, and colorama. It also shows a notice about a new release of pip being available.

```
Command Prompt
Microsoft Windows [Version 10.0.22621.4317]
(c) Microsoft Corporation. All rights reserved.

C:\Users\DELL LATITUDE E7470>cd desktop
C:\Users\DELL LATITUDE E7470\Desktop>md flask_project
C:\Users\DELL LATITUDE E7470\Desktop>cd flask_project
C:\Users\DELL LATITUDE E7470\Desktop\flask_project>python -m venv venv
C:\Users\DELL LATITUDE E7470\Desktop\flask_project>venv\scripts\activate

(venv) C:\Users\DELL LATITUDE E7470\Desktop\flask_project>pip --version
pip 25.2 from C:\Users\DELL LATITUDE E7470\Desktop\flask_project\venv\Lib\site-packages\pip (python 3.13)

(venv) C:\Users\DELL LATITUDE E7470\Desktop\flask_project>pip install flask

Collecting blinker>=1.9.0 (from flask)
  Downloading blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting click>=8.1.3 (from flask)
  Downloading click-8.3.1-py3-none-any.whl.metadata (2.6 kB)
Collecting itsdangerous>=2.2.0 (from flask)
  Downloading itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
Collecting jinja2>=3.1.2 (from flask)
  Downloading jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting markupsafe>=2.1.1 (from flask)
  Downloading markupsafe-3.0.3-cp314-cp314-win_amd64.whl.metadata (2.8 kB)
Collecting werkzeug>=3.1.0 (from flask)
  Downloading werkzeug-3.1.6-py3-none-any.whl.metadata (4.0 kB)
Collecting colorama (from click>=8.1.3->flask)
  Downloading colorama-0.4.6-py2.py3-none-any.whl.metadata (17 kB)
Downloading flask-3.1.3-py3-none-any.whl (103 kB)
Downloading blinker-1.9.0-py3-none-any.whl (8.5 kB)
Downloading click-8.3.1-py3-none-any.whl (108 kB)
Downloading itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Downloading jinja2-3.1.6-py3-none-any.whl (134 kB)
Downloading markupsafe-3.0.3-cp314-cp314-win_amd64.whl (15 kB)
Downloading werkzeug-3.1.6-py3-none-any.whl (225 kB)
Downloading colorama-0.4.6-py2.py3-none-any.whl (25 kB)
Installing collected packages: markupsafe, itsdangerous, colorama, blinker, werkzeug, jinja2, click, flask
Successfully installed blinker-1.9.0 click-8.3.1 colorama-0.4.6 flask-3.1.3 itsdangerous-2.2.0 jinja2-3.1.6 markupsafe-3.0.3 werkzeug-3.1.6

[notice] A new release of pip is available: 25.3 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip

(venv) C:\Users\Lenovo-User\Desktop\flask_project>
```

*Open a text editor such as Sublime Text or Notepad++

Then the app on the text editor:

```
app.py x
1  from flask import Flask
2  app = Flask(__name__) #Main Application object and location
3
4  #Home endpoint
5  @app.route("/") #which url are we responding to
6  def home(): #Method Home returns the statement
7      return "Welcome to my first Flask made API!"
8  if __name__ == "__main__":
9      app.run(debug = True)
```

from flask import Flask app = Flask(name) #Main Application object and location

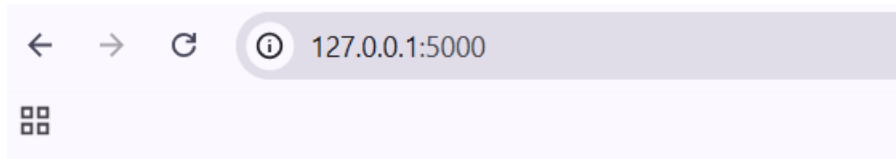
#Home endpoint @app.route("/") #which url are we responding to def home(): #Method Home returns the statement return "Welcome to my first Flask made API!" if name == "main": app.run(debug = True)

```
0.3 werkzeug-3.1.6
[notice] A new release of pip is available: 25.3 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
(venv) C:\Users\Lenovo-User\Desktop\flask_project>python app.py
```

*Copy the http link

```
(venv) C:\Users\Lenovo-User\Desktop\flask_project>python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 269-915-470
* Detected change in 'C:\\Users\\Lenovo-User\\Desktop\\flask_project\\app.py', reloading
* Restarting with stat
* Debugger is active!
* Debugger PIN: 269-915-470
127.0.0.1 - - [18/Mar/2026 15:26:51] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [18/Mar/2026 15:26:51] "GET /favicon.ico HTTP/1.1" 404 -
```

*Then paste the http link, voila!:



Adding an Information Page

```
app.py
1  from flask import Flask
2  app = Flask(__name__) #Main Application object and location
3
4  #Home endpoint
5  @app.route("/") #which url are we responding to
6  def home(): #Method Home returns the statement
7      return "Welcome to my first Flask made API!"
8
9  #First Api endpoint
10 @app.route("/info")
11 def info():
12     return { #returning a dictionary
13         "name": "LetsGo API",
14         "version": "1.1",
15         "status": "Runniing Successfully"
16     }
17
18 if __name__ == "__main__":
19     app.run(debug = True)
20
21
22
```

```
from flask import Flask app = Flask(name) #Main Application object
and location

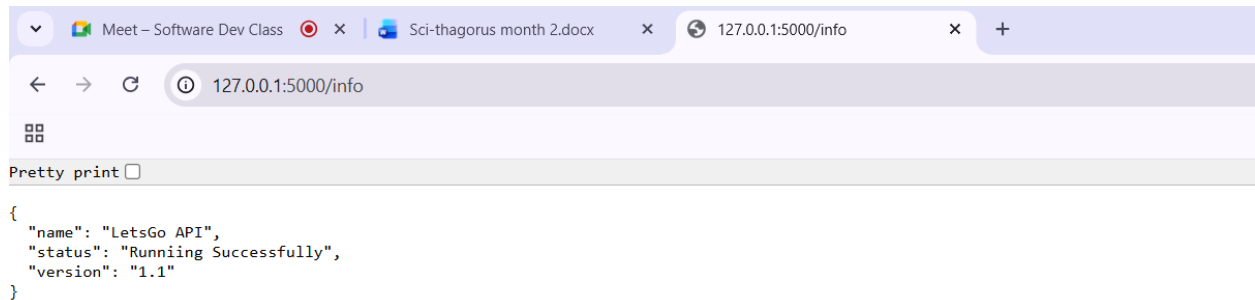
#Home endpoint

@app.route("/") #which url are we responding to def home(): #Method
Home returns the statement return "Welcome to my first Flask made
API!"
```

```
#First Api endpoint @app.route("/info") def info(): return
{ #returning a dictionary "name": "LetsGo API", "version": "1.1",
  "status": "Runniing Successfully" }

if name == "main": app.run(debug = True)
```

The display:



IF INTERNET CRASHES, USE THIS DOCUMENT. OFFLINE MODE

19 March 2026

*Explaining the code above, the route - "/" connects url to a python function, primarily for the API. *Import Flask first, it is a library, the render template for the html code, and jsonify for the API app = Flask(name) - Also very important is an object of flask

*The "/info" returns our exact website route, it's telling flask which url to respond to. @app.route("/")

*API doesn't communicate with humans, only machines. *An API is a Collection of JSon endpoints used by the machines {get, post, put, delete}, It's a middle man between two applications.

```
C:\Users\Lenovo-User\Desktop\flask_project\app.py - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help

app.py home.html
1 from flask import Flask, render_template, jsonify, request #added request
2 app = Flask(__name__) #Main Application object and location
3 #in-memory storage (We use Get Method)-Status endpoint
4 USERS = [
5     {"ID": 1, "Full_name": "Peter James", "Email": "james.com"},
6     {"ID": 2, "Full_name": "John Sibiya", "Email": "sibiya.com"}
7 ]
8 _next_id = 3 #simple auto-increment
9
10
11 #Home endpoint
12 @app.route("/") #which url are we responding to
13 def home(): #Method Home returns the statement
14     return render_template("home.html", page_title = "LetsGo API Home")
15
16 #First Api endpoint
17 @app.route("/info")
18 def info():
19     return { #returning a dictionary
20         "name": "LetsGo API",
21         "version": "1.1",
22         "status": "Runniing Successfully"
23     }
24 #Status endpoint = route + function
25 @app.route("/status", methods = ["GET"])
26 def status():
27     return jsonify({
28         "app": "App is running...",
29         "message": "All systems are optimal!",
30         "users count": len(USERS)
31     }), 200
32
33
34 if __name__ == "__main__":
35     app.run(debug = True)
36
```

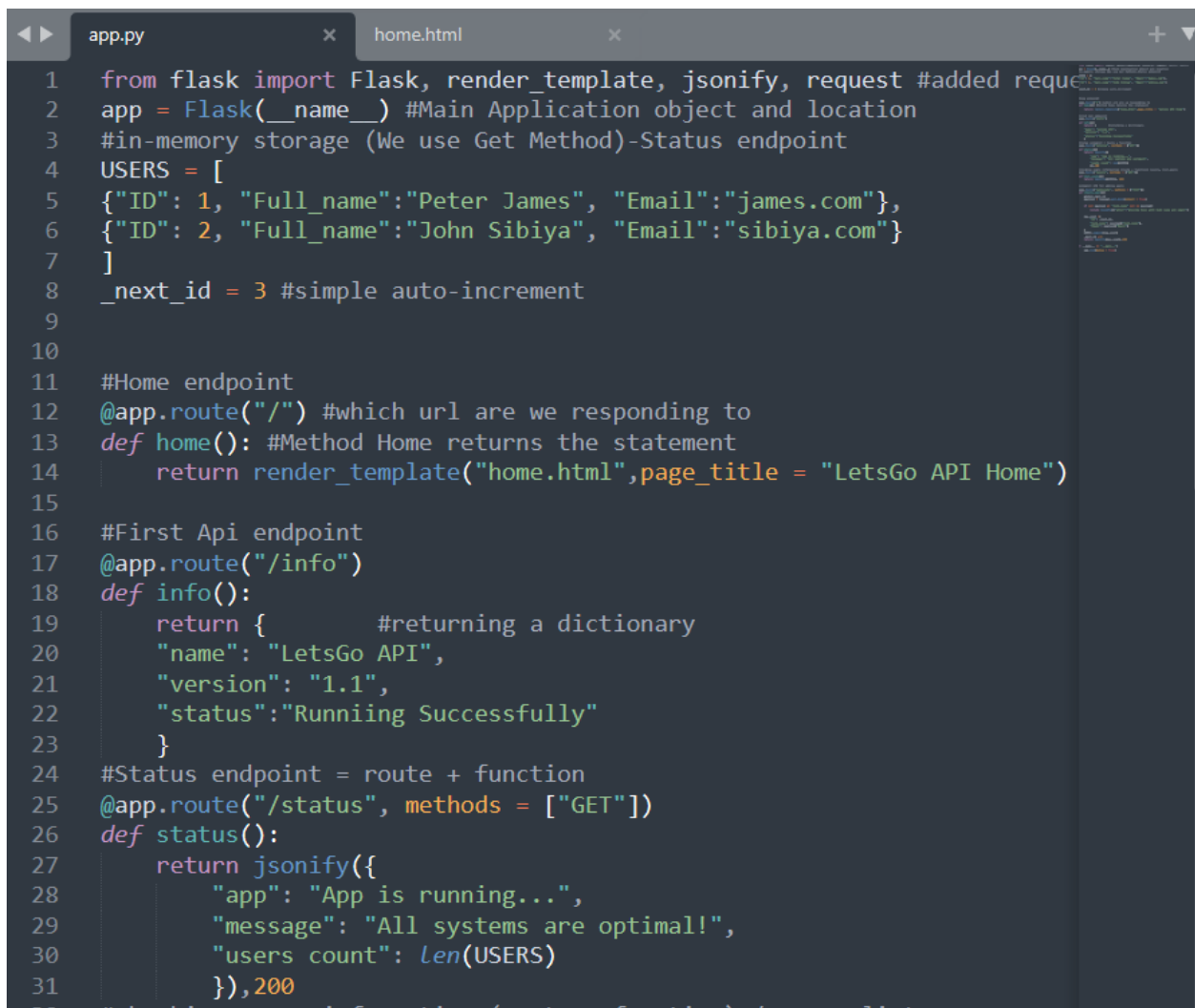
```
from flask import Flask, render_template, jsonify, request #added
request app = Flask(name) #Main Application object and location #in-
memory storage (We use Get Method)-Status endpoint USERS = [ {"ID":
1, "Full_name": "Peter James", "Email": "james.com"}, {"ID": 2,
"Full_name": "John Sibiya", "Email": "sibiya.com"} ] _next_id = 3
#simple auto-increment

#Home endpoint @app.route("/") #which url are we responding to def
home(): #Method Home returns the statement return
render_template("home.html", page_title = "LetsGo API Home")
```

```
#First Api endpoint @app.route("/info") def info(): return
{ #returning a dictionary "name": "LetsGo API", "version": "1.1",
"status": "Runniing Successfully" } #Status endpoint = route +
function @app.route("/status", methods = ["GET"]) def status():
return jsonify({ "app": "App is running...", "message": "All systems
are optimal!", "users count": len(USERS) })),200

if name == "main": app.run(debug = True)
```

#Adding Users and more operations



```
app.py home.html
1 from flask import Flask, render_template, jsonify, request #added request
2 app = Flask(__name__) #Main Application object and location
3 #in-memory storage (We use Get Method)-Status endpoint
4 USERS = [
5 {"ID": 1, "Full_name": "Peter James", "Email": "james.com"},
6 {"ID": 2, "Full_name": "John Sibiya", "Email": "sibiya.com"}
7 ]
8 _next_id = 3 #simple auto-increment
9
10
11 #Home endpoint
12 @app.route("/") #which url are we responding to
13 def home(): #Method Home returns the statement
14     return render_template("home.html", page_title = "LetsGo API Home")
15
16 #First Api endpoint
17 @app.route("/info")
18 def info():
19     return { #returning a dictionary
20         "name": "LetsGo API",
21         "version": "1.1",
22         "status": "Runniing Successfully"
23     }
24 #Status endpoint = route + function
25 @app.route("/status", methods = ["GET"])
26 def status():
27     return jsonify({
28         "app": "App is running...",
29         "message": "All systems are optimal!",
30         "users count": len(USERS)
31     })),200
32 #Adding users in Functions (route as function) (users list users)
```



```

32 #checking users information (route + function) /users, list_users
33 @app.route("/users", methods = ["GET"])
34 def list_users():
35     return jsonify(USERS), 200
36
37 #endpoint API for adding users
38 @app.route("/addusers", methods = ["POST"])
39 def create_user():
40     global _next_id
41     payload = request.get_json(silent = True)
42
43     if not payload or "Full_name" not in payload:
44         return jsonify({"error": "provide Json with Full name and email"})
45
46     new_user = {
47         "ID": _next_id,
48         "Full_name": payload["Full_name"],
49         "Email": payload["Email"]
50     }
51     USERS.append(new_user)
52     _next_id += 1
53     return jsonify(new_user), 200
54
55 if __name__ == "__main__":
56     app.run(debug = True)
57
58
59

```

```

from flask import Flask, render_template, jsonify, request #added
request app = Flask(name) #Main Application object and location #in-
memory storage (We use Get Method)-Status endpoint USERS = [ {"ID":
1, "Full_name": "Peter James", "Email": "james.com"}, {"ID": 2,
"Full_name": "John Sibiya", "Email": "sibiya.com"} ] _next_id = 3
#simple auto-increment

```

```

#Home endpoint @app.route("/") #which url are we responding to def
home(): #Method Home returns the statement return
render_template("home.html", page_title = "LetsGo API Home")

```

```

#First Api endpoint @app.route("/info") def info(): return
{ #returning a dictionary "name": "LetsGo API", "version": "1.1",
"status": "Runniing Successfully" } #Status endpoint = route +
function @app.route("/status", methods = ["GET"]) def status():
return jsonify({ "app": "App is running...", "message": "All systems
are optimal!", "users count": len(USERS) }), 200 #checking users
information (route + function) /users, list_users

```

```
@app.route("/users", methods = ["GET"]) def list_users(): return
jsonify USERS, 200

#endpoint API for adding users @app.route("/addusers", methods =
["POST"]) def create_user(): global _next_id payload =
request.get_json(silent = True)

if not payload or "Full_name" not in payload:
    return jsonify({"error": "provide Json with Full name and
email"}), 400

new_user = {
    "ID": _next_id,
    "Full_name": payload["Full_name"],
    "Email": payload["Email"]
}
USERS.append(new_user)
_next_id += 1
return jsonify(new_user), 200

if name == "main": app.run(debug = True)
```

Chapter 6 – Machine Learning

*

4.3 Introduction to Machine Learning

What does this page say about "machine"?

Machine learning (ML) is a part of artificial intelligence (AI) that lets computers learn from data to make decisions or predictions without being explicitly programmed. Instead of giving exact instructions, we train ML models with data, and they learn to find patterns.

Why is this important for business?

Machine learning is changing many industries by helping with:

- **Sales Forecasting:** Predicting future sales based on past data.

Python Fundamentals Level 2 - PCAP Certification Training | SciPy 2019

What does this page say about "machine"?

- **Customer Understanding:** Grouping customers to offer them better products.
- **Fraud Detection:** Spotting unusual transactions that might be fraud.

Learning the basics of **machine** learning will help you use these powerful tools to improve your business.

4.4 Simple Linear Regression: Predicting Sales from Advertising

Concept: Simple linear regression is a way to find a straight-line relationship between two numbers. For example, if you increase advertising (one number), do sales (the other number) also increase in a predictable way?

Adding a data-set:

*First make sure all the correct modules are loaded.

```
[ ]: #Linear Regression Model, Compares Two Entities together to gather the relationship between them.
import pandas as pd
from matplotlib import pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

#Loading data-sets
df = pd.read_csv(r"C:\Users\Lenovo-User\Downloads.csv")
```

```
#Linear Regression Model, Compares Two Entities together to gather
the relationship between them.
import pandas as pd
from matplotlib import pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
```

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

```
#Loading data-sets
```

```
df = pd.read_csv(r"C:\Users\Lenovo-User\Downloads.csv")
```

Step 2:

```
•[2]: #Linear Regression Model, Compares Two Entities together to gather the relationship between them.
```

```
import pandas as pd
from matplotlib import pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

```
#Loading data-sets
```

```
#The name of the file is: employee_salary_dataset
```

```
df = pd.read_csv(r"C:\Users\Lenovo-User\Downloads\employee_salary_dataset.csv")
```

```
#To display our data-set, the first 5 rows
```

```
df.head()
```

```
[2]:
```

	Experience_Years	Trainings_Completed	Age	Salary
--	------------------	---------------------	-----	--------

0	6	7	44	26386.89
---	---	---	----	----------

1	3	1	43	13696.15
---	---	---	----	----------

2	12	4	32	33655.75
---	----	---	----	----------

3	14	4	26	38177.61
---	----	---	----	----------

4	10	5	55	34344.67
---	----	---	----	----------

```
[ ]:
```

```
#Linear Regression Model, Compares Two Entities together to gather the relationship between them.
```

```
import pandas as pd
```

```
from matplotlib import pyplot as plt
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.linear_model import LinearRegression
```

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

```
#Loading data-sets
```

```
#The name of the file is: employee_salary_dataset
```

```
df = pd.read_csv(r"C:\Users\Lenovo-User\Downloads\employee_salary_dataset.csv")
```

```
#To display our data-set, the first 5 rows
```

```
df.head()
```

*Shows The first 5 rows and shows the summary of the data.

```
#Explore data
df.shape #dataset size
df.describe() #For summary stats
```

```
[4]:
```

	Experience_Years	Trainings_Completed	Age	Salary
count	200.000000	200.000000	200.000000	200.000000
mean	7.720000	4.725000	40.430000	28857.674250
std	4.431681	2.663616	11.243987	11531.501182
min	0.000000	1.000000	20.000000	-1371.690000
25%	4.000000	2.000000	31.000000	20930.947500
50%	8.000000	4.500000	41.500000	29145.305000
75%	11.000000	7.000000	50.250000	38431.737500
max	15.000000	9.000000	59.000000	57253.430000

*Then to find missing values in a data set.

```
[5]: #Check for missing values
df.isnull().sum()
```

```
[5]: Experience_Years      0
      Trainings_Completed  0
      Age                  0
      Salary               0
      dtype: int64
```

The above means there are no missing values

###

The Histogram modelling to show the relationships:

```
[6]: plt.hist(df["Salary"], bins = 30)
plt.title("Salary Distribution")
plt.xlabel("Salaries")
plt.ylabel("Frequency")
plt.show()
```



```
plt.hist(df["Salary"], bins = 30)
plt.title("Salary Distribution")
plt.xlabel("Salaries")
plt.ylabel("Frequency")
plt.show()
```

The next part is based strictly on the model:

```
[16]: #Selecting dependent and independent variable (X & Y), salary depends on the other Y values
X = df[['Experience_Years', "Trainings_Completed", "Age"]] #The double square brackets: df[] to read our file,
#[] then to extract it as a list that's why there are two
y = df["Salary"]
```

```
[20]: # Split data into training and testing
X_train, X_test, y_train, y_test = train_test_split(X,y, test_size=0.2, random_state=42)
```

```
[21]: # Train the regression model
model = LinearRegression()
model.fit(X_train, y_train)
```

```
[21]: ▾ LinearRegression ⓘ ⓘ
      ▶ Parameters
```

```
[22]: #predict using model
y_pred = model.predict(X_test)
```

```
[23]: # evaluate the model
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
mae, mse, r2
```

```
[23]: (2554.9452897194824, 9432699.176755361, 0.9220463107411718)
```

```
#Selecting dependent and independent variable (X & Y), salary depends
on the other Y values
X = df[['Experience_Years', "Trainings_Completed", "Age"]] #The
double square brackets: df[] to read our file,
#[] then to extract it as a list that's why there are two
y = df["Salary"]
```

Split data into training and testing

```
X_train, X_test, y_train, y_test = train_test_split(X,y,
test_size=0.2, random_state=42) #divides data randomly
```

Train the regression model

```
model = LinearRegression() model.fit(X_train, y_train)
```

```
#predict using model
y_pred = model.predict(X_test)
```

evaluate the model

```
mae = mean_absolute_error(y_test, y_pred) mse =  
mean_squared_error(y_test, y_pred) r2 = r2_score(y_test, y_pred) mae,  
mse, r2 #an r2(r^2) of 1 means perfect, +0.8 = great, 0,5 - 0,7 okay,  
and finally below 0.5 is poor (This is judging your model)
```

```
#an r2(r^2) of 1 means perfect, +0.8 = great, 0,5 - 0,7 okay, and  
finally below 0.5 is poor (This is judging your model)
```

Model coefficienty (How much salary will increase if experience increases)

```
[24]: # check model coefficienty for interpretation  
model.coef_, model.intercept_
```

```
[24]: (array([2518.09324749, 1169.03006933, 105.38625319]),  
      np.float64(-304.31331375868103))
```

```
[ ]:
```

check model coefficienty for interpretation

```
model.coef_, model.intercept_
```

Making a prediction:

```
[5]: #make a real prediction using the model
```

```
sample = [[5,3,30]]  
model.predict(sample)
```

```
#5 years working the company, 3 trainings and 30 years of age you must be making: (R18,954)
```

```
C:\Users\Lenovo-User\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2749: UserWarning: X  
on was fitted with feature names  
warnings.warn(  
    "X on was fitted with feature names  
    %s but you have now provided feature names %s" % (feature_names_in_, feature_names),  
    UserWarning
```

```
[5]: array([18954.83072741])
```



```
#make a real prediction using the model
sample = [[5,3,30]]
model.predict(sample)
#5 years working the company, 3 trainings and 30 years of age you
must be making: (R18,954)
```

.....

20 March 2026

*Most popular models:

```
] : # descriptive, regression, # sarima #arima # prophet
```

What each of the models are used for:

*The descriptive model is when we're focusing on one variable in a dataset. Maximizing all of the dataset's information by generating:

```
: # descriptive - max,min, std,stderr, median, mode, mean
#, regression, # sarima #arima # prophet
users    timespent    productsbought
user1     10         |         I
user2      5
```

Time-spent is our only variable.

*Linear regression - Statistical model that studies the relationship between two or more variables, it's usually the dependent and independent variables.

```
# descriptive - max,min, std,stderr, median, mode, mean
#, regression, # sarima #arima # prophet
users    timespent    productsbought
user1     10         20
user2      5         2
```

Time spent and products bought are the two variables.

The Modules/Libraries explained:

```
#Housing Linear regression
import pandas as pd #A library to read and load documents
from matplotlib import pyplot as plt #This library is powerful at plotting graphs
from sklearn.model_selection import train_test_split #Splits the data into training and testing sets. It's the dependant and independent variable.
from sklearn.linear_model import LinearRegression #building the linear regression model
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score #On the prediction model it tells us how well the dataset value are in...
#running, like we're checking the fault tolerance of the prediction model.
```

```
#Housing Linear regression
import pandas as pd #A library to read and load documents
from matplotlib import pyplot as plt #This library is powerful at
plotting graphs
from sklearn.model_selection import train_test_split #Splits the data
into training and testing sets. It's the dependant and independent
variable.
from sklearn.linear_model import LinearRegression #building the
linear regression model
from sklearn.metrics import mean_absolute_error, mean_squared_error,
r2_score #On the prediction model it tells us how well the dataset
value are in...
#running, like we're checking the fault tolerance of the prediction
model.
import numpy as np #provides support for large, fast, and efficient
multi-dimensional arrays and a vast collection of high-level
mathematical functions
#to operate on these arrays
import seaborn as sns #for creating attractive and informative
statistical graphics
```

The Linear Regression Model:

File: housing_data.csv

```
#Housing Linear regression
import pandas as pd #A library to read and load documents
from matplotlib import pyplot as plt #This library is powerful at plotting graphs
from sklearn.model_selection import train_test_split #Splits the data into training and testing sets. It's the dependant and independent variable.
from sklearn.linear_model import LinearRegression #building the linear regression model
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score #On the prediction model it tells us how well the dataset value are in...
#running, like we're checking the fault tolerance of the prediction model.
import numpy as np #provides support for large, fast, and efficient multi-dimensional arrays and a vast collection of high-level mathematical functions
#to operate on these arrays
import seaborn as sns #for creating attractive and informative statistical graphics
```

*All the modules and packages (Libraries) used for this model example

```
#Housing Linear regression
import pandas as pd #A library to read and load documents
from matplotlib import pyplot as plt #This library is powerful at
plotting graphs
```

```

from sklearn.model_selection import train_test_split #Splits the data
into training and testing sets. It's the dependant and independent
variable.
from sklearn.linear_model import LinearRegression #building the
linear regression model
from sklearn.metrics import mean_absolute_error, mean_squared_error,
r2_score #On the prediction model it tells us how well the dataset
value are in...
#running, like we're checking the fault tolerance of the prediction
model.
import numpy as np #provides support for large, fast, and efficient
multi-dimensional arrays and a vast collection of high-level
mathematical functions
#to operate on these arrays
import seaborn as sns #for creating attractive and informative
statistical graphics

```

Step 2:

```

#Step 2: Loading the dataset
df = pd.read_csv(r"C:\Users\Lenovo-User\Downloads\housing_data.csv")
#Display first 5 rows
print("The first 5 rows of the dataset:")
df.head()

```

The first 5 rows of the dataset:

```

8]:

```

	Square_Footage	Bedrooms	Bathrooms	House_Age	Distance_to_Center	Price
0	2248.0	4	3.0	73	31.9	357393.0
1	1931.0	2	2.0	65	49.4	253265.0
2	2324.0	1	1.2	23	20.5	345975.0
3	2762.0	2	2.0	37	17.3	441420.0
4	1883.0	5	3.5	27	31.6	378537.0

```

#Step 2: Loading the dataset
df = pd.read_csv(r"C:\Users\Lenovo-User\Downloads\housing_data.csv")
#Display first 5 rows
print("The first 5 rows of the dataset:")
df.head()

```

Step 3:

```
[5]: #Display basic statistical summary of the dataset
print("Statistical summary of the dataset:")
df.describe()
```

Statistical summary of the dataset:

```
[5]:
```

	Square_Footage	Bedrooms	Bathrooms	House_Age	Distance_to_Center	Price
count	3000.000000	3000.000000	3000.000000	3000.000000	3000.000000	3000.000000
mean	2016.045667	3.001667	2.589233	49.131667	25.454333	348076.390333
std	493.270041	1.409607	0.976117	28.607676	14.313722	87643.877579
min	500.000000	1.000000	1.000000	0.000000	1.000000	25450.000000
25%	1686.500000	2.000000	1.700000	24.000000	12.875000	292014.000000
50%	2012.000000	3.000000	2.600000	49.000000	25.000000	346059.500000
75%	2337.000000	4.000000	3.500000	74.000000	38.300000	407242.500000
max	3963.000000	5.000000	4.000000	99.000000	49.900000	678225.000000

```
#Display basic statistical summary of the dataset
print("Statistical summary of the dataset:")
df.describe()
```

Step 4:

```
# Displaying information about the dataframe, including data types and non-null values
print("Information about the dataset:")
df.info()
```

Information about the dataset:

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 3000 entries, 0 to 2999
```

```
Data columns (total 6 columns):
```

#	Column	Non-Null Count	Dtype
0	Square_Footage	3000 non-null	float64
1	Bedrooms	3000 non-null	int64
2	Bathrooms	3000 non-null	float64
3	House_Age	3000 non-null	int64
4	Distance_to_Center	3000 non-null	float64
5	Price	3000 non-null	float64

```
dtypes: float64(4), int64(2)
```

```
memory usage: 140.8 KB
```

Displaying information about the dataframe, including data types and non-null values

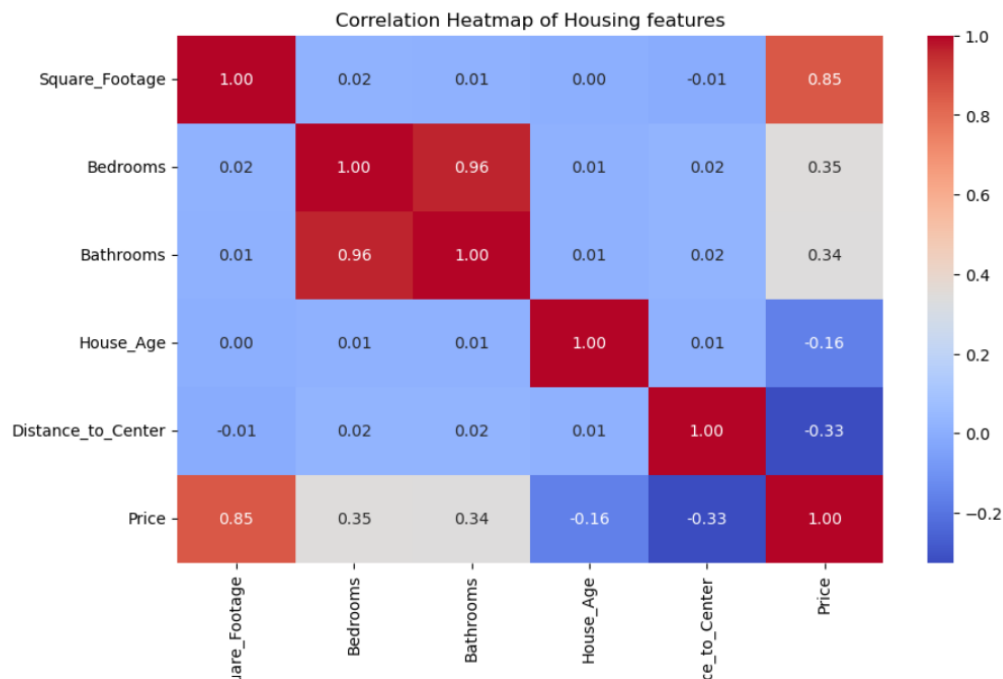
```
print("Information about the dataset:") df.info()
```

Step 5:

```
#EDA (Visualization)
#Correlation Heatmap
#Set size for better readability of the heatmap
plt.figure(figsize =(10,6))
sns.heatmap(df.corr(), annot=True, cmap ="coolwarm", fmt = ".2f") #df.corr is the gradient, cmap is to style it
plt.title("Correlation Heatmap of Housing features")
plt.savefig("correlation_heatmap.png")
plt.close
# Drop rows that has null (df_clean = df.dropna()) - when there is a null value
# Drop rows that has null (df_clean = df.dropna()) (df_clean = df.dropna(subset='Price',)) - specific column

#For our diagram below: #value range 0,7- 0.8 strong positive corr, -0.7, -0.8 -Ittrong -ve corr 0,1 and -1: Weak correlation

[12]: <function matplotlib.pyplot.close(fig: "None | int | str | Figure | Literal['all']" = None) -> 'None'>
```



```
#EDA (Visualization)
#Correlation Heatmap
#Set size for better readability of the heatmap
```

```
plt.figure(figsize=(10,6))
sns.heatmap(df.corr(), annot=True, cmap="coolwarm", fmt=".2f")
#df.corr is the gradient, cmap is to style it
plt.title("Correlation Heatmap of Housing features")
plt.savefig("correlation_heatmap.png")
plt.close
# Drop rows that has null (df_clean = df.dropna()) - when there is a
null value
# Drop rows that has null (df_clean = df.dropna()) (df_clean =
df.dropna(subset='Price',)) - specific column

#For our diagram below: #value range 0,7- 0.8 strong positive corr, -
0.7, -0.8 -Itrong -ve corr 0,1 and -1: Weak correlation
```

Step 6:

```
[16]: # Price vs. Square footage scatter plot
plt.figure(figsize=(10,6))
sns.scatterplot(data=df, x='Square_Footage', y='Price', alpha=0.5)
plt.title("Price vs Square Footage")
#save as a picture(png)
plt.savefig('price_vs_sqft.png')
```



Price vs. Square footage scatter plot

```
plt.figure(figsize = (10,6)) sns.scatterplot(data=df, x
='Square_Footage', y = 'Price', alpha = 0.5) plt.title("Price vs
Square Footage") #save as a picture(png)
plt.savefig('price_vs_sqft.png')
```

*Example looking at other variables for the scatter plot:

```
[19]: #scatter plot 2: (Price vs.Distance_to_Center)
# Price vs. Distance to Center scatter plot
plt.figure(figsize = (10,6))
sns.scatterplot(data=df, x = 'Distance_to_Center', y = 'Price', alpha = 0.5, color = "red")
plt.title("Price vs Distance to Center")
#save as a picture(png)
plt.savefig('price_vs_dst.png')
```



```
#scatter plot 2: (Price vs.Distance_to_Center)
# Price vs. Distance to Center scatter plot
plt.figure(figsize = (10,6))
sns.scatterplot(data=df, x = 'Distance_to_Center', y = 'Price', alpha =
0.5, color = "red")
plt.title("Price vs Distance to Center")
```

```
#save as a picture(png)
plt.savefig('price_vs_dst.png')
```

Stage 7: Building the Regression Model

```
# 4. Build Linear Regression Model
# X will contain all column except Price (The drop method removes the price column)
X = df.drop("Price", axis = 1) # axis =1 specifies that we are dropping a column and not row
y = df["Price"] #To show it is a dependant Variable
print("First 5 rows of the X variable, without the Price:")
X.head()
```

First 5 rows of the X variable, without the Price:

	Square_Footage	Bedrooms	Bathrooms	House_Age	Distance_to_Center
0	2248.0	4	3.0	73	31.9
1	1931.0	2	2.0	65	49.4
2	2324.0	1	1.2	23	20.5
3	2762.0	2	2.0	37	17.3
4	1883.0	5	3.5	27	31.6

4. Build Linear Regression Model

X will contain all column except Price (The drop method removes the price column)

```
X = df.drop("Price", axis = 1) # axis =1 specifies that we are
dropping a column and not row y = df["Price"] #To show it is a
dependant Variable print("First 5 rows of the X variable, without the
Price:") X.head()
```

Step 8:


```
[26]: #The split between the above and the below data-sets into X and y...
      #so the model can manipulate the data-set together
      print("The Price column 1st 5 rows (The dependent variable) Y:")
      y.head()
```

The Price column 1st 5 rows (The dependent variable) Y:

```
[26]: 0    357393.0
      1    253265.0
      2    345975.0
      3    441420.0
      4    378537.0
      Name: Price, dtype: float64
```

```
#The split between the above and the below data-sets into X and y...
#so the model can manipulate the data-set together
print("The Price column 1st 5 rows (The dependent variable) Y:")
y.head()
```

Step 9:

```
#80% is reserved for training, 20% is reserved for Testing, "test_size = 0.2/20%". Random is = 42
X_test, X_train, y_test, y_train = train_test_split(X,y, test_size = 0.2, random_state = 42)
#
print("Shape of the X_Train:", X_train.shape)
print("Shape of the X_Test:", X_test.shape)
print("Shape of the Y_Train:", y_train.shape)
print("Shape of the Y_Test:", y_test.shape)
```

```
Shape of the X_Train: (600, 5)
Shape of the X_Test: (2400, 5)
Shape of the Y_Train: (600,)
Shape of the Y_Test: (2400,)
```

```
# Model training
# initialize the regression model. This create an instance of the model
model = LinearRegression()
# Train the model using the training data
model.fit(X_train, y_train)
# After training, we inspect the model's Learned coefficients and intercepts y=mx+b
print("---Model coefficients---")
for feature, coef in zip (X.columns, model.coef_):
    print(f"{feature}:{coef:.2f}")
```

```
---Model coefficients---
Square_Footage:150.92
Bedrooms:13391.48
Bathrooms:11505.20
House_Age:-513.75
Distance_to_Center:-2038.69
```

```
#80% is reserved for training, 20% is reserved for Testing,
"test_size = 0.2/20%". Random is = 42
X_test, X_train, y_test, y_train = train_test_split(X,y, test_size =
0.2, random_state = 42)
#
print("Shape of the X_Train:", X_train.shape)
print("Shape of the X_Test:", X_test.shape)
print("Shape of the Y_Train:", y_train.shape)
print("Shape of the Y_Test:", y_test.shape)
```

Model training

**initialize the regression model.
This create an instance of the
model**

```
model = LinearRegression()
```

**Train the model using the training
data**

```
model.fit(X_train, y_train)
```

**After training, we inspect the
model's learned coefficients and
intercepts $y=mx+b$**

```
print("---Model coefficients---") for feature, coef in zip  
(X.columns, model.coef_): print(f"{feature}:{coef:.2f}")
```

Step 10:

```
# STEP 5: MODEL EVALUATION AND VISUALISATIONS
# making predictions
# Lets use the trained model to predict house prices for the test set features
# y_pred will contain the predicted price
y_pred = model.predict(X_test)
# Display the first few actual and predicted prices to compare
print('Actual vs Predicted Prices (first 5 from test set):')
comparison_df = pd.DataFrame({'Actual': y_test, 'Predicted': y_pred})
comparison_df.head()
```

Actual vs Predicted Prices (first 5 from test set):

	Actual	Predicted
642	312448.0	339163.089219
700	312205.0	329305.755744
226	369462.0	397709.053785
1697	375669.0	349143.754998
1010	442322.0	436341.237272

```
#Calculate MSE
mse = mean_squared_error(y_test,y_pred)
#Calculate r2 score (model ranking)
r2 = r2_score (y_test,y_pred)
print ("Model performance:")
print (f"Mean squared error (MSE): {mse:.2f}")
print (f"R-squared Score (R2): {r2:.4f}")
```

Model performance:
Mean squared error (MSE): 229831269.07
R-squared Score (R2): 0.9698

STEP 5: MODEL EVALUATION AND VISUALISATIONS

making predictions

Lets use the trained model to predict house prices for the test set features

y_pred will contain the predicted price

```
y_pred = model.predict(X_test)
```

Display the first few actual and predicted prices to compare

```
print('Actual vs Predicted Prices (first 5 from test set):')  
comparison_df = pd. DataFrame({'Actual': y_test,  
'Predicted' :y_pred}) comparison_df.head()
```

```
#Calculate MSE  
mse = mean_squared_error(y_test,y_pred)  
#Calculate r2 score (model ranking)  
r2 = r2_score (y_test,y_pred)  
print ("Model performance:")  
print (f"Mean squared error (MSE): {mse:.2f}")  
print (f"R-squared Score (R2): {r2:.4f}")
```

23 March 2026 - Monday

Smart business expense predictor (Machine learning project)

Welcome to the **Smart Business Expense Predictor** project manual! This hands-on guide is designed to take you through the exciting process of building a real-world application that combines the power of Machine Learning with the flexibility of a Flask web API. As aspiring developers and business professionals, understanding how to leverage data for forecasting is an invaluable skill. This project will provide you with a practical, step-by-step experience in creating an intelligent tool that can predict future business expenses, helping Joseph and other business owners make smarter financial decisions.

We will cover everything from setting up your development environment from scratch, preparing data, training a machine learning model, to deploying that model as a web service accessible via an API. Every line of code will be thoroughly commented, and each concept explained in detail, ensuring that even absolute beginners can follow along and gain a deep understanding.

By the end of this project, you will not only have a functional expense prediction API but also a solid foundation in integrating machine learning into web applications—a highly sought-after skill in today's tech landscape. Let's build something smart!

We're creating a folder first and creating the proper directories:

-Install libraries in the venv directory:

```
Command Prompt - pip insta  X + v
Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

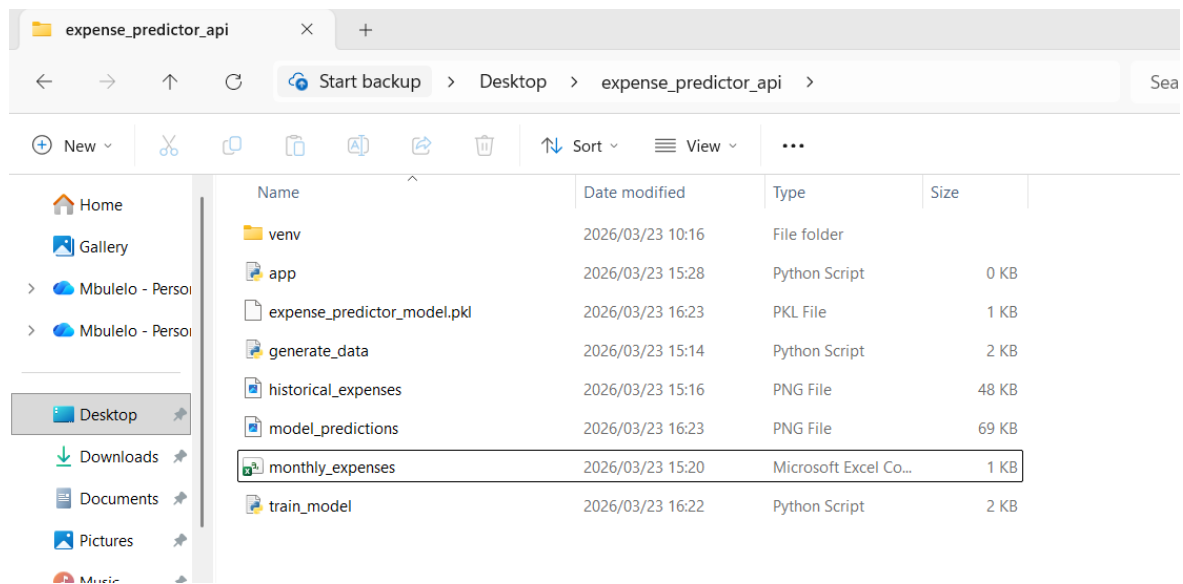
C:\Users\Lenovo-User>cd desktop
C:\Users\Lenovo-User\Desktop>mkdir expense_predictor_api
C:\Users\Lenovo-User\Desktop>cd expense_predictor_api
C:\Users\Lenovo-User\Desktop\expense_predictor_api>python -m venv venv
C:\Users\Lenovo-User\Desktop\expense_predictor_api>
C:\Users\Lenovo-User\Desktop\expense_predictor_api>venv\scripts\activate

(venv) C:\Users\Lenovo-User\Desktop\expense_predictor_api>install Flask numpy pandas matplotlib scikit-learn joblib
'install' is not recognized as an internal or external command,
operable program or batch file.

(venv) C:\Users\Lenovo-User\Desktop\expense_predictor_api> install Flask numpy pandas matplotlib scikit-learn joblib
'install' is not recognized as an internal or external command,
operable program or batch file.

(venv) C:\Users\Lenovo-User\Desktop\expense_predictor_api> pip install Flask numpy pandas matplotlib scikit-learn joblib
```

*We also worked on building a Linear Regression prediction model in sublime. Below are the files attached:



GENERATE_DATA:

```
generate_data.py  x  app.py  x  train_model.py  x  + ▾
1  import pandas as pd
2  import numpy as np
3  import matplotlib.pyplot as plt
4
5  # --- Config for data generations
6  num_months = 36
7  start_date = '2023-01-01'
8  base_expense = 10000
9  monthly_increase = 200
10 random_flactuation = 1500
11
12 print('--- Generating Sythetic Business Expense Data ---')
13
14 # creating a range of monthly dates
15 dates = pd.date_range(start = start_date, periods = num_months, freq =
16
17 # Generate expense
18 expenses = []
19 for i in range(num_months):
20     expense = base_expense + (i*monthly_increase)+ np.random.uniform(-r
21     expenses.append(max(5000, round(expense,2)))
22
23 #Create a pandas dataframe
24 data = pd.DataFrame({
25     'Month': dates,
26     'Expenses_R': expenses
27 })
28 data.set_index('Month', inplace = True)
29
30 print('Data generated successfully. First 5 rows')
31 print(data.head())
32
33 #Visualise the generated data
34 plt.figure(figsize = (12, 6))
35 plt.plot(data.index, data['Expenses_R'], marker = 'o', linestyle = '-',
36 plt.title('Historical Monthly Business Expense')
37 plt.show()
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# --- Config for data generations
num_months = 36
start_date = '2023-01-01'
base_expense = 10000
monthly_increase = 200
random_flactuation = 1500

print('--- Generating Sythetic Business Expense Data ---')
```

```

# creating a range of monthly dates
dates = pd.date_range(start = start_date, periods = num_months, freq
= 'MS')

# Generate expense
expenses = []
for i in range(num_months):
    expense = base_expense + (i*monthly_increase)+ np.random.uniform(-
random_flactuation,random_flactuation)
    expenses.append(max(5000, round(expense,2)))

#Create a pandas dataframe
data = pd.DataFrame({
'Month': dates,
'Expenses_R': expenses
})
data.set_index('Month', inplace = True)

print('Data generated successfully. First 5 rows')
print(data.head())

#Visualise the generated data
plt.figure(figsize = (12, 6))
plt.plot(data.index, data['Expenses_R'], marker = 'o', linestyle = '-
', color = 'blue', markersize=4)
plt.title('Historical Monthly Business Expense')
plt.ylabel('Expenses (R)')
plt.xlabel('Month')
plt.grid(True)
plt.tight_layout()
plt.savefig('historical_expenses.png')
plt.show()
# save data to csv
data.to_csv('monthly_expenses.csv')
print('Historical expense data saved to monthly_expenses.csv')

```

TRAIN_MODEL


```
generate_data.py x train_model.py x app.py +
10 print(" --- Training Smart Business Expense Predictor Model ---")
11
12 #This is in the folder so we don't have to copy properties
13 try:
14     data = pd.read_csv("monthly_expenses.csv", index_col = "Month", par
15     print("Data loaded successfully. Shape:", data.shape)
16 except FileNotFoundError:
17     print("Error: monthly_expense.csv not found. Please run generate_da
18     exit()
19
20 X = np.array(range(1,Len(data)+1)).reshape(-1,1)
21 y = data["Expenses_R"]
22
23
24 X_train,y_train = X, y
25 print(f"Train data size: {len(X_train)} samples.")
26
27 model = LinearRegression()
28
29 model.fit(X_train, y_train)
30
31 print("Model Training Complete")
32 print(f"Model Coefficient (slope): {model.coef_[0]: .2f} R/month")
33 print(f"Model Intercept: {model.intercept_: .2f}")
34
35 y_pred = model.predict(X_train)
36
37 mae = mean_absolute_error(y_train, y_pred)
38 r2 = r2_score(y_train, y_pred)
39 print(f"Model Evaluation")
40 print(f"Mean Absolute Error: {mae:.2f}")
41 print(f"R-Squared score: {r2:.2f}")
42
43 plt.figure(figsize = (12, 6))
44 plt.plot(data.index, y, label = "Actual Expense", marker = 'o', linestyle
45 plt.plot(data.index, y_pred, label= "Predicted Expenses (Model)", lines
46 plt.title("Monthly Business Expense (Actual vs Predicted)")
```

```
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, r2_score
import joblib
import matplotlib.pyplot as plt
from matplotlib import pyplot as plt

print(" --- Training Smart Business Expense Predictor Model ---")

#This is in the folder so we don't have to copy properties
```

```

try:
data = pd.read_csv("monthly_expenses.csv", index_col = "Month",
parse_dates=True)
print("Data loaded successfully. Shape:", data.shape)
except FileNotFoundError:
print("Error: monthly_expense.csv not found. Please run
generate_data.py first.")
exit()

X = np.array(range(1,len(data)+1)).reshape(-1,1)
y = data["Expenses_R"]

X_train,y_train = X, y
print(f"Train data size: {len(X_train)} samples.")

model = LinearRegression()

model.fit(X_train, y_train)

print("Model Training Complete")
print(f"Model Coefficient (slope): {model.coef_[0]: .2f} R/month")
print(f"Model Intercept: {model.intercept_: .2f}")

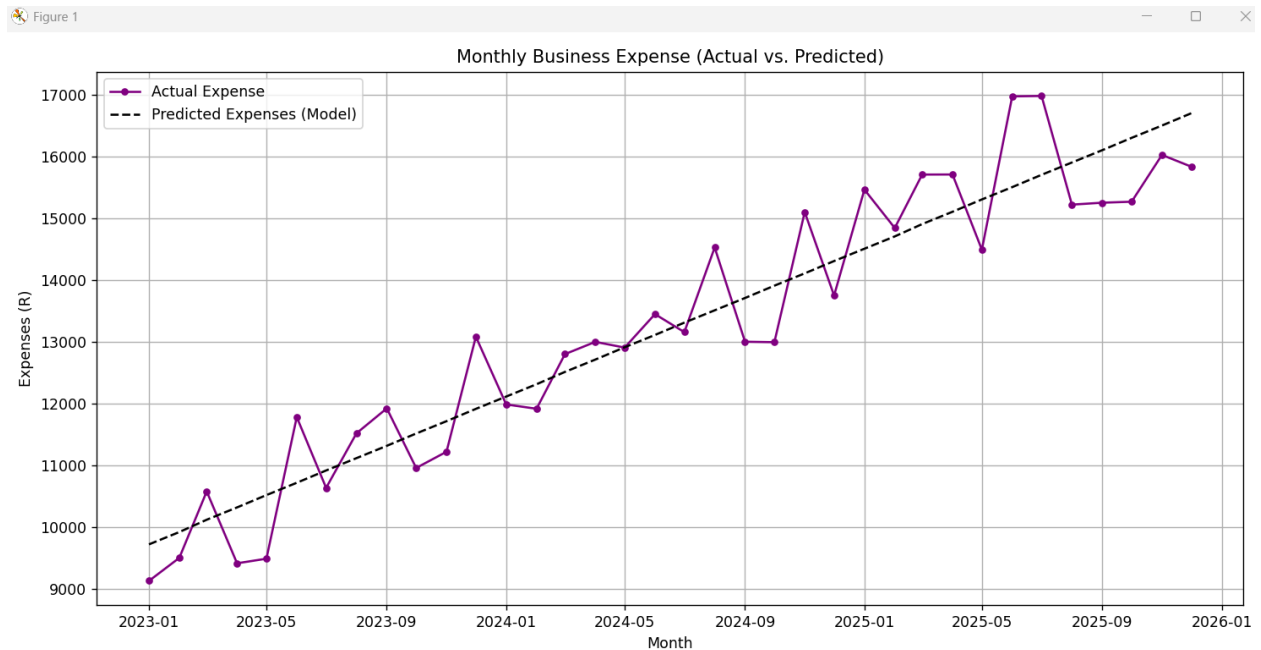
y_pred = model.predict(X_train)

mae = mean_absolute_error(y_train, y_pred)
r2 = r2_score(y_train, y_pred)
print(f"Model Evaluation")
print(f"Mean Absolute Error: {mae:.2f}")
print(f"R-Squared score: {r2:.2f}")

plt.figure(figsize = (12, 6))
plt.plot(data.index, y, label = "Actual Expense", marker = 'o',
linestyle = '-', color = 'purple', markersize=4)
plt.plot(data.index, y_pred, label= "Predicted Expenses (Model)",
linestyle = "--", color = "black")
plt.title("Monthly Business Expense (Actual vs. Predicted)")
plt.ylabel('Expenses (R)')
plt.xlabel('Month')
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.savefig('model_predictions.png')
model_filename = "expense_predictor_model.pkl"
joblib.dump(model, model_filename)

```

```
print("Machine learning model saved!")  
plt.show()
```



Continue tomorrow...

24 March 2026

Here's an app we doing:

```
generate_data.py x app.py x comments.py • home.html x train_model.py x  
1 # 1 user enters month number on the website  
2 # Website sends the number to the flask API  
3 # Flask API will feeds the number into the ML Model  
4 # Model predicts the expense  
5 #Flask sends prediction back to the website  
6 # Website displays the result nicely
```

#THE SYSTEM WE'RE DESIGNING

```
# 1 user enters month number on the website  
# Website sends the number to the flask API  
# Flask API will feeds the number into the ML Model  
# Model predicts the expense  
#Flask sends prediction back to the website  
# Website displays the result nicely
```

```
Command Prompt - python t  ×  +  ∨  
Microsoft Windows [Version 10.0.26200.8037]  
(c) Microsoft Corporation. All rights reserved.  
  
C:\Users\Lenovo-User>cd Desktop  
  
C:\Users\Lenovo-User\Desktop>cd expense_predictor_api  
  
C:\Users\Lenovo-User\Desktop\expense_predictor_api>venv\scripts\activate  
  
(venv) C:\Users\Lenovo-User\Desktop\expense_predictor_api>python train_model.py  
--- Training Smart Business Expense Predictor Model ---  
Data loaded successfully. Shape: (36, 1)  
Train data size: 36 samples.  
Model Training Complete  
Model Coefficient (slope): 199.45 R/month  
Model Intercept: 9521.42  
Model Evaluation  
Mean Absolute Error: 659.15  
R-Squared score: 0.89  
Machine learning model saved!
```

App.py:

```
generate_data.py x train_model.py x app.py x
1 from flask import Flask, jsonify, request, render_template
2 import joblib
3 import numpy as np
4 import os #Checks if the file exists
5
6 app = Flask(__name__) #Instance of the flask application
7
8 #machine learning model loading
9 MODEL_PATH = "expense_predictor_model.pkl" #Folder is within the Virtual
10 model = None
11
12 if os.path.exists(MODEL_PATH):
13     model = joblib.load(MODEL_PATH)
14     print(f"Machine learning model loaded successfully from {MODEL_PATH}")
15 else:
16     print(f"WARNING: Model file {MODEL_PATH} not found. Please run train_model.py")
17
18 @app.route("/")
19 def home():
20     return jsonify({'message': "Welcome to Mbulelo's Smart Business Expense Predictor"})
21
22 @app.route("/predict_expense", methods = ["POST"])
23 def predict_expense():
24     #check if the model was loaded successfully
25     if model == None:
26         return jsonify({"error": "Expense prediction model not available"})
27
28     request_data = request.get_json()
29
30     if not request_data or "month_number" not in request_data:
31         return jsonify({"error": "Please provide 'month_number' in the request"})
32     try:
33         month_number = int(request_data["month_number"])
34
35         if month_number <= 0:
36             return jsonify({"error": "month_number cannot be negative, please provide a positive integer"})
37         #prediction input = np.array([[month_number]])
```

```
from flask import Flask, jsonify, request, render_template
import joblib
import numpy as np
import os #Checks if the file exists

app = Flask(name) #Instance of the flask application

#machine learning model loading
MODEL_PATH = "expense_predictor_model.pkl" #Folder is within the Virtual environment
model = None

if os.path.exists(MODEL_PATH):
    model = joblib.load(MODEL_PATH)
    print(f"Machine learning model loaded successfully from
```

```

{MODEL_PATH}") else: print(f"WARNING: Model file {MODEL_PATH} not
found. Please run train_model.py first")

@app.route("/") def home(): return render_template("home.html")

@app.route("/predict_expense", methods = ["POST"]) def
predict_expense(): #check if the model was loaded successfully if
model == None: return jsonify({"error": "Expense prediction model not
available. Please train the model first."}),500

request_data = request.get_json()

if not request_data or "month_number" not in request_data:
    return({"error": "Please provide 'month_number' in the request
body"}), 400 #bad request error (400)
try:
    month_number = int(request_data["month_number"])

    if month_number <= 0:
        return jsonify({"error": "month_number cannot be negative,
must be positive integer"}),400
    prediction_input = np.array([[month_number]])

    predicted_expense = model.predict(prediction_input) [0]
    predicted_expense = max(0, round(predicted_expense, 2))

    return jsonify({
        "month_number": month_number,
        "predicted_expense_R": predicted_expense
    })
except ValueError:
    return jsonify({"error": "invalid 'month_number' value. Must be
an integer"}), 400
except Exception as e :
    return jsonify({"error": f"An unexpected error occurred during
prediction: {str(e)}"}), 500

#Run the flask Application if name == 'main': app.run(debug = True)

```

The prediction model is running through the second command-line window, the previous one is running in the back:

```
Command Prompt
C:\Users\Lenovo-User>cd Desktop
C:\Users\Lenovo-User\Desktop>>cd expense_predictor_api
'expense_predictor_api' is not recognized as an internal or external command,
operable program or batch file.
C:\Users\Lenovo-User\Desktop>cd expense_predictor_api
C:\Users\Lenovo-User\Desktop\expense_predictor_api>venv\scripts\activate
(venv) C:\Users\Lenovo-User\Desktop\expense_predictor_api> python app.py
Machine learning model loaded successfully from expense_predictor_model.pkl
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
Machine learning model loaded successfully from expense_predictor_model.pkl
* Debugger is active!
* Debugger PIN: 105-873-671
(venv) C:\Users\Lenovo-User\Desktop\expense_predictor_api> curl -X POST -H "Content-Type:application/json" -d '{"month_
number":37}' http://127.0.0.1:5000/predict_expense
{
  "month_number": 37,
  "predicted_expense_R": 16901.18
}
(venv) C:\Users\Lenovo-User\Desktop\expense_predictor_api>
```

*It's using the POST to send information from the 1st terminal to the 2nd.

```
curl -X POST -H "Content-Type:application/json" -d"
{"month_number":37}" http://127.0.0.1:5000/predict_expense
```

*All of the above was for the backend API, now we're displaying front-end for the users:

```
generate_data.py x train_model.py x app.py x home.html x + ▾
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>Business Expense Predictor</title>
5   <style>
6     body { font-family: Arial; background: #f4f4f4; margin:0; }
7     .navbar { background: #003366; padding: 15px; color: white;
8               text-align: center; }
9     .container { margin: 40px auto; width: 400px; background:
10                  white; padding: 25px;
11                  border-radius: 8px; box-shadow: 0 0 10px rgba(0,0
12                  ,0,0.1); }
13     input[type="number"] { width: 100%; padding: 10px; margin-top:
14                            10px;
15                            font-size: 16px; }
16     button { width: 100%; padding: 10px; background: #003366;
17               color: white;
18               border: none; margin-top: 10px; font-size: 17px;
19               cursor: pointer; }
20     button:hover { background: #0055aa; }
21     .result-box { margin-top: 20px; padding: 15px; background: #
22                    e8f0fe;
23                    border-left: 4px solid #003366; }
24     footer { text-align: center; padding: 20px; margin-top: 40px;
25               color: gray; }
26   </style>
27
28   <script>
29     async function predictExpense() {
30       const monthNumber = document.getElementById('month').value
31       ;
32
33       if (!monthNumber) {
34         alert("Please enter a month number.");
35         return;
36       }
37     }
38   </script>
39 </head>
40 <body>
41   <div class="navbar">
42     <h1>Business Expense Predictor</h1>
43   </div>
44   <div class="container">
45     <input type="number" id="month" value="" />
46     <button type="button" value="Predict Expense" />
47   </div>
48   <div class="result-box">
49     <div></div>
50   </div>
51   <div class="footer">
52     <p>© 2020 Business Expense Predictor</p>
53   </div>
54 </body>
55 </html>
```



```
generate_data.py x train_model.py x app.py x home.html x + ▾
20 <script>
21   async function predictExpense() {
22     const monthNumber = document.getElementById('month').value
23     ;
24     if (!monthNumber) {
25       alert("Please enter a month number.");
26       return;
27     }
28
29     const response = await fetch('/predict_expense', {
30       method: 'POST',
31       headers: {'Content-Type': 'application/json'},
32       body: JSON.stringify({ "month_number": monthNumber })
33     });
34
35     const data = await response.json();
36
37     if (data.error) {
38       document.getElementById("result").innerHTML =
39         `<div class='result-box'>Error: ${data.error}
40         </div>`;
41     } else {
42       document.getElementById("result").innerHTML =
43         `<div class='result-box'>
44         <b>Prediction</b> for Month: ${data.month_number}
45         </b><br>
46         Estimated Expense: <b>R ${data.
47         predicted_expense_R}</b>
48         </div>`;
49     }
50   }
51 </script>
52 </head>
```

```
generate_data.py  train_model.py  app.py  home.html
42     <div class='result-box'>
43         <b>Prediction for Month: ${data.month_number}</b><br>
44         Estimated Expense: <b>R ${data.
45             predicted_expense_R}</b>
46     </div>`;
47     }
48 </script>
49 </head>
50
51 <body>
52     <div class="navbar">
53         <h2>Mbulelo's Smart Business Expense Predictor</h2>
54     </div>
55
56     <div class="container">
57         <h3>Enter Month Number</h3>
58         <input type="number" id="month" placeholder="e.g., 37" min="1"
59             >
60         <button onclick="predictExpense()">Predict Expense</button>
61
62         <div id="result"></div>
63     </div>
64
65     <footer>
66         © 2026 Mbulelo Mtshu | Smart Predictive Systems
67     </footer>
68 </body>
</html>
```

Home.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Business Expense Predictor</title>
    <style>
        body { font-family: Arial; background: #f4f4f4; margin:0; }
        .navbar { background: #003366; padding: 15px; color: white;
text-align: center; }
        .container { margin: 40px auto; width: 400px; background:
white; padding: 25px;
border-radius: 8px; box-shadow: 0 0 10px
rgba(0,0,0,0.1); }
        input[type="number"] { width: 100%; padding: 10px; margin-
top: 10px;
font-size: 16px; }
```

```

        button { width: 100%; padding: 10px; background: #003366;
color: white;
                border: none; margin-top: 10px; font-size: 17px;
cursor: pointer; }
        button:hover { background: #0055aa; }
        .result-box { margin-top: 20px; padding: 15px; background:
#e8f0fe;
                border-left: 4px solid #003366; }
        footer { text-align: center; padding: 20px; margin-top: 40px;
color: gray; }
</style>

<script>
    async function predictExpense() {
        const monthNumber =
document.getElementById('month').value;

        if (!monthNumber) {
            alert("Please enter a month number.");
            return;
        }

        const response = await fetch('/predict_expense', {
            method: 'POST',
            headers: {'Content-Type': 'application/json'},
            body: JSON.stringify({ "month_number": monthNumber })
        });

        const data = await response.json();

        if (data.error) {
            document.getElementById("result").innerHTML =
                `<div class='result-box'>Error:
${data.error}</div>`;
        } else {
            document.getElementById("result").innerHTML =
                `<div class='result-box'>
                    <b>Prediction for Month:
${data.month_number}</b><br>
                    Estimated Expense: <b>R
${data.predicted_expense_R}</b>
                </div>`;
        }
    }
</script>
</head>

```

```

<body>
  <div class="navbar">
    <h2>Mbulelo's Smart Business Expense Predictor</h2>
  </div>

  <div class="container">
    <h3>Enter Month Number</h3>
    <input type="number" id="month" placeholder="e.g., 37"
min="1">
    <button onclick="predictExpense()">Predict Expense</button>

    <div id="result"></div>
  </div>

  <footer>
    © 2026 Mbulelo Mtshu | Smart Predictive Systems
  </footer>
</body>
</html>

```

The Human readable version:

← → ↻ ⓘ 127.0.0.1:5000

☰

Mbulelo's Smart Business Expense Predictor

Enter Month Number

Predict Expense

Prediction for Month: 12
Estimated Expense: R 11914.86

© 2026 Mbulelo Mtshu | Smart Predictive Systems

*Short way/fastest way to display an Html file. (Home.html)

A screenshot of a code editor with multiple tabs. The active tab is 'webapp.py'. The code in the editor is as follows:

```
1 from flask import Flask, render_template
2
3 app = Flask(__name__)
4
5 @app.route('/')
6 def home():
7     return render_template('home.html')
8 if __name__ == '__main__':
9     app.run(debug = True)
```

My understanding of the App:

We created 4 python files using Sublime text editor and command prompt, namely:

- generate_data.py (creates a csv file on our behalf, not really important but good to know, easier if you already have a file(csv))
- train_model.py (Working with the datasets, creating a model and training it for prediction)
- app.py (This is the API that links the model with the front-end, it links train_model.py to home.html)
- home.html (Displays the human readable version of the prediction model)

.....

PCAP Questions, notes to know:

-An interpreter reads line by line and its job is to convert from high-level language to a machine code, compiler is faster than an interpreter. A compiler builds everything all together, all the files then runs it after, for example Python is an interpreter language, and a compiler language is C#.

-Assembler is a low-level language

.....

25 March 2026

PyQt6 (Creating a GUI – Graphical User Interface)

```
Command Prompt

C:\Users\Lenovo-User>cd Desktop
C:\Users\Lenovo-User\Desktop>md pyqt6
C:\Users\Lenovo-User\Desktop>cd pyqt6
C:\Users\Lenovo-User\Desktop\pyqt6>pip install pyqt6
Collecting pyqt6
  Downloading pyqt6-6.10.2-cp39-abi3-win_amd64.whl.metadata (2.2 kB)
Collecting PyQt6-sip<14,>=13.8 (from pyqt6)
  Downloading pyqt6_sip-13.11.1-cp314-cp314-win_amd64.whl.metadata (516 bytes)
Collecting PyQt6-Qt6<6.11.0,>=6.10.0 (from pyqt6)
  Downloading pyqt6_qt6-6.10.2-py3-none-win_amd64.whl.metadata (552 bytes)
Download pyqt6-6.10.2-cp39-abi3-win_amd64.whl (26.0 MB)
26.0/26.0 MB 6.9 MB/s 0:00:03
Download pyqt6_qt6-6.10.2-py3-none-win_amd64.whl (78.4 MB)
78.4/78.4 MB 6.3 MB/s 0:00:12
Download pyqt6_sip-13.11.1-cp314-cp314-win_amd64.whl (55 kB)
Installing collected packages: PyQt6-Qt6, PyQt6-sip, pyqt6
2/3 [pyqt6] WARNING: The scripts pylupdate6.exe and pyuic6.exe are installed in 'C:\Users\Lenovo-User\AppData\Local\Python\pythoncore-3.14-64\Scripts' which is not on PATH.
Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.
Successfully installed PyQt6-Qt6-6.10.2 PyQt6-sip-13.11.1 pyqt6-6.10.2

[notice] A new release of pip is available: 25.3 -> 26.0.1
[notice] To update, run: C:\Users\Lenovo-User\AppData\Local\Python\pythoncore-3.14-64\python.exe -m pip install --upgrade pip

C:\Users\Lenovo-User\Desktop\pyqt6>

Command Prompt

C:\Users\Lenovo-User\Desktop\pyqt6>pip install pyqt6-tools
Collecting pyqt6-tools
  Downloading pyqt6_tools-6.4.2.3-py3-none-any.whl.metadata (8.3 kB)
Collecting click (from pyqt6-tools)
  Using cached click-8.3.1-py3-none-any.whl.metadata (2.6 kB)
Collecting pyqt6==6.4.2 (from pyqt6-tools)
  Downloading PyQt6-6.4.2-cp37-abi3-win_amd64.whl.metadata (2.2 kB)
INFO: pip is looking at multiple versions of pyqt6-tools to determine which version is compatible with other requirements. This could take a while.
Collecting pyqt6-tools
  Downloading pyqt6_tools-6.3.1.3-py3-none-any.whl.metadata (8.3 kB)
Collecting pyqt6==6.3.1 (from pyqt6-tools)
  Downloading PyQt6-6.3.1-cp37-abi3-win_amd64.whl.metadata (2.2 kB)
Collecting pyqt6-tools
  Downloading pyqt6_tools-6.1.0.3.2-py3-none-any.whl.metadata (8.3 kB)
Collecting pyqt6==6.1.0 (from pyqt6-tools)
  Downloading PyQt6-6.1.0.tar.gz (946 kB)
946.9/946.9 kB 4.7 MB/s 0:00:00

Installing build dependencies ... done
Getting requirements to build wheel ... done
Preparing metadata (pyproject.toml) ... error
error: subprocess-exited-with-error

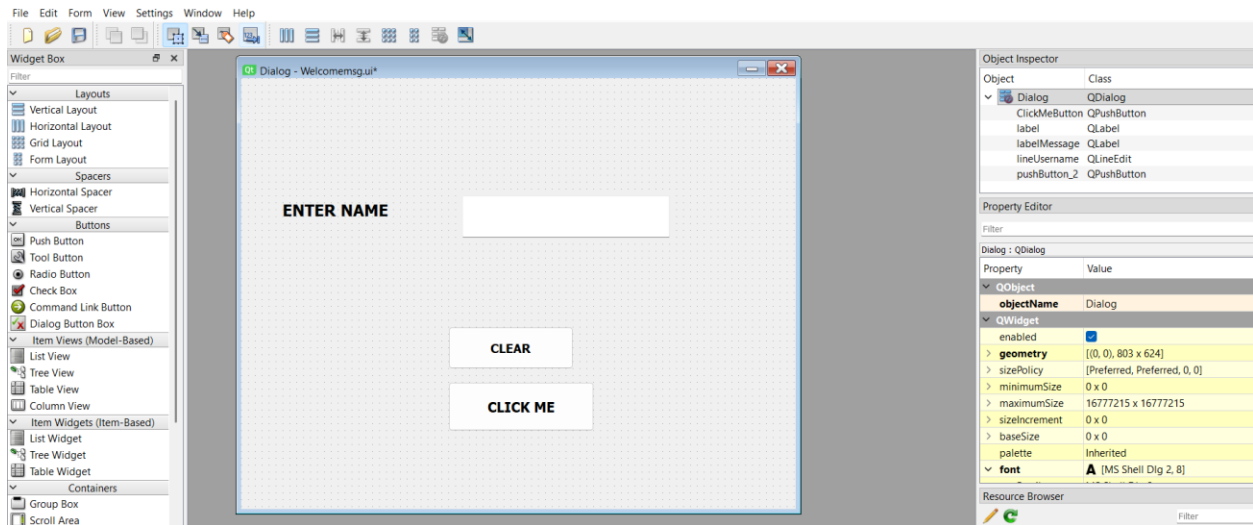
x Preparing metadata (pyproject.toml) did not run successfully.
  exit code: 1
  > [32 lines of output]
     pyproject.toml: line 7: using '[tool.sip.metadata]' to specify the project metadata is deprecated and will be removed in SIP v7.0.0, use '[project]' instead
     Traceback (most recent call last):
```

*We installed a PyQt6 application.

.....

Creating the GUI:

*Qt Designer



The file conversion done on the command prompt, our file was named welcomemsg (the above GUI file)

```

Command Prompt
Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Lenovo-User>cd Desktop
C:\Users\Lenovo-User\Desktop>cd pyqt6
C:\Users\Lenovo-User\Desktop\pyqt6>python -m PyQt6.uic.pyuic -o welcomemsg.py welcomemsg.ui

```

`python -m PyQt6.uic.pyuic -o ...`

`python -m PyQt6.uic.pyuic -o till.py till.ui`

*Open up the Python IDLE:

```
callwelcomemsg.py - C:/Users/Lenovo-User/Desktop/pyqt6/callwelcomemsg.py (3.14.3)
File Edit Format Run Options Window Help
import sys
from PyQt6 import QtWidgets
from Welcomemsg import Ui_Dialog

#Standard parameter
class MyForm(QtWidgets.QDialog):
    def __init__(self, parent = None):
        super().__init__(parent)

        self.ui = Ui_Dialog()
        self.ui.setupUi (self)

        #Connect button
        self.ui.ClickMeButton.clicked.connect(self.dispmesssage)

    def dispmesssage(self):
        username = self.ui.lineUsername.text()#calling an object is self.ui.Name_of_object
        self.ui.labelMessage.setText(f"Hello {username}")

if __name__ == "__main__":
    app = QtWidgets.QApplication(sys.argv)
    myapp = MyForm()
    myapp.show()
    sys.exit(app.exec())
```

```
import sys
from PyQt6 import QtWidgets
from Welcomemsg import Ui_Dialog

#Standard parameter
class MyForm(QtWidgets.QDialog):
    def init (self, parent = None):
        super().init(parent)

        self.ui = Ui_Dialog()
        self.ui.setupUi (self)

        #Connect button
        self.ui.ClickMeButton.clicked.connect(self.dispmesssage)

    def dispmesssage(self):
        username = self.ui.lineUsername.text()#calling an object is self.ui.Name_of_object
        self.ui.labelMessage.setText(f"Hello {username}")

if name == "main":
    app = QtWidgets.QApplication(sys.argv)
    myapp = MyForm()
    myapp.show()
    sys.exit(app.exec())
```


The Till

```
*untitled
File Edit Format Run Options Window Help

import sys
from PyQt6 import QtWidgets
from till import Ui_Dialog

class MyForm(QtWidgets.QDialog):
    def __init__(self, parent = None):
        super().__init__(parent)

        self.ui = Ui_Dialog()
        self.ui.setupUi(self)

        # Connect button
        self.ui.calculate.clicked.connect(self.dispmesssage)

    def dispmesssage(self):
        q = int(self.ui.quantity.text())
        p = int(self.ui.price.text())
        d = int(self.ui.discount.text())

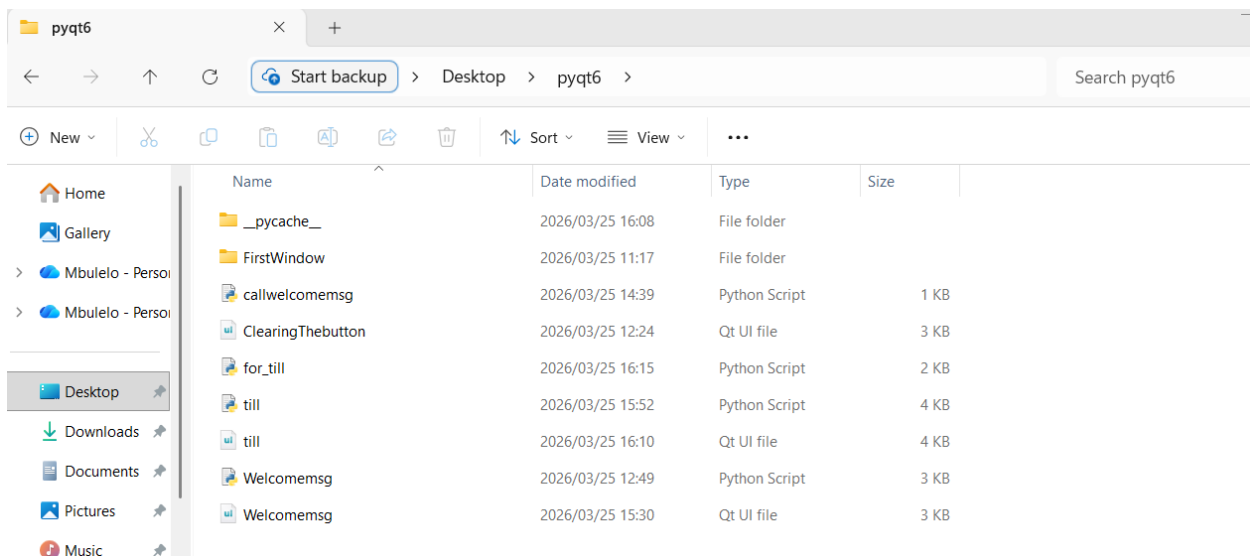
        totalamt = q*p
        disc = totalamt*(d/100)
        netamount = totalamt - disc

        self.ui.results.setText(f'Total amount: {totalamt}, Discount amount: {disc}, Net amount: {netamount}')

if __name__ == "__main__":
    app = QtWidgets.QApplication(sys.argv)
    myapp = MyForm()
    myapp.show()
    sys.exit(app.exec())
```

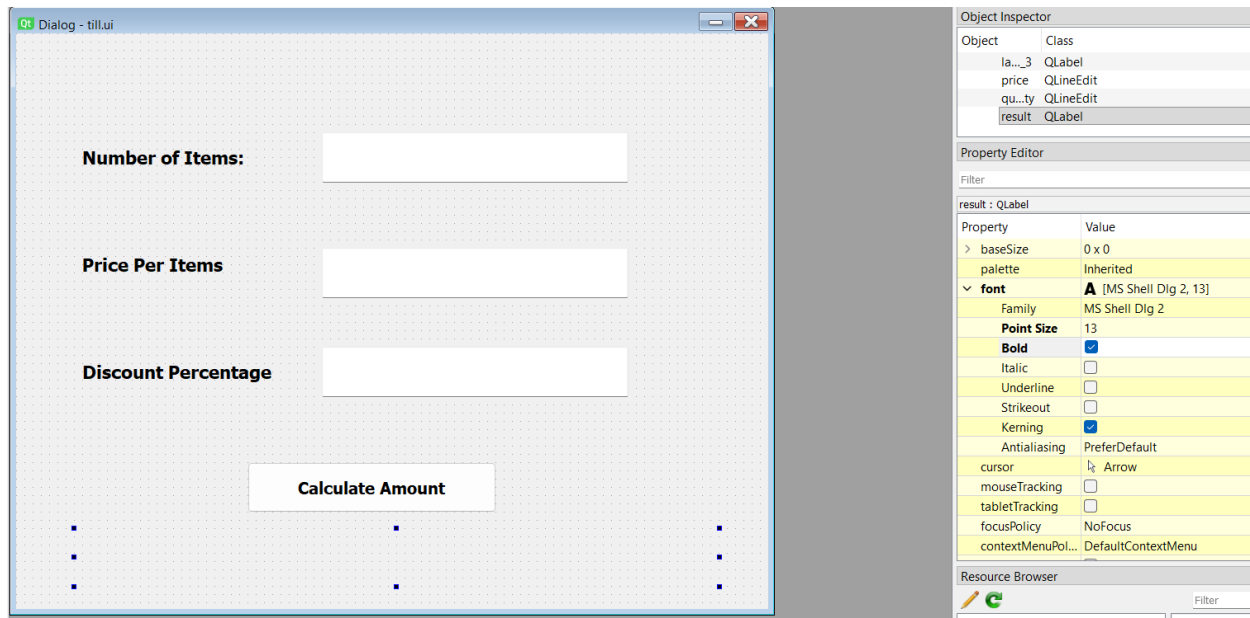
My Version (Calculates Discount):

-The files created by the CMD



Name	Date modified	Type	Size
__pycache__	2026/03/25 16:08	File folder	
FirstWindow	2026/03/25 11:17	File folder	
callwelcomemsg	2026/03/25 14:39	Python Script	1 KB
ClearingThebutton	2026/03/25 12:24	Qt UI file	3 KB
for_till	2026/03/25 16:15	Python Script	2 KB
till	2026/03/25 15:52	Python Script	4 KB
till	2026/03/25 16:10	Qt UI file	4 KB
Welcomemsg	2026/03/25 12:49	Python Script	3 KB
Welcomemsg	2026/03/25 15:30	Qt UI file	3 KB

-The GUI Saved locally inside the file



-The Command prompt to link the GUI with the IDLE (back end code)- we have to convert the file from a .ui to a .py format in order for the IDLE to be able to fetch it:

In this case the till.ui (GUI) must be converted to a till.py for us to use it with the back end code in the IDLE named for_till.py (All the back end functionality)

```

Command Prompt
Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Lenovo-User>cd Desktop
C:\Users\Lenovo-User\Desktop>cd pyqt6
C:\Users\Lenovo-User\Desktop\pyqt6>python -m PyQt6.uic.pyuic -o till.py till.ui
C:\Users\Lenovo-User\AppData\Local\Python\pythoncore-3.14-64\python.exe: can't open file 'C:\\Users\\Lenovo-User\\Desktop\\pyqt6\\-m': [Errno 2] No such file or directory
C:\Users\Lenovo-User\Desktop\pyqt6>python -m PyQt6.uic.pyuic -o till.py till.ui
C:\Users\Lenovo-User\AppData\Local\Python\pythoncore-3.14-64\python.exe: can't open file 'C:\\Users\\Lenovo-User\\Desktop\\pyqt6\\-m': [Errno 2] No such file or directory
C:\Users\Lenovo-User\Desktop\pyqt6>python -m PyQt6.uic.pyuic -o till.py till.ui
C:\Users\Lenovo-User\AppData\Local\Python\pythoncore-3.14-64\python.exe: Error while finding module specification for 'PyQt6.uic.pyuic' (ModuleNotFoundError: No module named 'PyQt6')
C:\Users\Lenovo-User\Desktop\pyqt6>python -m PyQt6.uic.pyuic -o till.py till.ui

```

-The for_till.py code, it is also a template

```

*for_till.py - C:/Users/Lenovo-User/Desktop/pyqt6/for_till.py (3.14.3)*
File Edit Format Run Options Window Help
import sys
from PyQt6 import QtWidgets
from till import Ui_Dialog

#Standard parameter
class MyForm(QtWidgets.QDialog):
    def __init__(self, parent = None):
        super().__init__(parent)

        self.ui = Ui_Dialog()
        self.ui.setupUi (self)

        #Connect button
        self.ui.calculate.clicked.connect(self.dispmesssage)

    def dispmesssage(self):
        numOfItems =int(self.ui.quantity.text())#calling an object is self.ui.Name_of_object
        pricePerItems = int(self.ui.price.text())#calling an object is self.ui.Name_of_object
        discountPercentage = int(self.ui.discount.text())#calling an object is self.ui.Name_of_object

        totalamt = numOfItems*pricePerItems
        disc = totalamt*(discountPercentage/100)
        netamount = totalamt - disc
        self.ui.result.setText(f'Total amount: {totalamt}, Discount amount: {disc}, Net amount: {netamount} ')

if __name__ == "__main__":
    app = QtWidgets.QApplication(sys.argv)
    myapp = MyForm()
    myapp.show()
    sys.exit(app.exec())

```

```

import sys from PyQt6 import QtWidgets from till import Ui_Dialog

#Standard parameter class MyForm(QtWidgets.QDialog): def init (self,
parent = None): super().init(parent)

    self.ui = Ui_Dialog()
    self.ui.setupUi (self)

    #Connect button
    self.ui.calculate.clicked.connect(self.dispmesssage)

def dispmesssage(self):
    numOfItems =int(self.ui.quantity.text())#calling an object is
self.ui.Name_of_object
    pricePerItems = int(self.ui.price.text())#calling an object is
self.ui.Name_of_object
    discountPercentage = int(self.ui.discount.text())#calling an
object is self.ui.Name_of_object

```

```
totalamt = numOfItems*pricePerItems
disc = totalamt*(discountPercentage/100)
netamount = totalamt - disc
self.ui.result.setText(f'Total amount: {totalamt}, Discount
amount: {disc}, Net amount: {netamount} ')

if name == "main": app = QtWidgets.QApplication(sys.argv) myapp =
MyForm() myapp.show() sys.exit(app.exec())
```

.....

26 March 2026

The Arithmetic Operation:

The conversion function: `python -m PyQt6.uic.pyuic -o calculatefunc.py calculatefunc.ui`

The screenshot shows a PyQt6 calculator application window. It has a light gray background. At the top, there are two input fields for numbers. The first field is labeled "Enter 1st No:" and contains the value "120". The second field is labeled "Enter 2nd No:" and contains the value "2". Below these fields, there are four radio buttons for arithmetic operations: "Add", "Subtract", "Multiply", and "Divide". The "Divide" radio button is selected, indicated by a blue dot. To the right of the radio buttons is a blue-outlined button labeled "COMPUTE". At the bottom left, there is a text label that says "The result is 60.0".

```
callcalculatefunc.py - C:/Users/Lenovo-User/Desktop/pyqt6/callcalculatefunc.py (3.14.3)
File Edit Format Run Options Window Help

import sys
from PyQt6 import QtWidgets
from calculatefunc import Ui_Dialog

#Standard parameter
class MyForm(QtWidgets.QDialog):
    def __init__(self, parent = None):
        super().__init__(parent)

        self.ui = Ui_Dialog()
        self.ui.setupUi(self)

        #Connect button
        self.ui.compute.clicked.connect(self.arithmetic_func)
        self.ui.add.setChecked(True)

    def arithmetic_func(self):
        f_num = int(self.ui.num1.text())
        s_num = int(self.ui.num2.text())

        if self.ui.add.isChecked() == True:
            result = (f_num + s_num)
        if self.ui.subtract.isChecked() == True:
            result = (f_num - s_num)
        if self.ui.multiply.isChecked() == True:
            result = (f_num * s_num)
        if self.ui.divide.isChecked() == True:
            result = (f_num / s_num)
        self.ui.results.setText(f"The result is {result}") #Set text displays on

if __name__ == "__main__":
    app = QtWidgets.QApplication(sys.argv)
    myapp = MyForm()
    myapp.show()
    sys.exit(app.exec())
```

```
import sys from PyQt6 import QtWidgets from calculatefunc import
Ui_Dialog

#Standard parameter class MyForm(QtWidgets.QDialog): def init (self,
parent = None): super().init(parent)

    self.ui = Ui_Dialog()
    self.ui.setupUi(self)

    #Connect button
    self.ui.compute.clicked.connect(self.arithmetic_func)
```

```
self.ui.add.setChecked(True)

def arithmetic_func(self):
    f_num = int(self.ui.num1.text())
    s_num = int(self.ui.num2.text())

    if self.ui.add.isChecked() == True:
        result = (f_num + s_num)
    if self.ui.subtract.isChecked() == True:
        result = (f_num - s_num)
    if self.ui.multiply.isChecked() == True:
        result = (f_num * s_num)
    if self.ui.divide.isChecked() == True:
        result = (f_num / s_num)
    self.ui.results.setText(f"The result is {result}") #Set text
displays on the label

if name == "main": app = QtWidgets.QApplication(sys.argv) myapp =
MyForm() myapp.show() sys.exit(app.exec())
```

The XYZ Gourmet:

Dialog

XYZ ONLINE GOURMET

☒ PIZZA -R220

☐ HOT DOG -R50

☐ FRENCH FRIES -R60

☒ SODAS -R30

☐ ORANGE JUICE -R35

☐ BEEF BURGER -R110

Order

The Total Amount: R250

Alternatively, this is how to implement the same code:

```
callgourmet.py - C:/Users/Lenovo-User/Desktop/pyqt6/callgourmet.py (3.14.3)
File Edit Format Run Options Window Help
import sys
from PyQt6 import QtWidgets
from gourmetonline import Ui_Dialog

#Standard parameter
class MyForm(QtWidgets.QDialog):
    def __init__(self, parent = None):
        super().__init__(parent)

        self.ui = Ui_Dialog()
        self.ui.setupUi(self)

        #Connect button
        self.ui.orderbtn.clicked.connect(self.order_menu)

    def order_menu(self):
        result = 0
        # Evaluate each checkbox state individually
        if self.ui.pz.isChecked():
            result += 220
        if self.ui.hg.isChecked():
            result += 50
        if self.ui.ff.isChecked():
            result += 60
        if self.ui.sd.isChecked():
            result += 30
        if self.ui.oj.isChecked():
            result += 35
        if self.ui.bb.isChecked():
            result += 110
        self.ui.result.setText(f"The Total Amount: R{result}") #Set text display

if __name__ == "__main__":
    app = QtWidgets.QApplication(sys.argv)
    myapp = MyForm()
    myapp.show()
    sys.exit(app.exec())
```

```
import sys from PyQt6 import QtWidgets from gourmetonline import
Ui_Dialog

#Standard parameter class MyForm(QtWidgets.QDialog): def init (self,
parent = None): super().init(parent)

    self.ui = Ui_Dialog()
    self.ui.setupUi(self)

    #Connect button
    self.ui.orderbtn.clicked.connect(self.order_menu)
```



```
def order_menu(self):
    result = 0
    # Evaluate each checkbox state individually
    if self.ui.pz.isChecked():
        result += 220
    if self.ui.hg.isChecked():
        result += 50
    if self.ui.ff.isChecked():
        result += 60
    if self.ui.sd.isChecked():
        result += 30
    if self.ui.oj.isChecked():
        result += 35
    if self.ui.bb.isChecked():
        result += 110
    self.ui.result.setText(f"The Total Amount: R{result}") #Set text
displays on the label

if name == "main": app = QtWidgets.QApplication(sys.argv) myapp =
MyForm() myapp.show() sys.exit(app.exec())
```

*Recap using the PCAP questions

.....

```
callgourmet.py - C:/Users/Lenovo-User/Desktop/pyqt6/callgourmet.py (3.14.3)
File Edit Format Run Options Window Help
import sys
from PyQt6 import QtWidgets
from gourmetonline import Ui_Dialog

#Standard parameter
class MyForm(QtWidgets.QDialog):
    def __init__(self, parent = None):
        super().__init__(parent)

        self.ui = Ui_Dialog()
        self.ui.setupUi(self)

        #Connect button
        self.ui.orderbtn.clicked.connect(self.order_menu)

    def order_menu(self):
        result = 0
        price = {
            self.ui.pz:220,
            self.ui.hg:50,
            self.ui.ff:60,
            self.ui.sd:30,
            self.ui.oj:35,
            self.ui.bb:110
        }
        for chb, amnt in price.items():
            if chb.isChecked():
                result += amnt
        self.ui.result.setText(f"The Total Amount: R{result}") #Set text display

if __name__ == "__main__":
    app = QtWidgets.QApplication(sys.argv)
    myapp = MyForm()
    myapp.show()
    sys.exit(app.exec())
```

```
import sys from PyQt6 import QtWidgets from gourmetonline import
Ui_Dialog

#Standard parameter class MyForm(QtWidgets.QDialog): def init (self,
parent = None): super().init(parent)

    self.ui = Ui_Dialog()
    self.ui.setupUi(self)

    #Connect button
```

```
self.ui.orderbtn.clicked.connect(self.order_menu)

def order_menu(self):
    result = 0
    price = {
        self.ui.pz:220,
        self.ui.hg:50,
        self.ui.ff:60,
        self.ui.sd:30,
        self.ui.oj:35,
        self.ui.bb:110
    }
    for chb, amnt in price.items():
        if chb.isChecked():
            result += amnt
    self.ui.result.setText(f"The Total Amount: R{result}") #Set text
    displays on the label

if name == "main": app = QtWidgets.QApplication(sys.argv) myapp =
MyForm() myapp.show() sys.exit(app.exec())
```

.....

The Client Registration App

Mbulelo's Client Registration System

CLIENT REGISTRATION SYSTEM

Company Name		Address Line 1	
VAT Number		Address Line 2	
Contact Person		City	
Company Email		Postal Code	
Contact Phone		Select Country	South Africa

SAVE

CLEAR

```
C:\Users\Lenovo-User\Desktop\client_registration_app\client_view.py - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help
database_setup.py main_app.py client_view.py

/
8 def view_all_clients():
9     """Connects to the database and prints all registered clients."""
10    conn = None
11
12    try:
13        conn = sqlite3.connect(DATABASE_NAME)
14        cursor = conn.cursor()
15
16        cursor.execute("SELECT * FROM clients")
17        clients = cursor.fetchall()
18
19        if clients:
20            print("\n--- Registered Clients ---\n")
21
22            # Header
23            print(
24                f"{'ID':<4} {'Company Name':<25} {'VAT Number':<15} "
25                f"{'Contact Person':<20} {'Email':<25} {'Phone':<15} "
26                f"{'City':<15} {'Country':<15}"
27            )
28            print("-" * 154)
29
30            # Data rows
31            for client in clients:
32                print(
33                    f"{client[0]:<4} "
34                    f"{client[1]:<25} "
35                    f"{client[2] if client[2] else 'N/A':<15} "
36                    f"{client[3]:<20} "
37                    f"{client[4]:<25} "
38                    f"{client[5]:<15} "
39                    f"{client[8]:<15} "
40                    f"{client[10]:<15}"
41                )
```

client_view.py

```
import sqlite3

DATABASE_NAME = "clients.db"

def view_all_clients(): """Connects to the database and prints all
registered clients.""" conn = None

try:
    conn = sqlite3.connect(DATABASE_NAME)
    cursor = conn.cursor()

    cursor.execute("SELECT * FROM clients")
    clients = cursor.fetchall()

    if clients:
        print("\n--- Registered Clients ---\n")

        # Header
        print(
            f"{'ID':<4} {'Company Name':<25} {'VAT Number':<15} "
            f"{'Contact Person':<20} {'Email':<25} {'Phone':<15} "
            f"{'City':<15} {'Country':<15}"
        )
        print("-" * 154)

        # Data rows
        for client in clients:
            print(
                f"{client[0]:<4} "
                f"{client[1]:<25} "
                f"{client[2] if client[2] else 'N/A':<15} "
                f"{client[3]:<20} "
                f"{client[4]:<25} "
                f"{client[5]:<15} "
                f"{client[8]:<15} "
                f"{client[10]:<15}"
            )

    else:
        print("No clients registered yet.")

except sqlite3.Error as e:
```

```

        print(f"Error viewing clients: {e}")

finally:
    if conn:
        conn.close()

if name == "main": view_all_clients()

```

The second part:

```

1  #Connector API
2  # This is the API connecting the database_setup (storage of data)...
3  #With the Ui.Dialog, whatever the user enters get stored on the databa
4
5  import sys
6  import sqlite3
7  from datetime import datetime
8
9  from PyQt6.QtWidgets import QApplication, QDialog, QMessageBox
10 from client_registration_sys import Ui_Dialog  # ✓ FIXED IMPORT
11
12 DATABASE_NAME = "clients.db"
13
14
15 class ClientRegistrationApp(QDialog):  # ✓ Use QDialog (matches UI)
16
17     def __init__(self):
18         super().__init__()
19
20         self.ui = Ui_Dialog()
21         self.ui.setupUi(self)
22         self.setWindowTitle("Mbulelo's Client Registration System")
23
24         # ✓ Connect buttons (fixed names)
25         self.ui.pushButtonSave.clicked.connect(self.save_client)
26         self.ui.pushButtonClear.clicked.connect(self.clear_form)
27
28         # Init DB
29         self.init_db_connection()
30
31     def init_db_connection(self):
32         """Initialize database connection"""
33         try:
34             self.conn = sqlite3.connect(DATABASE_NAME)

```

Main_app.py

#Connector API

This is the API connecting the database_setup (storage of data)...

#With the Ui.Dialog, whatever the user enters get stored on the database_setup because of the main_app.py API

```
import sys import sqlite3 from datetime import datetime
```

```
from PyQt6.QtWidgets import QApplication, QDialog, QMessageBox from  
client_registration_sys import Ui_Dialog #  FIXED IMPORT
```

```
DATABASE_NAME = "clients.db"
```

```
class ClientRegistrationApp(QDialog): #  Use QDialog (matches UI)
```

```
def __init__(self):  
    super().__init__()
```

```
    self.ui = Ui_Dialog()  
    self.ui.setupUi(self)  
    self.setWindowTitle("Mbulelo's Client Registration System")
```

```
    #  Connect buttons (fixed names)  
    self.ui.pushButtonSave.clicked.connect(self.save_client)  
    self.ui.pushButtonClear.clicked.connect(self.clear_form)
```

```
    # Init DB  
    self.init_db_connection()
```

```
def init_db_connection(self):  
    """Initialize database connection"""  
    try:  
        self.conn = sqlite3.connect(DATABASE_NAME)  
        self.cursor = self.conn.cursor()  
        print("Database connection established.")  
    except sqlite3.Error as e:  
        QMessageBox.critical(self, "Database Error", f"Could not  
connect: {e}")  
        sys.exit(1)
```



```

def save_client(self):
    """Save client to database"""

    company_name = self.ui.lineEditCompanyName.text().strip()
    vat_number = self.ui.lineEditVatNumber.text().strip()
    contact_person = self.ui.lineEditContactPerson.text().strip()
    contact_email = self.ui.lineEditContactEmail.text().strip()
    contact_phone = self.ui.lineEditContactPhone.text().strip()
    address_line1 = self.ui.lineEditAddress1.text().strip()
    address_line2 = self.ui.lineEditAddress2.text().strip()
    city = self.ui.lineEditCity.text().strip() #  FIXED (your UI
uses this for city)
    postal_code = self.ui.lineEditPostalCode.text().strip()
    country = self.ui.comboBoxCountry.currentText()
    registration_date = datetime.now().strftime("%Y-%m-%d")

    # Validation
    if not company_name or not contact_person or not contact_email or \
        not contact_phone or not address_line1 or not city or \
        not postal_code or not country:

        QMessageBox.warning(self, "Input Error", "Please fill in all
required fields.")
        return

    try:
        self.cursor.execute("""
            INSERT INTO clients (
                company_name, vat_number, contact_person,
contact_email,
                contact_phone, address_line1, address_line2, city,
                postal_code, country, registration_date
            ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
        """, (
            company_name,
            vat_number if vat_number else None,
            contact_person,
            contact_email,
            contact_phone,
            address_line1,
            address_line2 if address_line2 else None,
            city,
            postal_code,

```

```

        country,
        registration_date
    ))

    self.conn.commit()

    QMessageBox.information(self, "Success", f"{company_name}
saved successfully!")
    self.clear_form()

    except sqlite3.IntegrityError as e:
        if "UNIQUE constraint failed: clients.vat_number" in str(e):
            QMessageBox.critical(self, "Error", "VAT Number already
exists.")
        else:
            QMessageBox.critical(self, "Error", f"Integrity error:
{e}")

    except sqlite3.Error as e:
        QMessageBox.critical(self, "Error", f"Database error: {e}")

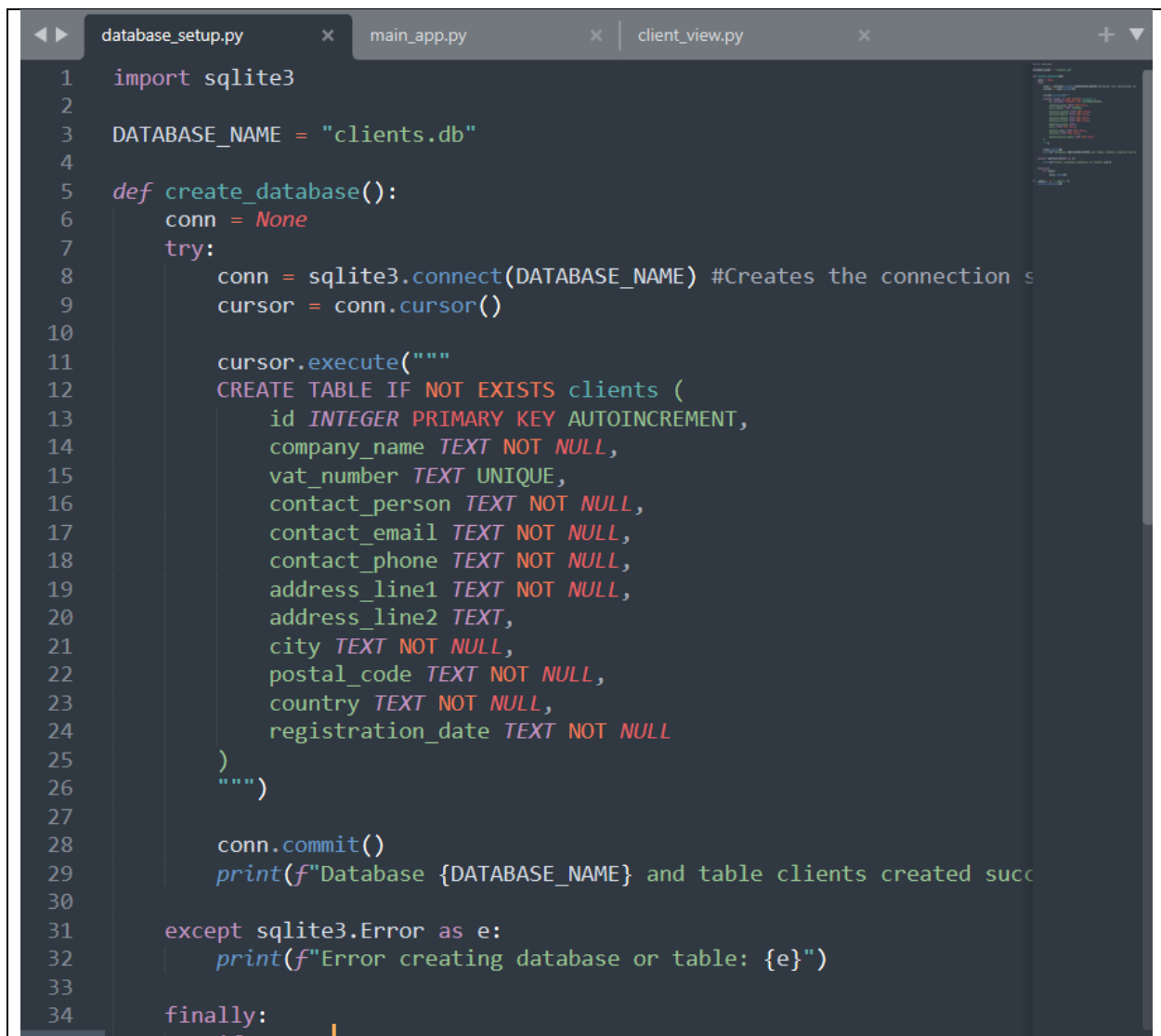
def clear_form(self):
    """Clear all inputs"""
    self.ui.lineEditCompanyName.clear()
    self.ui.lineEditVatNumber.clear()
    self.ui.lineEditContactPerson.clear()
    self.ui.lineEditContactEmail.clear()
    self.ui.lineEditContactPhone.clear()
    self.ui.lineEditAddress1.clear()
    self.ui.lineEditAddress2.clear()
    self.ui.lineEditCity.clear() #  city field
    self.ui.lineEditPostalCode.clear()
    self.ui.comboBoxCountry.setCurrentIndex(0)

def closeEvent(self, event):
    """Close DB on exit"""
    if hasattr(self, "conn"):
        self.conn.close()
        print("Database connection closed.")
    event.accept()

if name == "main": app = QApplication(sys.argv) window =
ClientRegistrationApp() window.show() sys.exit(app.exec())

```

The third part:



```
1 import sqlite3
2
3 DATABASE_NAME = "clients.db"
4
5 def create_database():
6     conn = None
7     try:
8         conn = sqlite3.connect(DATABASE_NAME) #Creates the connection string
9         cursor = conn.cursor()
10
11         cursor.execute("""
12 CREATE TABLE IF NOT EXISTS clients (
13     id INTEGER PRIMARY KEY AUTOINCREMENT,
14     company_name TEXT NOT NULL,
15     vat_number TEXT UNIQUE,
16     contact_person TEXT NOT NULL,
17     contact_email TEXT NOT NULL,
18     contact_phone TEXT NOT NULL,
19     address_line1 TEXT NOT NULL,
20     address_line2 TEXT,
21     city TEXT NOT NULL,
22     postal_code TEXT NOT NULL,
23     country TEXT NOT NULL,
24     registration_date TEXT NOT NULL
25 )
26 """)
27
28     conn.commit()
29     print(f"Database {DATABASE_NAME} and table clients created successfully")
30
31 except sqlite3.Error as e:
32     print(f"Error creating database or table: {e}")
33
34 finally:
35     if conn:
36         conn.close()
```

Database_setup.py

```
import sqlite3
```

```
DATABASE_NAME = "clients.db"
```

```
def create_database(): conn = None try: conn =
sqlite3.connect(DATABASE_NAME) #Creates the connection string cursor
= conn.cursor()
```

```
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS clients (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```

        company_name TEXT NOT NULL,
        vat_number TEXT UNIQUE,
        contact_person TEXT NOT NULL,
        contact_email TEXT NOT NULL,
        contact_phone TEXT NOT NULL,
        address_line1 TEXT NOT NULL,
        address_line2 TEXT,
        city TEXT NOT NULL,
        postal_code TEXT NOT NULL,
        country TEXT NOT NULL,
        registration_date TEXT NOT NULL
    )
    """

    conn.commit()
    print(f"Database {DATABASE_NAME} and table clients created
successfully.")

except sqlite3.Error as e:
    print(f"Error creating database or table: {e}")

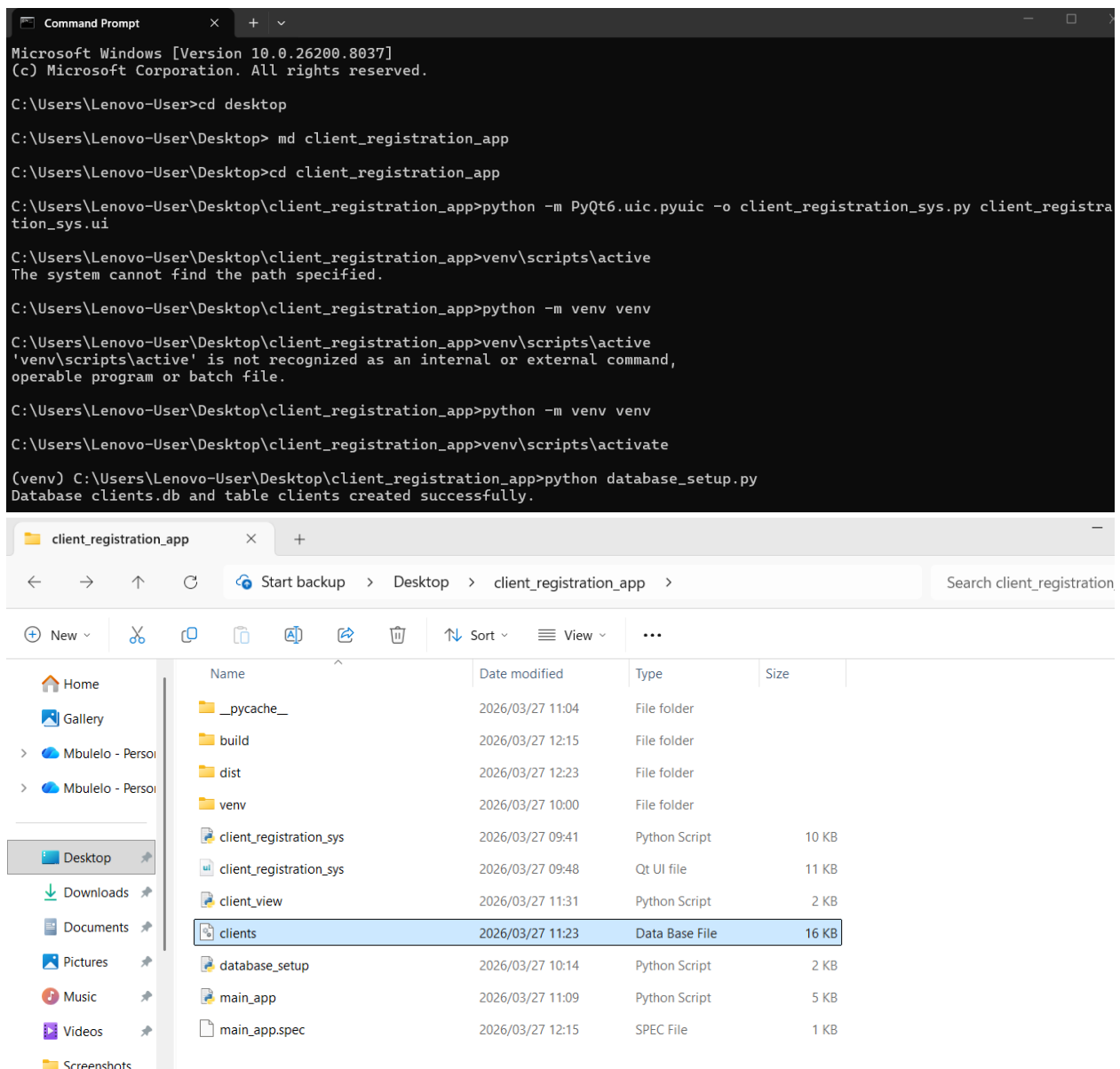
finally:
    if conn:
        conn.close()

if name == "main": create_database()

```

The command prompts:

1. To create the database file as a file we used this command:



From this point on we view the information entered into the app:

```
(venv) C:\Users\Lenovo-User\Desktop\client_registration_app>python client_view.py

--- Registered Clients ---
```

ID	Company Name	VAT Number	Contact Person	Email	Phone	City
1	SciThagorus South Africa	123321	Mbulelo Mtshu	mbulelo@scithagorus.com	0663330000	Johannesburg

On a separate note, we can create the executable version of the app still on the command prompt virtual environment (venv):

```
(venv) C:\Users\Lenovo-User\Desktop\client_registration_app>pip install pyinstaller
Collecting pyinstaller
  Downloading pyinstaller-6.19.0-py3-none-win_amd64.whl.metadata (8.5 kB)
Collecting altgraph (from pyinstaller)
  Downloading altgraph-0.17.5-py2.py3-none-any.whl.metadata (7.5 kB)
Collecting packaging>=22.0 (from pyinstaller)
  Using cached packaging-26.0-py3-none-any.whl.metadata (3.3 kB)
Collecting pefile>=2022.5.30 (from pyinstaller)
  Downloading pefile-2024.8.26-py3-none-any.whl.metadata (1.4 kB)
Collecting pyinstaller-hooks-contrib>=2026.0 (from pyinstaller)
  Downloading pyinstaller_hooks_contrib-2026.3-py3-none-any.whl.metadata (16 kB)
Collecting pywin32-ctypes>=0.2.1 (from pyinstaller)
  Downloading pywin32_ctypes-0.2.3-py3-none-any.whl.metadata (3.9 kB)
Collecting setuptools>=42.0.0 (from pyinstaller)
  Using cached setuptools-82.0.1-py3-none-any.whl.metadata (6.5 kB)
Downloading pyinstaller-6.19.0-py3-none-win_amd64.whl (1.4 MB)
1.4/1.4 MB 8.0 MB/s 0:00:00
```

pip install pyinstaller

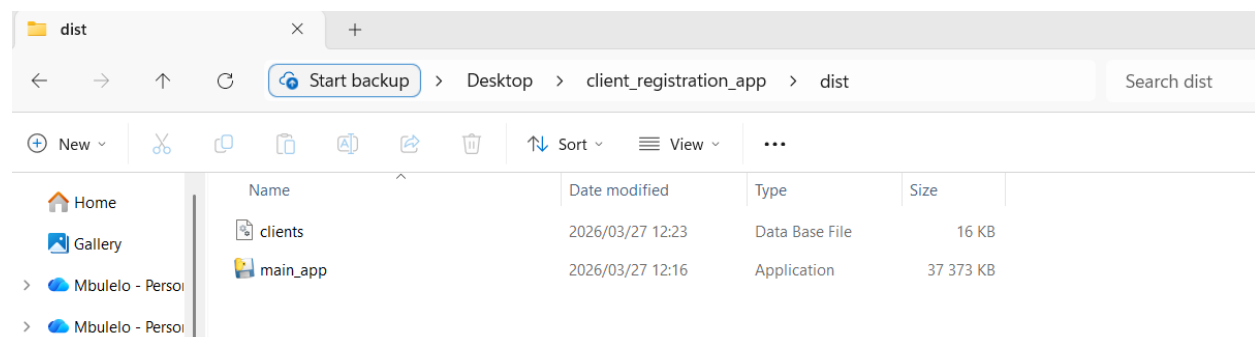
*

```
(venv) C:\Users\Lenovo-User\Desktop\client_registration_app>pyinstaller --onefile --windowed main_app.py
182 INFO: PyInstaller: 6.19.0, contrib hooks: 2026.3
183 INFO: Python: 3.14.3
211 INFO: Platform: Windows-11-10.0.26200-SP0
211 INFO: Python environment: C:\Users\Lenovo-User\Desktop\client_registration_app\venv
212 INFO: wrote C:\Users\Lenovo-User\Desktop\client_registration_app\main_app.spec
229 INFO: Module search paths (PYTHONPATH):
['C:\\Users\\Lenovo-User\\Desktop\\client_registration_app\\venv\\Scripts\\pyinstaller.exe',
'C:\\Users\\Lenovo-User\\AppData\\Local\\Python\\pythoncore-3.14-64\\python314.zip',
'C:\\Users\\Lenovo-User\\AppData\\Local\\Python\\pythoncore-3.14-64\\Python314\\Lib\\site-packages']
```

* pyinstaller --onefile --windowed main_app.py

```
46191 INFO: Copying bootloader EXE to C:\Users\Lenovo-User\Desktop\client_registration_app\dist\main_app.exe
46332 INFO: Copying icon to EXE
46411 INFO: Copying 0 resources to EXE
46411 INFO: Embedding manifest in EXE
46474 INFO: Appending PKG archive to EXE
46570 INFO: Fixing EXE headers
49623 INFO: Building EXE from EXE-00.toc completed successfully.
49628 INFO: Build complete! The results are available in: C:\Users\Lenovo-User\Desktop\client_registration_app\dist
(venv) C:\Users\Lenovo-User\Desktop\client_registration_app>C:\Users\Lenovo-User\Desktop\client_registration_app\clients.db
```

This is showing us where the new app is installed:



The database file is copied into the folder as well.

